

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CIÊNCIAS DA COMPUTAÇÃO

João Paulo Araujo Bonomo

Deteccção de Ataques em Redes Intraveiculares CAN com Técnicas de Machine Learning

Florianópolis
15 de julho de 2025

João Paulo Araujo Bonomo

Detecção de Ataques em Redes Intraveiculares CAN com Técnicas de Machine Learning

Trabalho de Conclusão de Curso submetido ao Curso de Graduação em Ciências da Computação do Centro Tecnológico da Universidade Federal de Santa Catarina como requisito para obtenção do título de Bacharel em Ciências da Computação.

Orientador: Prof. Giovani Gracioli, Dr.

Coorientador: Prof. Jônata Tyska Carvalho, Dr.

Florianópolis
15 de julho de 2025

Ficha catalográfica gerada por meio de sistema automatizado gerenciado pela BU/UFSC.
Dados inseridos pelo próprio autor.

Araujo Bonomo, João Paulo

Detecção de Ataques em Redes Intraveiculares CAN com
Técnicas de Machine Learning / João Paulo Araujo Bonomo ;
orientador, Giovani Gracioli, coorientador, Jônata Tyska
Carvalho, 2025.

132 p.

Trabalho de Conclusão de Curso (graduação) -
Universidade Federal de Santa Catarina, Centro Tecnológico,
Graduação em Ciências da Computação, Florianópolis, 2025.

Inclui referências.

1. Ciências da Computação. 2. Machine Learning. 3.
Segurança. 4. Redes veiculares. 5. Protocolo CAN. I.
Gracioli, Giovani. II. Tyska Carvalho, Jônata. III.
Universidade Federal de Santa Catarina. Graduação em
Ciências da Computação. IV. Título.

João Paulo Araujo Bonomo
Detecção de Ataques em Redes Intraveiculares CAN com Técnicas de Machine Learning

Este Trabalho de Conclusão de Curso foi julgado adequado para obtenção do Título de Bacharel em Ciências da Computação e aprovado em sua forma final pelo curso de Graduação em Ciências da Computação.

Florianópolis, 15 de julho de 2025.

Banca Examinadora:

Prof. Giovani Gracioli, Dr.
Orientador
Universidade Federal de Santa Catarina

Prof. Jônata Tyska Carvalho, Dr.
Coorientador
Universidade Federal de Santa Catarina

Prof. Antonio Augusto Medeiros Fröhlich, Dr.
Avaliador
Universidade Federal de Santa Catarina

Prof. Eduardo Camilo Inacio, Dr.
Avaliador
Universidade Federal de Santa Catarina

A Deus,
aos meus pais, Helaine e Robson,
ao meu irmão Lucas,
à minha amada Ana.

AGRADECIMENTOS

Agradeço acima de tudo a Deus, por me acompanhar ao lado durante toda a minha vida, e me carregar no colo nos momentos de fraqueza. Agradeço pela iluminação durante o desenvolvimento e escrita deste trabalho, sempre após a oração de São Tomás.

Aos meus pais, Helaine e Robson, que jamais mediram esforços para que seus filhos estudassem. Por cada conselho, incentivo, suporte e puxão de orelha (todos merecidos). À minha mãe, exemplo de amor, zelo e dedicação às pessoas ao seu redor, sempre se colocando em última da fila, no melhor "estilo mineirinho". Ao meu pai, exemplo de homem, pai, trabalhador e acadêmico, que nunca deixou que o título de doutor estragasse sua simplicidade. Ao meu irmão mais velho Lucas, por me acolher em Florianópolis, quando tive a oportunidade de conhecer melhor essa figura antes distante, mas sempre tão inspiradora em minha infância. À minha tia Penha, a tia que sempre foi a vó, a segunda mãe, a irmã mais velha.

À minha amada Ana Klara, a melhor escolha que fiz em minha vida, mesmo que ainda tão novo. Nunca questionou minha decisão de me mudar para tão longe em busca do meu sonho, apenas me apoiou e amou, como sempre fez em tudo. Sempre que pensava em desistir, era meu motivo de levantar e continuar. Obrigado por todo companheirismo, mesmo nas vezes em que só pude te oferecer assistir "Rocky" em chamada de vídeo.

Ao meu orientador Prof. Giovani Gracioli, por me aceitar ainda na terceira fase no LISHA, por me inserir no projeto do IASE, uma grande experiência em minha vida pessoal, profissional e acadêmica. Também por todos os ensinamentos, pela orientação e, claro, pelos churrascos ao longo dos últimos três anos.

Ao meu coorientador Prof. Jônata Tyska, pelos ensinamentos em aula, de POO2 a Data Mining, e pelo suporte burocrático e administrativo neste trabalho.

Aos "irmãos de guerra" que a graduação e o LISHA me deram: João Victor Volpato, Rodrigo de Carvalho (RDG) e Thiago Bewiahn (Alemão). Pela amizade, pela troca de conhecimento, pelas idas ao bar e pelas conversas — proveitosas ou não. Vocês foram minha família em Floripa, seja na Páscoa, no Dia das Mães, na escrita deste TCC ou em SO2. *"Quem estará nas trincheiras ao teu lado? E isso importa? Mais do que a própria guerra".*

Aos amigos que a UFSC me deu, Rafael Vargas e sua esposa Akemi, Fillipi Mangrich, Leonardo Lima, Arthur Scarpatto, Rafael Oliveira e Heitor Tosta (o calouro), pela amizade, pelo conhecimento compartilhado e, principalmente, por tornarem essa jornada mais leve.

Aos amigos do LISHA, em especial Jozimar Custódio, Josafat (des)Leal e Matheus Wagner, pelos conhecimentos passados, almoços no RU, conversas no ME até de madrugada, críticas e avaliações duras, mas sempre honestas. Aprendi muito com vocês e sou grato por isso. OBS: ao "Jozigol", por nunca negar um futsal às 22h de uma terça-feira.

A Lucas Schatz, chefe que considero um amigo, pela primeira oportunidade de estágio, pela confiança no meu trabalho a ponto de me efetivar ainda durante minha graduação, mas principalmente por o todo conhecimento passado. Pela paciência e respeito durante minha graduação e principalmente durante o desenvolvimento deste trabalho, dando tempo ao tempo.

Aos colegas/amigos de SRS, em especial Caio Prá, Ana Paula Althoff e Erik Orsolin, por tornarem o trabalho melhor, por serem profissionais inspiradores, pela amizade e pelo café no escritório. Me orgulho de trabalhar com pessoas como vocês.

Aos Professores do curso de Ciências da Computação da UFSC que trabalham e prezam para que a UFSC mereça o prestígio que tem. Em especial, aos professores Guto Fröhlich, Luiz Claudio dos Santos, Odorico Mendizabal, Renato Fileto, Jean Martina e Aldo Von Wangenheim. Foi em busca do conhecimento que tais professores têm e transmitem que vim à UFSC. A estes mestres, e a outros que não foram citados nominalmente: é uma honra poder dizer que fui vosso aluno.

Por fim, agradeço ao CNPq pela concessão da bolsa PIBIC que permitiu a realização do projeto de pesquisa que culminou neste TCC.

“Creator ineffabilis, qui de thesauris sapientiae tuae tres Angelorum hierarchias designasti et eas super caelum empyreum miro ordine collocasti atque universi partes elegantissime distribuisti: tu, inquam, qui verus fons luminis et sapientiae diceris ac supereminens principium, infundere digneris super intellectus mei tenebras tuae radium claritatis, duplices, in quibus natus sum, a me removens tenebras, peccatum scilicet et ignorantiam. Tu, qui linguas infantium facis disertas, linguam meam erudias atque in labiis meis gratiam tuae benedictionis infundas. Da mihi intelligendi acumen, retinendi capacitatem, addiscendi modum et facilitatem, interpretandi subtilitatem, loquendi gratiam copiosam. Ingressum instruas, progressum dirigas, egressum compleas. Tu, qui es verus Deus et homo, qui vivis et regnas in saecula saeculorum.
Amen.”

(SÃO TOMÁS DE AQUINO, Creator Ineffabilis ou "Oração de São Tomás de Aquino antes do estudo", século XIII)

RESUMO

Veículos modernos, especialmente os elétricos e autônomos, dependem cada vez mais de componentes eletrônicos interconectados pelo barramento CAN, que, apesar de eficiente, apresenta vulnerabilidades de segurança. Essas fragilidades expõem o sistema a ataques de injeção de mensagens maliciosas, capazes de comprometer o controle do veículo e a integridade de seus componentes, colocando em risco a segurança dos passageiros e de terceiros. Este trabalho investiga o uso de algoritmos de *Machine Learning* (ML) para detectar ataques à rede CAN dos tipos DoS e *Impersonation*. Os modelos foram treinados e testados com dados públicos de um veículo real em condições normais e sob ataque, e posteriormente validados com *datasets* distintos, oriundos de outro fabricante, disponibilizados pelo LISHA-UFSC, contendo ataques sintetizados. Essa validação buscou demonstrar a capacidade de generalização dos modelos para diferentes contextos automotivos. Os resultados apresentaram até 100% de precisão e sensibilidade, superando trabalhos anteriores. Além disso, os modelos foram embarcados em duas plataformas de arquiteturas distintas, RISC-V e ARM, apresentando tempos de execução da ordem de 1,2 milissegundos, compatíveis com a periodicidade das mensagens CAN, evidenciando a viabilidade de um IDS eficiente e em tempo real para proteção veicular contra ataques cibernéticos.

Palavras-chave: CAN. *Machine Learning* (ML). Redes intraveiculares (IVNs). ECU. Ataque DoS. Ataque de *Impersonation*. Sistema de Detecção de Intrusão(IDS).

ABSTRACT

Modern vehicles, especially electric and autonomous ones, increasingly rely on interconnected electronic components via the CAN bus, which, despite its efficiency, presents critical security vulnerabilities. These weaknesses expose the system to malicious message injection attacks that can compromise vehicle control and component integrity, posing risks to passenger safety and third parties. This work investigates the use of Machine Learning (ML) algorithms to detect DoS and Impersonation attacks on CAN networks. The models were trained and tested using public datasets from a real vehicle operating under both normal and attack conditions, and later validated on distinct datasets from another manufacturer, provided by LISHA-UFSC, which included synthesized attacks. This validation aimed to demonstrate the models' ability to generalize across different automotive contexts. The results achieved up to 100% accuracy and sensitivity, surpassing previous studies. Additionally, the models were deployed on two embedded platforms with different architectures, RISC-V and ARM, achieving execution times on the order of 1.2 milliseconds, which is compatible with CAN message periodicity. These findings confirm the feasibility of an efficient, near real-time IDS for protecting vehicles against cyber-attacks.

Keywords: CAN. Machine Learning (ML). In-Vehicle Networks (IVNs). ECU. DoS attack. Impersonation attack. Intrusion Detection System (IDS).

LISTA DE FIGURAS

Figura 1 – Arquitetura de um veículo protegido por um IDS. Fonte: (YANG; MOUBAYED; SHAMI, 2021).	25
Figura 2 – Estrutura de mensagens CAN. Fonte: (LEE; JEONG; KIM, 2017).	28
Figura 3 – Barramento de rede CAN em um veículo inteligente. Fonte: (ZHOU; LI; SHEN, 2019).	29
Figura 4 – Dois tipos de cenário de ataques em um barramento CAN. Fonte: (LEE; JEONG; KIM, 2017).	30
Figura 5 – Fluxo de trabalho comum em aprendizado de máquina baseado na metodologia CRISP-DM. Fonte: Adaptado de (HARRISON, 2019).	33
Figura 6 – Ilustração dos elementos de uma Árvore de Decisão. Fonte: Elaboração própria.	35
Figura 7 – Exemplo visual do algoritmo KNN aplicado a um problema de classificação. Fonte: Elaboração própria.	37
Figura 8 – Célula básica de uma rede neural de <i>Long Short-Term Memory</i> (LSTM). Fonte: (BARBOSA et al., 2021).	39
Figura 9 – Representação de uma matriz de confusão. Fonte: Elaboração própria.	40
Figura 10 – Visão geral do hardware de aquisição de dados do projeto IASE. Fonte: (BEDRETSCHUK et al., 2023).	47
Figura 11 – Ilustração do processo de <i>feature engineering</i> realizado nos conjuntos de dados utilizados. Fonte: Elaboração própria.	48
Figura 12 – Curvas de aprendizado do modelo XGBoost para o conjunto de dados de ataques de <i>Impersonation</i>.	58
Figura 13 – Tempo médio de classificação de uma mensagem como normal ou ataque DoS em milissegundos (ms) para as diferentes configurações da LSTM. A legenda do gráfico ressalta a periodicidade do barramento CAN de 1 ms.	61
Figura 14 – Tempo médio de classificação de uma mensagem como normal ou ataque de <i>Impersonation</i> em milissegundos (ms) para as diferentes configurações da LSTM. A legenda do gráfico ressalta a periodicidade do barramento CAN de 1 ms.	61
Figura 15 – Tempo médio de classificação de uma mensagem como normal ou ataque DoS em milissegundos (ms) na SBC VisionFive 2. A legenda da figura descreve quantas instâncias foram fornecidas ao modelo.	63
Figura 16 – Tempo médio de classificação de uma mensagem como normal ou ataque de <i>impersonation</i> em milissegundos (ms) na SBC VisionFive 2. A legenda da figura descreve quantas instâncias foram fornecidas ao modelo.	63

Figura 17 – Tempo médio de classificação de uma mensagem como normal ou ataque DoS em milissegundos (ms) na Raspberry Pi 3 Model B. A legenda da figura descreve quantas instâncias foram fornecidas ao modelo. . . .	65
Figura 18 – Tempo médio de classificação de uma mensagem como normal ou ataque de <i>Impersonation</i> em milissegundos (ms) na Raspberry Pi 3 Model B. A legenda da figura descreve quantas instâncias foram fornecidas ao modelo.	66
Figura 19 – Organização interna da pasta <i>dataset/</i> do repositório do trabalho no GitHub.	77
Figura 20 – Organização interna da pasta <i>results/</i> do repositório do trabalho no GitHub.	78
Figura 21 – Organização interna da pasta <i>src/</i> do repositório do trabalho no GitHub.	78

LISTA DE TABELAS

Tabela 1	– Descrição do conjunto de dados publicado pelo HCRL.	46
Tabela 2	– Comparação dos resultados de <i>F1-score</i> para detecção de ataques de DoS. Trabalhos utilizados para comparação: A (ALSHAMMARI et al., 2018), B (GUNDU; MALEKI, 2022), C (PUROHIT; GOVINDARASU, 2022), D (ANJUM et al., 2022).	55
Tabela 3	– Resultados de detecção de ataques de DoS para a rede LSTM com diferentes valores para o parâmetro <i>time_steps</i>	55
Tabela 4	– Comparação dos resultados de <i>F1-score</i> para detecção de ataques de <i>Impersonation</i> . A: Este Trabalho (2025); B: (ALSHAMMARI et al., 2018); C: (GUNDU; MALEKI, 2022); D: (PUROHIT; GOVINDARASU, 2022); E: (ANJUM et al., 2022).	57
Tabela 5	– Resultados de detecção de ataques de <i>Impersonation</i> para a rede LSTM com diferentes valores de <i>time_steps</i>	57
Tabela 6	– Tempo, em segundos (s), de treino e teste combinados de cada algoritmo para os <i>datasets</i> do HCRL de ataques dos tipos DoS e <i>Impersonation</i> . . .	59
Tabela 7	– Comparação do tempo, em segundos (s), de treino e teste combinados de cada algoritmo para os <i>datasets</i> do HCRL de ataques dos tipos DoS e <i>Impersonation</i> nas duas plataformas embarcadas selecionadas.	59
Tabela 8	– Comparação dos tempos de treino de cada configuração de <i>time_steps</i> usada no treinamento da rede LSTM com os <i>datasets</i> do HCRL de ataques dos tipos DoS e <i>Impersonation</i> , com e sem a <i>feature</i> de identificação das ECUs, medidos em segundos (s).	60
Tabela 9	– Tempo de classificação embarcada na VisionFive 2 em milissegundos (ms).	64
Tabela 10	– Tempo médio de classificação embarcada na Raspberry Pi 3 Model B em milissegundos (ms).	65
Tabela 11	– Especificações da VisionFive 2. Adaptado e traduzido do site oficial da fabricante StarFive (STARFIVE, 2024).	73
Tabela 12	– Especificações da Raspberry Pi 3 Model B. Informações adaptadas e traduzidas do site oficial da Raspberry Pi Foundation (FOUNDATION, 2025).	75

LISTA DE ABREVIACÕES

CAN	<i>Controller Area Network</i>
CAN-bus	<i>Control Network Area Bus</i> (Barramento CAN)
CRISP-DM	<i>Cross Industry Standard Process for Data Mining</i>
DT	<i>Decision Tree</i> (Árvore de Decisão)
DoS	<i>Denial of Service</i> (Negação de Serviço)
ECU	<i>Electronic Control Unit</i> (Unidade de Controle Eletrônico)
HCRL	<i>Hacking and Countermeasure Research Lab</i>
IA	Inteligência Artificial
IASE	<i>Intelligent Acquisition and Analysis System for ECUs</i>
IDS	<i>Intrusion Detection System</i> (Sistema de Detecção de Intrusão)
IVN	<i>Intra-Vehicle Network</i> (Rede Intraveicular)
KNN	<i>K-Nearest Neighbors</i> (K Vizinhos Mais Próximos)
LISHA	Laboratório de Integração de Software e Hardware
LSTM	<i>Long Short-Term Memory</i> (Memória Longa de Curto Prazo)
ML	<i>Machine Learning</i> (Aprendizado de Máquina)
RF	<i>Random Forest</i> (Floresta Aleatória)
RNN	<i>Recurrent Neural Network</i> (Rede Neural Recorrente)
SBC	<i>Single Board Computer</i> (Computador de Placa Única)
TCC	Trabalho de Conclusão de Curso

SUMÁRIO

	Lista de Figuras	15
	Lista de tabelas	17
	Lista de Abreviações	19
1	INTRODUÇÃO	23
1.1	MOTIVAÇÃO	23
1.2	OBJETIVOS	25
1.2.1	Objetivos Específicos	25
1.3	ORGANIZAÇÃO DO TEXTO	26
2	REVISÃO BIBLIOGRÁFICA	27
2.1	FUNDAMENTAÇÃO TEÓRICA	27
2.1.1	Ataques ao Barramento CAN	28
2.1.2	Ataque de DoS	30
2.1.3	Ataque de <i>Impersonation</i>	30
2.1.4	Sistemas de Detecção de Intrusão (IDS)	31
2.1.5	<i>Machine Learning</i> (ML)	31
2.1.5.1	<i>Cross Industry Standard Process for Data Mining (CRISP-DM)</i>	32
2.1.5.2	<i>Entendimento e Tratamento dos Dados</i>	32
2.1.5.3	<i>Aprendizado Supervisionado e Não Supervisionado</i>	34
2.1.5.4	<i>Decision Tree (Árvore de Decisão)</i>	34
2.1.5.5	<i>Random Forest (Floresta Aleatória)</i>	35
2.1.5.6	<i>K-Nearest Neighbors (KNN)</i>	36
2.1.5.7	<i>XGBoost</i>	36
2.1.5.8	<i>K-Means</i>	37
2.1.5.9	<i>Long Short-Term Memory (LSTM)</i>	38
2.1.5.10	<i>Validação</i>	39
2.1.5.10.1	<i>Validação Cruzada (Cross-Validation)</i>	40
2.1.6	Navalha de Ockham	41
2.2	TRABALHOS CORRELATOS	41
3	MATERIAIS E MÉTODOS	45
3.1	ENTENDIMENTO E PREPARAÇÃO DOS DADOS	45
3.1.1	HCRL <i>Dataset</i>	45
3.1.2	IASE <i>Dataset</i>	46
3.1.3	Preparação dos Dados	47
3.2	MODELAGEM	48

3.2.1	Classificadores	49
3.2.2	Clusterizador	49
3.2.3	Rede Neural Recorrente (RNN)	49
3.3	PLATAFORMAS EMBARCADAS	50
3.3.1	VisionFive 2	50
3.3.2	Raspberry Pi 3 Model B	51
4	AVALIAÇÃO	53
4.1	MÉTRICAS E MÉTODOS DE AVALIAÇÃO E VALIDAÇÃO	53
4.1.1	Validação de Generalização dos Modelos	53
4.2	AVALIAÇÃO DE DESEMPENHO	54
4.2.1	<i>DoS Attack</i>	54
4.2.1.1	<i>Generalização</i>	56
4.2.2	<i>Impersonation Attack</i>	56
4.2.2.1	<i>Generalização</i>	57
4.3	AVALIAÇÃO DE TEMPO DE EXECUÇÃO	58
4.3.1	Tempo de Treino e Teste	59
4.3.2	Avaliação do tempo de treino, teste e classificação da LSTM	60
4.3.3	Tempo de Execução Embarcada	62
4.3.3.1	<i>VisionFive 2</i>	62
4.3.3.2	<i>Raspberry Pi 3 Model B</i>	64
4.4	DISCUSSÃO DOS RESULTADOS	65
5	CONCLUSÃO	67
5.1	TRABALHOS FUTUROS	68
	REFERÊNCIAS	69
A	ESPECIFICAÇÃO DE HARDWARE - VISIONFIVE 2	73
B	ESPECIFICAÇÃO DE HARDWARE - RASPBERRY PI 3 MODEL B	75
C	CÓDIGO-FONTE	77
C.1	SRC/	79
C.2	SRC/CLASSIFIERS/	79
C.3	SRC/DATA_PROCESSING/	96
C.4	SRC/EXPERIMENTS/	100
C.5	SRC/INJECTORS/	106
C.6	SRC/PLOTS/	106
C.7	SRC/RNN/	112
D	ARTIGO NO PADRÃO SBC	119

1 INTRODUÇÃO

Nas últimas quatro décadas, a presença de componentes eletrônicos em veículos automotores evoluiu significativamente, especialmente em veículos elétricos e autônomos. Segundo análise realizada pela empresa norte-americana Deloitte, a parte eletrônica de um veículo nos anos 2000 era responsável por apenas 18% do custo de fabricação de um carro. O valor saltou para 40% em 2020, havendo previsões de que esse percentual atingiria 45% já em 2030 (TINGWALL, 2020). Esse avanço foi impulsionado, entre outros fatores, pelo surgimento do protocolo Controller Area Network (CAN) na indústria automotiva europeia durante a segunda metade dos anos 80.

Atualmente, veículos modernos são sistemas altamente sofisticados, integrando inúmeras Unidades de Controle Eletrônico (Electronic Control Units - ECUs) para gerenciar diversas funções. Levantamentos apontam que modelos de entrada possuem em torno de 70 ECUs (SHUKLA, 2015). Já modelos de ponta podem conter até 150 ECUs, enquanto veículos elétricos e autônomos chegam a integrar cerca de 200 unidades. (PRASHANTH, 2019).

Essas ECUs são interligadas em uma rede complexa que suporta funções essenciais, como diagnósticos, sistemas de infoentretenimento e medidas de segurança. O aumento exponencial no número de ECUs reflete diretamente a evolução das capacidades dos veículos, especialmente no contexto de uma indústria que avança em direção a automóveis mais inteligentes e autônomos. Para garantir essa integração, as ECUs se comunicam por meio de um barramento baseado no protocolo CAN.

O CAN é um protocolo de rede serial, originalmente projetado para a indústria automotiva, especialmente para carros europeus, embora este protocolo também tenha se tornado popular na automação industrial e outras aplicações. O barramento CAN é uma tecnologia de rede que garante comunicação rápida entre microcontroladores para sistemas de tempo real, eliminando a necessidade da indústria utilizar tecnologias muito mais caras e complexas como uma *Dual-Ported RAM* (VOSS, 2008).

1.1 MOTIVAÇÃO

Apesar de a simplicidade do barramento CAN possibilitar a comunicação rápida e barata em redes veiculares, ela traz alguns problemas também, especialmente relacionados à segurança do sistema. O CAN é um protocolo de baixo nível e não oferece suporte a nenhum recurso de segurança intrinsecamente, o que torna a rede vulnerável a ataques de diversas naturezas, como o corrompimento de uma ECU conectada ao barramento ou a intrusão de um dispositivo externo na rede do veículo.

É muito importante que os mecanismos de segurança de um sistema - como uma rede veicular - sejam projetados de forma a impedir o acesso não autorizado aos recursos e dados do sistema. No entanto, prevenir completamente as violações de segurança parece, no momento, irrealista. Podemos, entretanto, tentar detectar essas tentativas de intrusão para que ações possam

ser tomadas para prevenir ou reparar os danos. Este campo de pesquisa é chamado *Intrusion Detection* (Detecção de Intrusões) (SUNDARAM, 1996). Como o protocolo CAN não fornece nenhum sistema de detecção de intrusão próprio, há uma tendência atual em universidades e na indústria automotiva de desenvolver Sistemas de Detecção de Intrusão (IDS) para solucionar essa falha de segurança intrínseca do protocolo CAN.

Um caso muito popular na última década foi publicado pela WIRED (GREENBERG, 2015). Dois *hackers* que realizaram ataques ao sistema *Uconnect* de um carro da Jeep, o que levou ao *recall* de aproximadamente 1,4 milhão de veículos. Ao explorar as vulnerabilidades do automóvel, a dupla pôde obter controle remoto para manipular e comprometer o veículo, alterando sua aceleração e direção do volante com um notebook.

Outro exemplo mais recente foi o roubo de uma Toyota RAV4, usado como exemplo pelo Dr. Ken Tindell, CTO da *Canis Automotive Labs*, em um artigo publicado em seu blog. Tindell revelou técnicas de roubo de carros, usadas no caso da RAV4, baseadas em injeções de mensagens maliciosas na rede CAN interna do veículo e que dispensavam a necessidade de ter a chave do carro ou do veículo estar aberto (TINDELL, 2023).

Atualmente, os IDS baseados em modelos de *Machine Learning* (ML) têm se destacado pela capacidade de identificar mensagens anômalas em conjuntos de dados que contenham mensagens CAN com conteúdos falsos e maliciosos injetados. Esses modelos oferecem uma alternativa simples e econômica para a detecção de ataques em redes intraveiculares baseadas no protocolo CAN.

A Figura 1 mostra a arquitetura de um IDS para proteger a rede intraveicular (IVN) de um carro. Para proteger IVNs, IDSs podem ser colocados no topo do barramento CAN para identificar mensagens CAN maliciosas. Embora não seja o escopo deste trabalho, a Figura 1 ainda mostra como IDSs também podem ser incorporados aos *gateways*, os quais atuam como pontos de interligação entre diferentes redes do veículo, para detectar pacotes maliciosos provenientes de fontes externas.

A Figura 1 apresenta a arquitetura de um veículo protegido por um IDS. O diagrama mostra duas camadas principais de proteção: a rede intra-veicular (*Intra-Vehicle Network* - IVN) e a rede externa (*External Network*). Na IVN, o IDS é integrado diretamente à rede CAN, onde monitora as mensagens trafegadas entre os nós eletrônicos (ECUs). O sistema utiliza *network taps* para capturar o tráfego do barramento CAN em tempo real. Quando mensagens maliciosas são detectadas, alarmes são disparados. O acesso externo ao veículo pode ocorrer via a porta OBD-II, por onde um atacante pode tentar injetar mensagens CAN maliciosas (YANG; MOUBAYED; SHAMI, 2021).

Diversos trabalhos exploram diferentes tipos de ataque possíveis a redes veiculares (PALANCA et al., 2017; SHUKLA, 2015; TINDELL, 2023), assim como diversas formas de detectar e impedir esses ataques, com IDS baseados em modelos de ML (ALSHAMMARI et al., 2018; ANJUM et al., 2022; GUNDU; MALEKI, 2022) ou em outras técnicas, como análise de tensão (YIN et al., 2023) e periodicidade das mensagens no barramento (LEE; JEONG; KIM, 2017). Entretanto, poucos trabalhos na literatura avaliam e comparam o desempenho e a via-

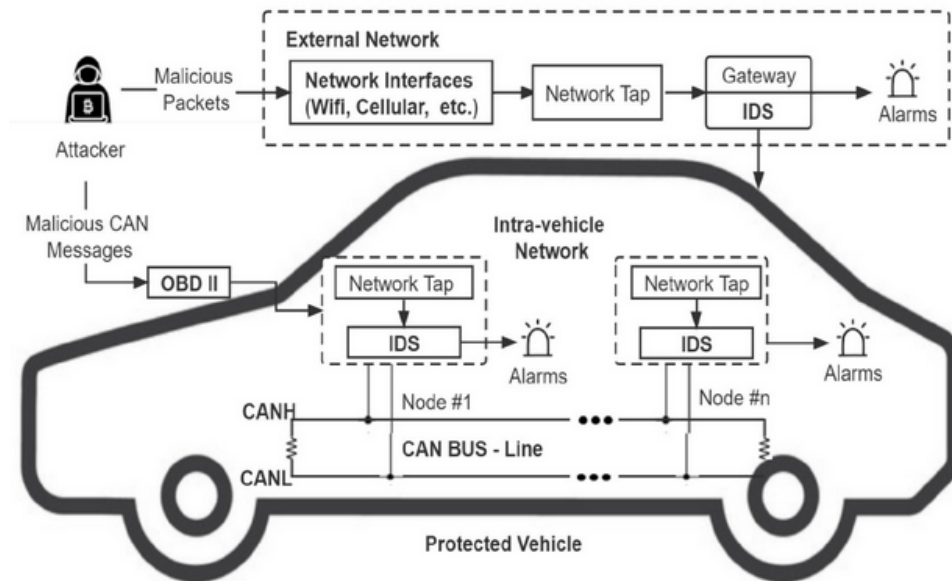


Figura 1 – Arquitetura de um veículo protegido por um IDS. Fonte: (YANG; MOUBAYED; SHAMI, 2021).

bilidade dos modelos propostos em sistemas embarcados, com poder computacional reduzido. Além disso, poucos trabalhos utilizam mais de um conjunto de dados em seu teste e validação, o que dificulta a viabilização de um sistema de detecção de intrusão que seja aplicável a contextos de redes e carros diferentes. Este trabalho busca contribuir para esse cenário por meio da avaliação de modelos de ML em contextos com restrição de recursos computacionais, utilizando diferentes conjuntos de dados e duas plataformas embarcadas distintas.

1.2 OBJETIVOS

O objetivo geral deste trabalho é investigar e comparar o desempenho de diferentes algoritmos de aprendizado de máquina (ML) aplicados à detecção de intrusões em redes veiculares baseadas no protocolo CAN, considerando a viabilidade de sua aplicação em sistemas embarcados com recursos computacionais limitados. A ênfase no contexto automotivo se justifica pelo fato de que, em 2008, a indústria de veículos já representava aproximadamente 80% do mercado de controladores CAN e microcontroladores com suporte nativo a esse protocolo (VOSS, 2008).

1.2.1 Objetivos Específicos

A fim de alcançar o objetivo geral deste trabalho, foram definidos os seguintes objetivos específicos:

- Estudar os conceitos do protocolo CAN, de IDS e de algoritmos de ML;
- Realizar uma revisão da literatura para compreender o problema, identificar os algoritmos e os conjuntos de dados a serem utilizados;

- Desenvolver código para treinamento, validação e teste dos modelos;
- Avaliar o desempenho dos modelos propostos;
- Verificar a viabilidade de empregar o sistema desenvolvido em um ambiente real, através da avaliação do tempo de execução da classificação de ataques em duas plataformas embarcadas.

1.3 ORGANIZAÇÃO DO TEXTO

O restante deste documento está organizado da seguinte forma. O Capítulo 2 é destinado a apresentar a fundamentação teórica necessária para a realização deste trabalho, bem como desenvolver uma análise aprofundada dos trabalhos correlatos. No Capítulo 3, são vistos os materiais e métodos utilizados para a realização deste trabalho, como os datasets e algoritmos selecionados. Já no Capítulo 4, os resultados alcançados são apresentados, bem como suas respectivas métricas e uma análise a respeito dos experimentos realizados. Por fim, no Capítulo 5 são apresentadas as conclusões e as sugestões de trabalhos futuros.

Após o capítulo de Conclusão, estão listadas as Referências e os Apêndices - como o código-fonte, especificações de hardware e um artigo no formato da Sociedade Brasileira de Computação - deste Trabalho de Conclusão de Curso (TCC).

2 REVISÃO BIBLIOGRÁFICA

Este capítulo está subdividido em 2 seções. A Seção 2.1 fornece definições importantes para este trabalho, incluindo a descrição do protocolo CAN e dos tipos de mensagens CAN. Além disso, a Subseção 2.1.1 apresenta uma visão geral sobre os tipos de ataque mais comuns ao barramento veicular, enquanto as Subseções 2.1.2 e 2.1.3 descrevem os dois ataques explorados neste trabalho. Por fim, a Subseção 2.1.4 aborda definições sobre IDSs e a Subseção 2.1.5 trata de conceitos de ML fundamentais para este trabalho. A Seção 2.2 aborda os principais trabalhos correlatos que serviram de base de estudo para esta escrita, além de alguns serem usados como comparativos para seus resultados.

2.1 FUNDAMENTAÇÃO TEÓRICA

O esforço da indústria automotiva para padronizar a comunicação entre os diferentes componentes do carro, impulsionado pela necessidade de um sistema de rede que pudesse lidar com a crescente complexidade das comunicações internas do veículo, levou à criação do *Controller Network Area Bus* (CAN-bus), definido pela ISO 11898-1 (STANDARDIZATION, 2015). O CAN-bus é considerado a espinha dorsal de um carro, permitindo que todos os componentes separados troquem informações e coordenem funções para controlar várias partes do veículo.

O protocolo especifica quatro tipos de mensagens CAN ou "*frames*": *remote frame*, *error frame*, *overload frame* e *data frame*, esse último é o tipo de mensagem CAN padrão. Um transmissor envia um *remote frame* para solicitar dados de um nó específico. Um *error frame* pode ser transmitido por qualquer nó que detecte um erro. Já os *overload frames* são usados para introduzir um atraso adicional entre *frames* de dados ou *frames* remotos.

Conforme ilustrado esquematicamente pela Figura 2, uma mensagem CAN é composta por sete campos, sendo eles:

1. **Start Of Frame (SOF):** Indica o início da mensagem;
2. **Arbitration Field:** Contém o identificador e o bit de Solicitação de Transmissão Remota (RTR), que prioriza mensagens quando vários nós transmitem simultaneamente;
3. **Control Field:** Inclui o bit de Extensão do Identificador (IDE), o bit reservado (r0) e o Código de Comprimento de Dados (DLC), que especifica o número de bytes de dados;
4. **Data Field:** Contém até 8 bytes de dados.
5. **CRC Field:** Inclui a sequência de Verificação de Redundância Cíclica (CRC) e o delimitador CRC para detecção de erros.
6. **ACK Field:** Consiste no *slot* de *Acknowledgment* (ACK) e no delimitador ACK, onde os receptores indicam o recebimento da mensagem, e;

7. *End Of Frame (EOF)*: Marca o fim da mensagem.

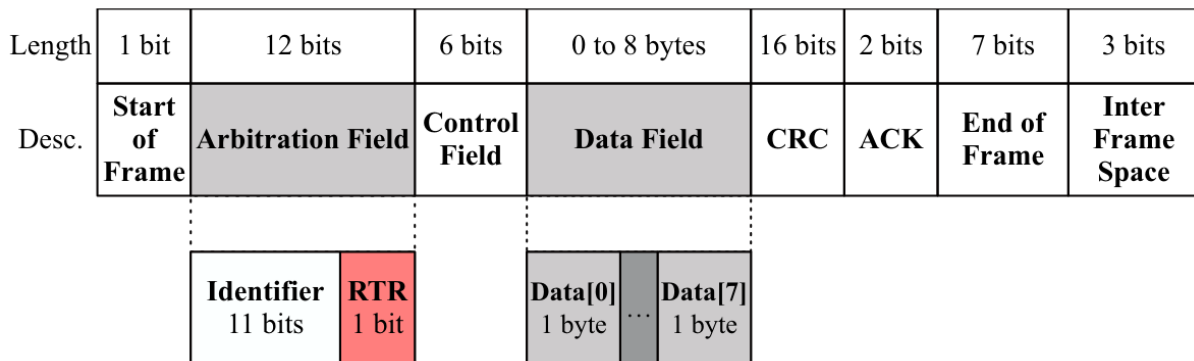


Figura 2 – **Estrutura de mensagens CAN**. Fonte: (LEE; JEONG; KIM, 2017).

Na rede CAN, todos os nós que aguardam para transmitir se sincronizam com o SOF e tentam iniciar a transmissão simultaneamente. Um esquema de arbitragem determina qual dos nós tentando transmitir controlará o barramento. O esquema de arbitragem CAN garante que a mensagem de maior prioridade seja transmitida. Nas redes CAN, a prioridade da mensagem é determinada pelo valor numérico do identificador no campo de arbitragem, sendo que o menor valor numérico tem a maior prioridade. A rede opera sob o protocolo de acesso múltiplo por detecção de portadora com detecção de colisão (CSMA/CD). Durante a transmissão, cada nó monitora o estado do barramento. Se um nó transmitir um bit recessivo e ler de volta um bit dominante, isso significa que houve uma colisão. Nesse caso, o nó para imediatamente de transmitir (VOSS, 2008).

O barramento CAN coleta o fluxo de dados de ramificação de vários sistemas de controle central de automóveis, como motor, sistema de transmissão, sistema de carroceria e diversos equipamentos elétricos do veículo, conforme mostrado na Figura 3, que ilustra de maneira simplificada a conexão de alguns dos vários componentes conectados ao barramento em um "veículo inteligente", nome dado a veículos altamente conectados.

2.1.1 Ataques ao Barramento CAN

No protocolo CAN, todos os dados de ramificação são transmitidos para o barramento na forma de transmissão, e qualquer nó pode enviar mensagens para a rede a qualquer momento. Isso indica que qualquer nó na rede CAN do veículo pode ser atacado por informações internas ou externas maliciosas a qualquer momento (ZHOU; LI; SHEN, 2019).

Ao longo dos anos, fabricantes de automóveis se esforçam para aumentar o conforto de seus clientes ao adicionar mais componentes tecnológicos ao carro, como sistemas de entretenimento multimídia e controle digital sobre componentes críticos, como suspensão, motor e transmissão. Consequentemente, mais dispositivos estão interconectados à rede *CAN-Bus*.

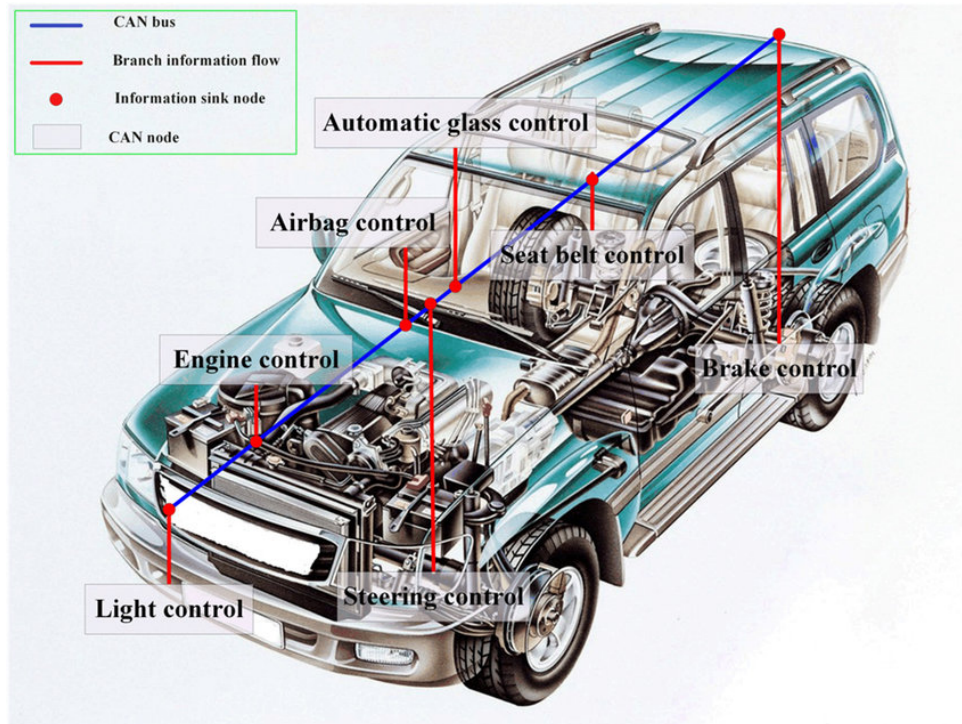


Figura 3 – Barramento de rede CAN em um veículo inteligente. Fonte: (ZHOU; LI; SHEN, 2019).

No entanto, isso traz um *trade-off* em termos de segurança, pois aumenta a exposição da rede CAN, já que há mais pontos de vulnerabilidade a ataques. Embora, assim como em qualquer outra rede, o simples acesso ao meio de comunicação não represente uma ameaça imediata, ele abre caminho para que invasores explorem vulnerabilidades e realizem ataques que podem ser extremamente perigosos.

Essa vulnerabilidade é ainda mais impactante em veículos autônomos, conectados e elétricos, que vêm ganhando significativo espaço no mercado. O número de sistemas interconectados e a complexidade das redes de comunicação aumentam substancialmente nesses tipos de veículos, visto sua dependência de comunicação em tempo real entre vários sistemas críticos, tornando-os ainda mais suscetíveis a ataques ao barramento CAN do que veículos à combustão tradicionais. A adição de dispositivos como sensores, câmeras, e sistemas de condução autônoma contribui para a exposição a riscos de segurança, uma vez que qualquer falha de segurança em um componente pode comprometer todo o sistema de controle do veículo. Já existem trabalhos na literatura focados em detectar ataques a IVNs em veículos autônomos (ZHOU; LI; SHEN, 2019).

Apesar da literatura apresentar outros possíveis ataques, o escopo deste TCC está centrado em ataques do tipo *Denial of Service* (DoS), Negação de Serviço em português, e ataques de *Impersonation* (Impersonificação). Tais cenários de ataque são ilustrados na Figura 4 e detalhados nas subseções que seguem.

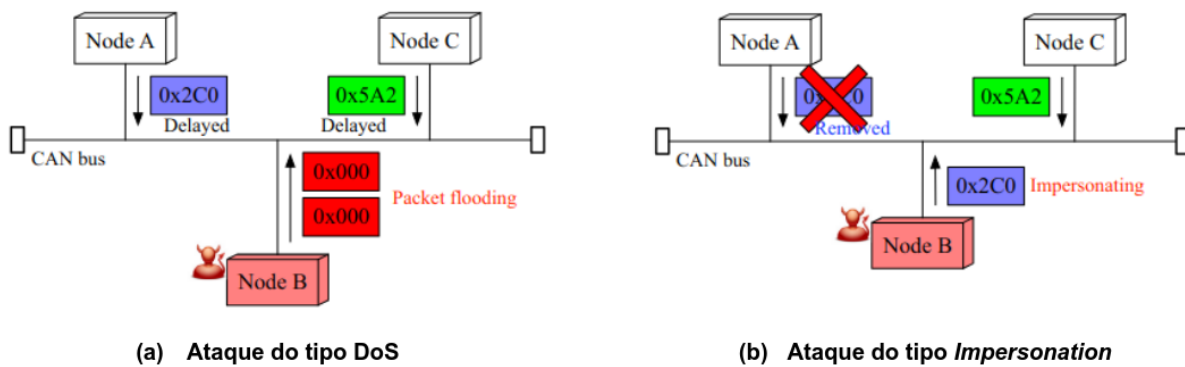


Figura 4 – Dois tipos de cenário de ataques em um barramento CAN. Fonte: (LEE; JEONG; KIM, 2017).

2.1.2 Ataque de DoS

Um invasor pode injetar mensagens de alta prioridade em um ciclo curto no barramento, explorando a ausência de um gerenciador centralizado na rede CAN. Como nenhum nó é explicitamente controlado, um nó malicioso pode violar as regras do protocolo para obter acesso não autorizado. Nesse tipo de ataque, mensagens DoS são injetadas utilizando o identificador de prioridade mais alta, geralmente 0x000, para monopolizar o barramento. Como ilustrado na Figura 4(a), ao inserir mensagens 0x000 entre as mensagens de solicitação e resposta legítimas, o atacante causa atrasos inevitáveis na transmissão, comprometendo a comunicação regular entre os nós (LEE; JEONG; KIM, 2017).

2.1.3 Ataque de Impersonation

Em 2016, Cho e Shin introduziram um invasor no barramento CAN que pode interromper a transmissão de mensagens controlando um nó alvo e plantar/manipular um nó imitador (CHO; SHIN, 2016). Se um nó vítima parar de transmitir, todas as mensagens enviadas pelo nó-alvo serão removidas do barramento, uma ilustração dessa substituição pode ser vista na Figura 4(b). No entanto, quando uma ECU recebe um *frame* remoto, ela é projetada para ter que transmitir um *frame* de dados imediatamente. Se um nó receptor não receber uma resposta a um *frame* remoto, pode-se saber que o nó existente foi atacado ou quebrado. Nesse caso, um adversário pode plantar um nó imitador para responder a um *frame* remoto. Assim, o nó imitador transmite *frames* de dados periodicamente e responde a um *frame* remoto como o nó-alvo, para ocultar se cada nó muda sua função (LEE; JEONG; KIM, 2017).

Tendo em vista o risco apresentado por ataques dessa natureza a um veículo, um sistema capaz de detectar intrusões em tempo real é fundamental para manter sua segurança, fornecendo alertas a uma unidade central de controle ou até mesmo agindo para evitar danos adicionais ao veículo, passageiros e pessoas ao redor.

2.1.4 Sistemas de Detecção de Intrusão (IDS)

A detecção de intrusões é um campo que ganhou relevância significativa ao longo das últimas décadas, especialmente devido ao aumento das ameaças cibernéticas e à popularização da Internet. No trabalho seminal de Aurobindo Sundaram, "*An Introduction to Intrusion Detection*", publicado em 1996, a importância da detecção de intrusões foi enfatizada em um momento em que o crescimento exponencial da Internet começava a transformar o ambiente computacional. Sundaram destacou a incapacidade de sistemas de segurança de prevenir totalmente todas as violações, argumentando que identificar intrusões de maneira eficaz era essencial para mitigar os danos. Esse trabalho estabeleceu uma base teórica para a área, classificando as intrusões em tentativas de acessar, manipular ou tornar sistemas inutilizáveis, e explorando técnicas iniciais para a identificação desses comportamentos (SUNDARAM, 1996).

Ainda segundo Sundaram, os modelos predominantes de detecção de intrusões eram baseados na detecção de anomalias e no uso indevido, cada qual com suas limitações. O modelo de anomalias busca identificar comportamentos que desviam do padrão esperado, enquanto o modelo de uso indevido se concentra na identificação de padrões conhecidos de ataques. Ele previu, corretamente, que uma abordagem híbrida, combinando ambos os modelos, seria essencial para aumentar a eficiência dos sistemas. Na época, ele apontou o sistema NIDES (*Next-generation Intrusion Detection Expert System*) como exemplo prático dessa estratégia, que se tornaria uma tendência predominante em pesquisas subsequentes (SUNDARAM, 1996).

Desde a publicação desse trabalho, os IDSs evoluíram consideravelmente, incorporando técnicas modernas como ML, Inteligência Artificial (IA) e análise de grandes volumes de dados (*big data*). No entanto, o arcabouço teórico estabelecido por Sundaram permanece relevante, pois os desafios fundamentais da detecção de intrusões — como a identificação de padrões desconhecidos e a mitigação de falsos positivos — ainda são centrais para a área.

Como também analisado em um *survey* de 2023, atualmente, as altas capacidades de generalização da IA tornam os IDSs *IA-based* uma excelente contramedida contra ataques cibernéticos automatizados (RAJAPAKSHA et al., 2023). É nesse tipo de abordagem que esse trabalho se baseia.

2.1.5 Machine Learning (ML)

Machine learning é um ramo da IA que permite que sistemas aprendam com dados, identifiquem padrões e tomem decisões com o mínimo de intervenção humana. Ele se baseia em algoritmos capazes de melhorar seu desempenho ao longo do tempo conforme são expostos a mais dados. Essa abordagem tem sido amplamente utilizada em diversas áreas, como detecção de anomalias, reconhecimento de fala, e análises preditivas, devido à sua capacidade de lidar com grandes volumes de informações de maneira eficiente e adaptativa. Para implementar projetos eficazes de aprendizado de máquina, é comum adotar metodologias estruturadas, como o CRISP-DM, que guia todo o ciclo de desenvolvimento de forma sistemática e iterativa.

2.1.5.1 Cross Industry Standard Process for Data Mining (CRISP-DM)

O CRISP-DM é uma estrutura sistemática que fornece um *framework* robusto para a condução de projetos de ciência de dados e mineração de dados. Desenvolvida em 1996, a metodologia foi projetada para padronizar o processo desses projetos, tornando-os mais consistentes e eficientes. Embora existam singelas diferenças entre a mineração de dados e o aprendizado de máquina, o CRISP-DM também é amplamente aplicável a projetos de ML, ajudando a simplificar o fluxo de trabalho e a alcançar resultados mais precisos. Sua estrutura baseia-se em seis fases principais: compreensão do negócio, compreensão dos dados, preparação dos dados, modelagem, avaliação e implantação, que são iterativas e flexíveis, permitindo adaptações a diferentes contextos (SHEARER, 2000).

Essa metodologia ajuda a superar desafios comuns, como o tratamento de conjuntos grandes e desbalanceados de dados, além de simplificar a adaptação a mudanças ao longo do projeto quando necessário. As seis etapas do CRISP-DM são:

1. **Entendimento do negócio:** define os objetivos do projeto e traduz os problemas do domínio em questões que podem ser resolvidas com ciência de dados.
2. **Entendimento dos dados:** coleta, explora e avalia a qualidade dos dados disponíveis para assegurar sua relevância para o projeto;
3. **Preparação dos dados:** realiza limpeza, transformação e seleção dos dados para criar o conjunto necessário para o desenvolvimento do modelo;
4. **Desenvolvimento do estudo (modelagem):** cria e treina modelos, aplicando técnicas de mineração de dados para responder às questões levantadas;
5. **Validação:** avalia o desempenho do modelo, verificando se ele atende aos objetivos do negócio e é aplicável ao problema, e;
6. **Implantação do projeto e acompanhamento:** implementa a solução em produção e monitora seu desempenho para garantir a continuidade dos resultados esperados.

O livro "*Machine Learning – Guia de Referência Rápida: Trabalhando com Dados Estruturados em Python*" (HARRISON, 2019) apresenta um fluxo de trabalho para criação de um modelo preditivo. A Figura 5 apresenta um diagrama adaptado desse fluxo.

2.1.5.2 Entendimento e Tratamento dos Dados

O processo de entendimento dos dados é uma etapa fundamental em um trabalho de ML, pois é nela que as informações são examinadas para garantir que o conjunto de dados seja adequado para o problema em questão. Isso inclui a análise de *features* (atributos dos dados), como sua distribuição, identificação de valores ausentes, detecção de *outliers* e compreensão

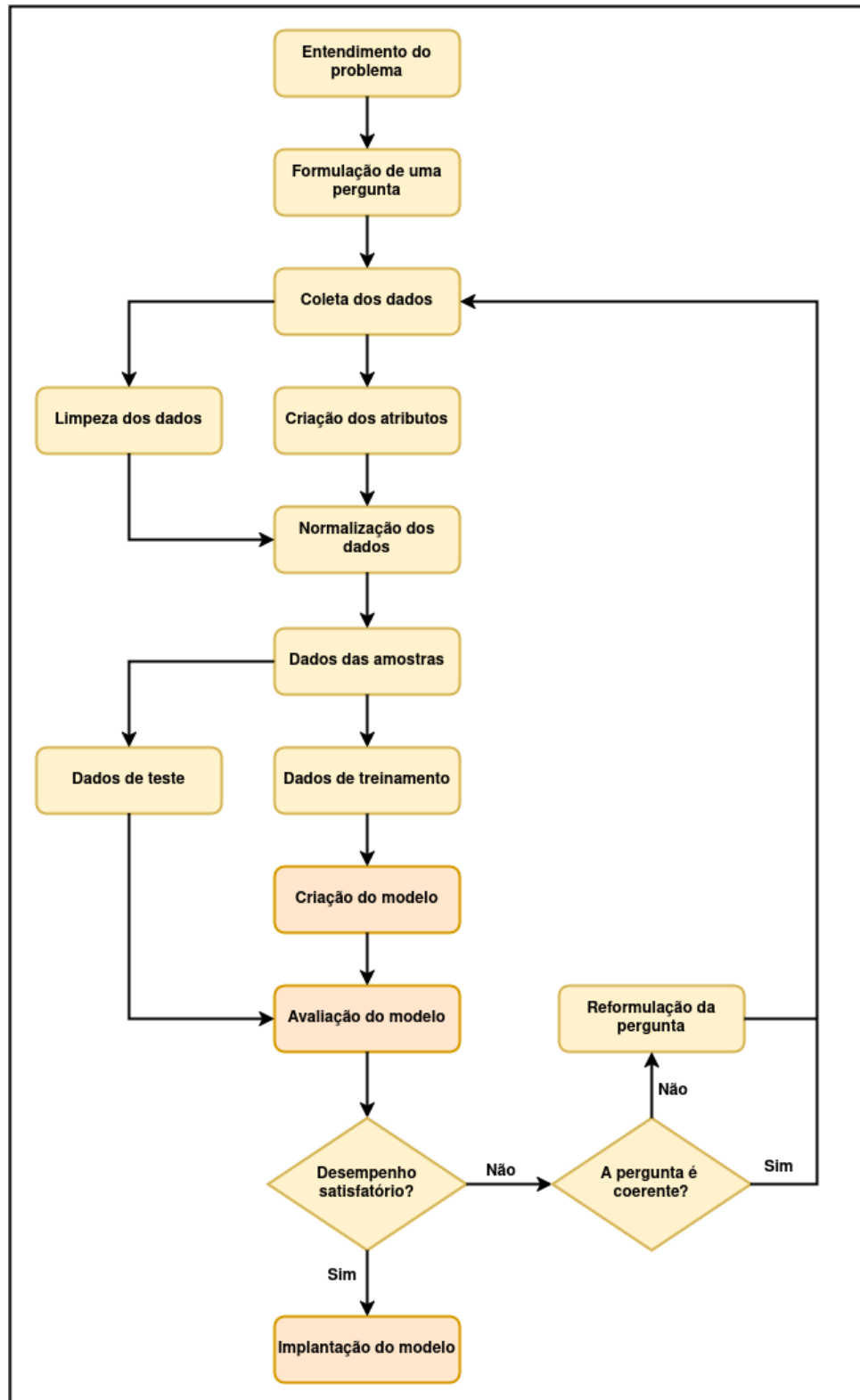


Figura 5 – Fluxo de trabalho comum em aprendizado de máquina baseado na metodologia CRISP-DM.
Fonte: Adaptado de (HARRISON, 2019).

das variáveis que podem ter maior relevância para a classificação. No caso de dados de ataques em redes CAN, é necessário entender o formato e a natureza das mensagens CAN presentes no *dataset*, bem como a relação entre os diferentes parâmetros que podem indicar a presença de

um ataque.

Após entender o comportamento dos dados, a etapa de tratamento envolve as ações necessárias para tornar os dados prontos para o modelo de ML. O tratamento de dados pode incluir a normalização, para garantir que os atributos estejam na mesma escala, a imputação ou remoção de valores ausentes e a transformação de variáveis categóricas em numéricas, necessária para alguns algoritmos. O processo de tratamento de dados costuma ser o mais longo em um trabalho de ML, embora varie de acordo com a qualidade dos dados previamente disponíveis.

Após a coleta, estudo e preparação dos dados, é fundamental a escolha dos algoritmos de ML corretos a serem utilizados naquele conjunto de dados para a etapa de desenvolvimento do estudo ou análise. Esses algoritmos podem ser divididos em diferentes classes, como algoritmos supervisionados e não supervisionados, redes neurais entre outros.

2.1.5.3 *Aprendizado Supervisionado e Não Supervisionado*

O aprendizado supervisionado é um tipo de ML onde o modelo é treinado com dados rotulados, ou seja, cada entrada de dados já possui a resposta esperada, permitindo que o modelo aprenda a mapear entradas para saídas. Já o aprendizado não supervisionado trabalha com dados não rotulados, buscando identificar padrões ou estruturas subjacentes nos dados, como *clusters* ou agrupamentos, sem a necessidade de respostas definidas previamente. A principal diferença entre ambos está na presença ou ausência de *labels* (rótulos) para guiar o processo de treinamento (HARRISON, 2019).

Este trabalho foca principalmente na utilização de algoritmos de aprendizado supervisionado, já que se trata de uma tarefa de predição de um valor (identificar ataques em conjuntos de mensagens CAN) e os dados são rotulados como 'ataque' ou 'não ataque'. Dentro dos algoritmos supervisionados, existem duas categorias principais: algoritmos de classificação e algoritmos de regressão. A primeira classe é voltada para a predição de valores discretos, enquanto a segunda é usada para prever valores contínuos. Como os ataques são eventos discretos e binários, o foco deste trabalho é na utilização de algoritmos de classificação.

Embora algoritmos não supervisionados não sejam a escolha ideal para tarefas de classificação, sua utilização não deve ser descartada. Se um bom desempenho for obtido, modelos não supervisionados podem ser vantajosos em tarefas de classificação, pois não necessitam de dados rotulados. Isso significa que um *dataset* de mensagens CAN, no qual não se sabe se contém ou não ataques, pode ser aplicado a esses modelos com confiança em sua capacidade de classificação.

2.1.5.4 *Decision Tree (Árvore de Decisão)*

A Árvore de Decisão (DT) é uma técnica de aprendizado de máquina supervisionado não paramétrico amplamente utilizada para tarefas de classificação e regressão. O algoritmo baseia-se em uma estrutura em forma de árvore, onde os nós internos representam pontos de

decisão sobre atributos, os ramos indicam os resultados dessas decisões e os nós folhas correspondem às previsões de valor para a variável alvo (QUINLAN, 1986).

Uma DT começa com um nó raiz que representa o conjunto de dados completo. Critérios de divisão, como Índice Gini, Ganho de Informação ou Entropia, são utilizados para particionar iterativamente o conjunto de dados em subconjuntos, garantindo a máxima homogeneidade dentro de cada subconjunto. O processo de divisão continua até que uma das seguintes condições de parada seja atendida:

- Todos os pontos de dados em um nó pertencem à mesma classe;
- Uma profundidade máxima pré-definida é alcançada;
- Divisões adicionais não melhoram significativamente a precisão preditiva.

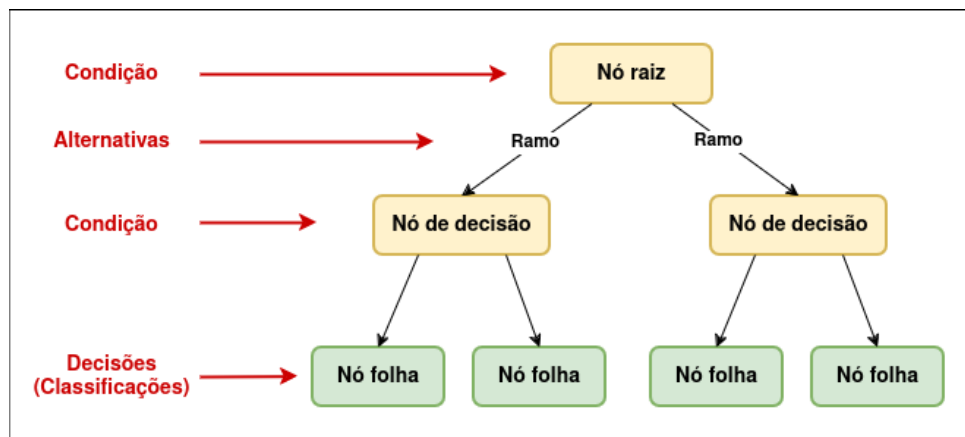


Figura 6 – Ilustração dos elementos de uma Árvore de Decisão. Fonte: Elaboração própria.

As DTs são interpretáveis e fáceis de visualizar, tornando-as adequadas para que não especialistas compreendam. Além disso, elas lidam eficazmente com dados numéricos e categóricos sem exigir pré-processamento extenso. No entanto, estão suscetíveis a *overfitting*, especialmente quando a árvore se torna muito profunda. Técnicas como poda, métodos de *ensemble* (por exemplo, *Random Forest*) ou limitação da profundidade da árvore podem mitigar esse problema.

Além disso, as DTs também são os componentes fundamentais de outros algoritmos de aprendizado de máquina, como as *Random Forests* (Subsubseção 2.1.5.5), que estão entre os algoritmos de ML mais poderosos disponíveis atualmente (GÉRON, 2019).

2.1.5.5 *Random Forest (Floresta Aleatória)*

A *Random Forest* (RF) combina diversas DTs em um único classificador final, oferecendo maior robustez e reduzindo a tendência ao *overfitting*, problema comum em DTs. Essa técnica introduz aleatoriedade no processo de construção das árvores e é composta por dois passos principais, sendo eles:

1. **Tree Bagging:** Gera aleatoriedade nos dados de treino ao criar subconjuntos aleatórios que serão usados no treinamento de múltiplas árvores de decisão. A decisão final é obtida pela média das predições (para regressão) ou pela classe mais votada (para classificação).
2. **Randomized Decision Tree Training:** Introduz aleatoriedade na escolha dos atributos durante a divisão dos nós. Em vez de avaliar todos os atributos disponíveis, apenas um subconjunto aleatório é considerado. Essa abordagem reduz a correlação entre as árvores, prevenindo redundância, especialmente quando poucos atributos têm alto poder preditivo.

A combinação final das predições de cada árvore resulta em um modelo mais geral e robusto, com menor suscetibilidade a erros específicos dos dados de treino (GÉRON, 2019).

2.1.5.6 *K-Nearest Neighbors (KNN)*

O algoritmo dos K Vizinhos Mais Próximos, como é conhecido em português, faz a classificação com base na distância até algumas amostras (k) de treinamento. A família de algoritmos à qual pertence o KNN é chamada de *instance-based learning* (aprendizado baseado em instâncias), pois não há parâmetros para aprender. O modelo pressupõe que a distância entre as instâncias é suficiente para fazer uma inferência; afora isso, nenhuma pressuposição é feita sobre os dados subjacentes ou suas distribuições (HARRISON, 2019).

A dificuldade ao adotar o KNN está em escolher o valor de k , sendo isso feito de maneira empírica na maioria dos casos. Além disso, a dimensionalidade pode atrapalhar as métricas de distância, pois haverá pouca diferença entre os vizinhos mais próximos e mais distantes em conjuntos de dados com muitas dimensões (HARRISON, 2019).

A Figura 7 exemplifica de forma visual o processo de classificação com o algoritmo KNN adotando $k = 5$. Na figura, há três grupos de dados, ω_1 , ω_2 e ω_3 , representados pelas cores vermelha, verde e azul, respectivamente. O ponto cinza X representa uma nova instância que precisa ser classificada em uma das classes com base em seus cinco vizinhos mais próximos. O raio preto em volta de X define a vizinhança considerada pelo algoritmo, englobando os pontos mais próximos dele. No exemplo ilustrativo, X seria classificado como uma instância da classe ω_1 , por ter três vizinhos desta classe, enquanto tem apenas dois vizinhos da classe ω_2 e zero vizinhos da classe ω_3 . Embora o exemplo contenha três classes, para utilizar o KNN em classificações binárias, bastando definir o número de grupos diferentes como dois.

2.1.5.7 *XGBoost*

O *XGBoost* é uma biblioteca otimizada de *gradient boosting* distribuído, projetada para ser altamente eficiente, flexível e portátil. Ele implementa algoritmos de aprendizado de máquina dentro do *framework* de *Gradient Boosting*. O *XGBoost* oferece um método de *boosting* paralelo de árvores (também conhecido como GBDT ou GBM), capaz de resolver diversos problemas de ciência de dados de forma rápida e precisa. O mesmo código pode ser

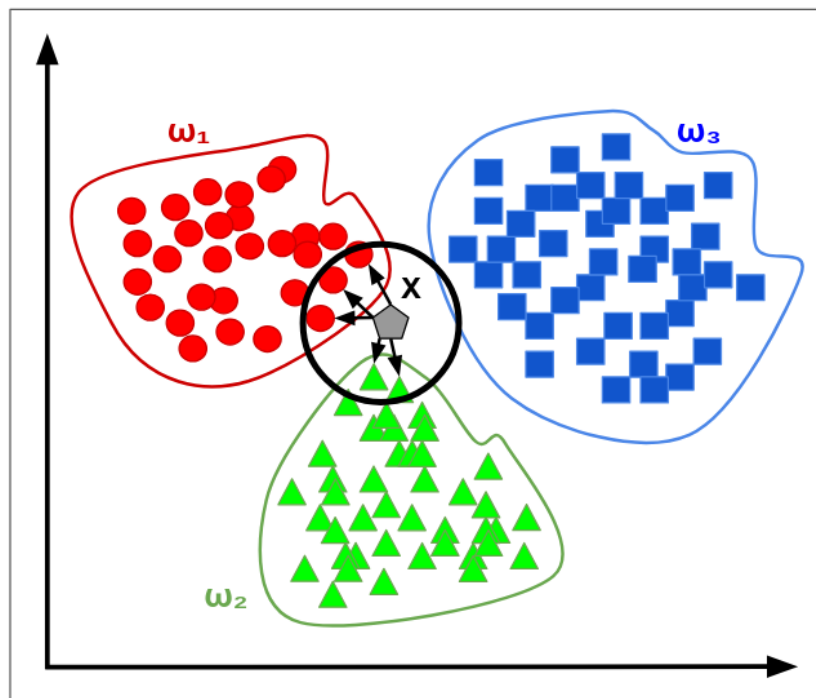


Figura 7 – Exemplo visual do algoritmo KNN aplicado a um problema de classificação. Fonte: Elaboração própria.

executado em grandes ambientes distribuídos (como Hadoop, SGE e MPI) e é capaz de lidar com problemas que envolvem bilhões de exemplos. A biblioteca tem suporte para C++, Java, Python, R, Julia, Perl e Scala (CHO, 2022).

O *Gradient Boosting* (GB) em si é um algoritmo popular de *boosting*. “*Boosting*” se refere a qualquer método *ensemble* que combina vários aprendizes fracos em um forte. A ideia geral da maioria dos métodos de *boosting* é treinar sequencialmente os previsores, cada um tentando corrigir seu antecessor. No caso do GB, ele adiciona previsores sequencialmente a um conjunto, cada um corrigindo seu antecessor. O que diferencia o GB de outros métodos de *boosting*, como o *AdaBoost*, é que ele não ajusta os pesos da instância a cada iteração. Ao invés disso, o *Gradient Boosting* tenta ajustar o novo predictor aos erros residuais feitos pelo predictor anterior (GÉRON, 2019).

2.1.5.8 K-Means

K-Means é um algoritmo de ML não supervisionado que agrupa pontos de dados em *clusters* (grupos) com base em suas características. O *K-Means* recebe por parâmetro o número de *clusters*, isto é, “*k*”. Então, ele escolhe aleatoriamente “*k*” centroides e atribui cada amostra a um *cluster* com base na métrica de distância a partir do centroide. Após a atribuição, os centroides são recalculados com base no centro de todas as amostras atribuídas a um rótulo. Em seguida, a atribuição das amostras aos *clusters* se repete com base nos novos centroides. Após algumas iterações, deve haver alguma convergência (HARRISON, 2019).

Como o *clustering* utiliza métricas de distância para determinar quais amostras são semelhantes, o comportamento poderá mudar dependendo da escala dos dados. Para contornar esse problema, pode-se padronizar os dados e colocar todos os atributos em uma mesma escala, embora alguns especialistas na área não aconselhem essa decisão caso a escala sinalize que algum atributo de fato tem mais importância que os demais (HARRISON, 2019).

2.1.5.9 Long Short-Term Memory (LSTM)

Diferentemente dos demais algoritmos abordados neste trabalho, uma LSTM é uma rede neural, um modelo computacional que imita o funcionamento do cérebro humano para processar informação e aprender com dados, utilizando nós interconectados organizados em camadas. As LSTMs são uma variante das Redes Neurais Recorrentes (RNNs), propostas em 1997 por Hochreiter e Schmidhuber, que introduzem avanços significativos em relação ao modelo padrão, apresentando desempenho superior em diversas aplicações modernas, como tradução automática e modelagem de linguagens. Essas redes são particularmente eficazes para prever séries temporais de durações variáveis, destacando-se por sua capacidade de lidar com grandes intervalos de dependência temporal, graças à sua estrutura que permite uma memória prolongada (BARBOSA et al., 2021).

O principal diferencial da LSTM está em sua arquitetura composta por quatro redes neurais e blocos de células especializadas. Essas células armazenam informações e manipulam memórias por meio de mecanismos chamados *gates* (portas), divididos em três tipos principais, são eles:

- **Forget Gate:** determina quais informações devem ser descartadas das células. Isso é feito através de uma saída binária, calculada por uma função de ativação que considera as matrizes de peso;
- **Input Gate:** adiciona novas informações relevantes às células. Para isso, combina os valores dos vetores de entrada com uma função de ativação reguladora, permitindo identificar quais dados são úteis, e;
- **Output Gate:** extrai as informações úteis das células para serem utilizadas como saída da célula atual e entrada da próxima. Essa saída é definida pela multiplicação dos valores internos e os regulados pela função de ativação.

A arquitetura básica de uma célula LSTM é mostrada na Figura 8. Com essa estrutura sofisticada, as LSTMs são capazes de capturar padrões em dados sequenciais de forma eficiente, superando as limitações das RNNs tradicionais, especialmente em tarefas que exigem retenção de informações por longos períodos (HOCHREITER; SCHMIDHUBER, 1997).

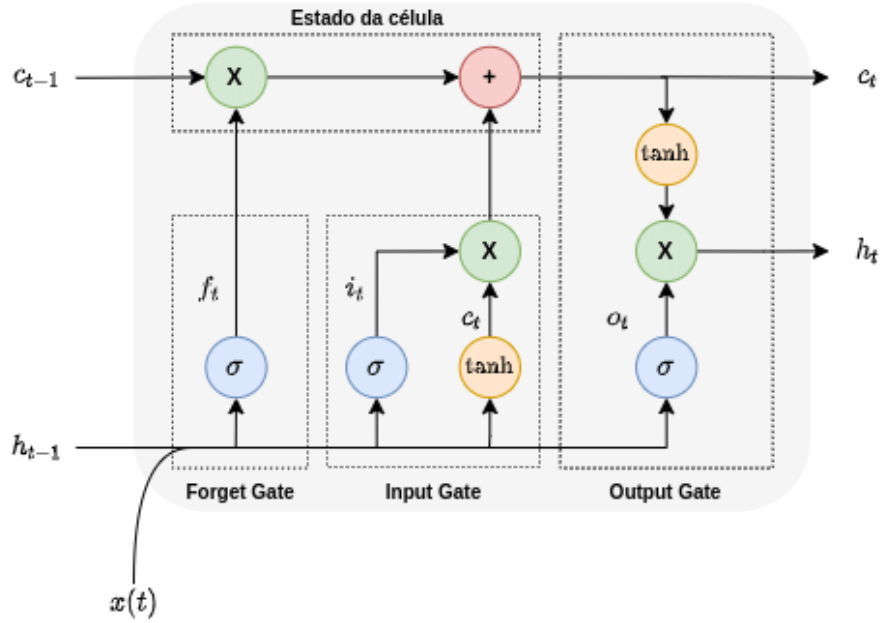


Figura 8 – Célula básica de uma rede neural de *Long Short-Term Memory* (LSTM). Fonte: (BARBOSA et al., 2021).

2.1.5.10 Validação

Realizado o treinamento dos modelos, chega-se a quinta etapa do CRISP-DM: a validação. Nessa etapa, é necessário avaliar o desempenho dos modelos em um ou mais conjuntos de teste, utilizando de métricas como acurácia e métodos como a matriz de confusão.

A validação é uma etapa crucial no processo de ML, pois permite avaliar o desempenho e a capacidade de generalização de um modelo em dados não vistos. A principal função da validação é garantir que o modelo não esteja apenas memorizando os dados de treinamento (*overfitting*), mas que ele seja capaz de fazer previsões corretas em dados novos, essa capacidade em ML é chamada de generalização. As métricas utilizadas na avaliação do modelo dependem do tipo de problema a ser resolvido. Para tarefas de classificação, é comum utilizar métricas como acurácia, precisão, *recall* (sensibilidade), e *F1-score*. As equações que descrevem essas métricas são apresentadas abaixo.

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN}, \quad (2.1)$$

onde VP significa Verdadeiro Positivo, VN significa Verdadeiro Negativo, FP significa Falso Positivo, e FN significa Falso Negativo,

$$\text{Precisão} = \frac{VP}{VP + FP}, \quad (2.2)$$

$$\text{Recall} = \frac{VP}{VP + FN}, \quad (2.3)$$

$$F1\text{-Score} = 2 \cdot \frac{\text{Precisão} \cdot \text{Recall}}{\text{Precisão} + \text{Recall}}. \quad (2.4)$$

Um método de visualização para resultados de algoritmos de classificação é a matriz de confusão, uma tabela que decompõe o número de instâncias da verdade fundamental de uma classe específica em relação ao número de instâncias de classes previstas (GÉRON, 2019). A Figura 9 ilustra uma matriz de confusão genérica. A matriz de confusão ideal é uma matriz diagonal, cujos elementos exteriores à diagonal principal, ou seja, FP e FN, são nulos.

Matriz de Confusão		Valor Predito	
		Positivo	Negativo
Valor Real	Positivo	VP	FN
	Negativo	FP	VN

Figura 9 – Representação de uma matriz de confusão. Fonte: Elaboração própria.

Embora sejam desejáveis bons resultados nessas métricas de avaliação, eles sozinhos não são suficientes para validar a presença de *overfitting* no modelo, sendo necessário o uso de técnicas como a divisão simples do conjunto de dados em *train-test*, onde uma parte é utilizada para treinamento e a outra para teste. Nesse caso, o desempenho do modelo é avaliado com base na sua capacidade de generalizar o aprendizado para os dados de teste. Há entretanto técnicas mais sofisticadas que podem ser adotadas em conjunto com o *train-test*, como a validação cruzada.

2.1.5.10.1 Validação Cruzada (*Cross-Validation*)

A *cross-validation* é uma técnica estatística usada para avaliar a capacidade de generalização de um modelo, a partir de um conjunto de dados. Esta técnica é amplamente empregada em problemas onde o objetivo da modelagem é a predição, como este trabalho.

O conceito central das técnicas de validação cruzada é o particionamento do conjunto de dados em subconjuntos mutuamente exclusivos, e posteriormente, o uso de alguns destes subconjuntos para a estimação dos parâmetros do modelo (dados de treinamento), sendo os subconjuntos restantes (dados de validação ou de teste) empregados na validação do modelo. Diversas formas de realizar o particionamento dos dados foram sugeridas, sendo as três mais utilizadas: o método *holdout*, o *k-fold* e o *leave-one-out* (KOHAVI, 2001).

Para todos os métodos de particionamento citados acima, a precisão final do modelo estimada é obtida por

$$Ac_f = \frac{1}{v} \sum_{i=1}^v \varepsilon_{y_i, \hat{y}_i} = \frac{1}{v} \sum_{i=1}^v (y_i - \hat{y}_i), \quad (2.5)$$

onde v é o número de dados de validação e $\varepsilon_{y_i, \hat{y}_i}$ é o resíduo dado pela diferença entre o valor real da saída i e o valor predito.

Feita a validação correta do modelo desenvolvido, define-se se o modelo está pronto para a etapa final de implantação ou se é preciso ser feita uma abordagem diferente no conjunto de dados, revisitar o problema para redefinir o que se espera do modelo ou escolher um outro algoritmo para resolver o problema.

2.1.6 Navalha de Ockham

A Navalha de Ockham (ou Occam) é a base de diversas técnicas de regularização e critérios de seleção de modelos no escopo de ML (MURPHY, 2012). O princípio afirma que diante de modelos que explicam os dados com a mesma eficácia, a preferência deve ser pelo mais simples. Segundo esse princípio, hipóteses mais simples tendem a generalizar melhor para novos dados — uma ideia que, na teoria do aprendizado, é formalizada por meio de limites relacionados à complexidade dos modelos (SHALEV-SHWARTZ; BEN-DAVID, 2014).

Muitos algoritmos de ML incorporam a Navalha de Ockham de forma implícita, buscando modelos que equilibrem simplicidade e capacidade de ajuste aos dados, com o objetivo de melhorar seu poder de generalização (BISHOP, 2006).

2.2 TRABALHOS CORRELATOS

Esta seção apresenta estudos que evidenciam a vulnerabilidade da rede CAN a ataques e, em seguida, revisa propostas de IDS, com foco especial no uso de algoritmos de ML. A análise enfatiza tanto os tipos de ataques quanto as abordagens de detecção, suas limitações e o uso de conjuntos de dados amplamente adotados na literatura.

Diversos casos de ataques à rede CAN e suas consequências foram documentados na literatura. Koscher et al. demonstraram a vulnerabilidade de vários módulos críticos — incluindo o painel de instrumentos, o módulo de controle da carroceria, o módulo de controle de freios e o módulo de controle do motor — a ataques de *Impersonation* e ataques *Fuzzy*. Sua pesquisa conseguiu comprometer a funcionalidade do motor, permitindo manipulação do tempo de ignição e das RPM. Além disso, os autores conseguiram liberar os freios e impedir sua ativação enquanto o veículo trafegava a 40 milhas por hora (mph) (KOSCHER et al., 2010).

Em outro estudo, Mukherjee et al. realizaram um ataque DoS, inundando a rede com mensagens de solicitação excessivas, levando a um *overflow* de *buffer* e à saturação da rede (MUKHERJEE et al., 2016). Da mesma forma, Palanca e outros conduziram um ataque

DoS furtivo usando um nó malicioso que injetava quadros de erro e acionava mecanismos de confinamento de erros do CAN, forçando os nós transmissores a um estado de "*bus-off*", desabilitando assim sua capacidade de comunicação (PALANCA et al., 2017). Notavelmente, os pesquisadores optaram por excluir a desativação do sensor de estacionamento por considerações éticas.

Um dos estudos mais relevantes da área foi realizado por Lee et al., que propuseram um método baseado na análise da razão de deslocamento e do intervalo de tempo entre mensagens de solicitação e resposta (LEE; JEONG; KIM, 2017). Eles observaram que, em um estado sem ataque, os nós da rede CAN apresentam intervalos de tempo e razões de deslocamento fixos, que tendem a variar significativamente durante um ataque. Assim, o método monitora essas variações, conseguindo detectar intrusões de forma rápida e precisa, sem a necessidade de modificar o protocolo CAN. Esse trabalho foi capaz de detectar três tipos de ataques: DoS, *Fuzzy* e *Impersonation*, além de identificar as mensagens injetadas e os nós (ECUs) comprometidos na rede. Vale ressaltar que os autores disponibilizaram os conjuntos de dados utilizados no website do HCRL, os quais foram obtidos registrando o tráfego CAN de um veículo real durante a realização de ataques de injeção de mensagens. Esses dados foram utilizados em diversos trabalhos posteriormente, incluindo este, e será referenciado de agora em diante neste trabalho como "conjunto de dados do HCRL" ou "*dataset* do HCRL".

Outro trabalho relevante foi o de Song et al., que desenvolveram um IDS eficiente, confiável e de baixo custo computacional. A proposta deles baseia-se na análise dos intervalos de tempo entre as mensagens CAN para identificar três tipos de ataques de injeção no barramento: injeção dupla, quádrupla e desacoplamento de velocidade. Adicionalmente, eles testaram ataques DoS injetando um grande número de mensagens no barramento, elevando a taxa normal de 2.000 para cerca de 6.000 mensagens por segundo. Para detectar esses ataques, foi definido um intervalo de corte de 0,2 milissegundos. Além disso, os autores implementaram um método de pontuação para reduzir os falsos positivos, aumentando a pontuação para cada mensagem consecutiva com intervalo abaixo de 0,2 milissegundos. Seus testes mostraram que a precisão da detecção variava conforme o limiar de pontuação utilizado, alcançando 100% de precisão com limiares mais altos. A rapidez na detecção permitiu que os ataques DoS fossem identificados em menos de um milissegundo, o que é crucial para evitar acidentes (SONG; KIM; KIM, 2016).

Já Alshammari et al. investigaram o uso de algoritmos de ML supervisionados, como KNN e SVM, para detectar intrusões no barramento CAN. Eles treinaram e testaram esses algoritmos utilizando dois dos conjuntos de dados de *hacking* automotivo fornecidos pelo HCRL - anteriormente citado - o "*DoS dataset*" e o "*Fuzzy dataset*" (ALSHAMMARI et al., 2018). Embora tenham alcançado bons resultados, com precisão acima de 96%, eles não aplicaram técnicas adicionais para verificar e evitar *overfitting*, além de não fornecerem detalhes sobre o tempo de classificação das mensagens de ataque. Em um trabalho semelhante, Gundu et al. também utilizaram o método KNN, porém avaliaram os modelos Random Forest e XGBoost em vez do SVM (GUNDU; MALEKI, 2022).

Outro estudo de destaque foi realizado por Purohit et al., que propuseram três modelos de ML para detecção de intrusões: *Decision Tree*, *Random Forest* e *XGBoost*. Este foi um dos poucos trabalhos que mediu o tempo de treinamento e teste dos modelos, mas, assim como os outros, não mensuraram o tempo necessário para classificar uma amostra de dados, algo essencial para a implementação do IDS em uma plataforma real (PUROHIT; GOVINDARASU, 2022).

Por fim, Anjum et al. utilizaram o algoritmo *XGBoost* para detectar ataques de injeção e mau funcionamento em redes CAN. Diferentemente dos outros trabalhos, eles desenvolveram modelos específicos para cada tipo de ataque, como DoS, *Fuzzy* e *Spoofing* de RPM. Seus resultados, ao avaliar o conjunto de dados do HCRL, foram promissores, alcançando *F1-scores* de 94,74% e 96,49% para ataques de *flooding* e mau funcionamento, respectivamente (ANJUM et al., 2022). No entanto, assim como os trabalhos anteriores, eles não analisaram o tempo de execução real dos modelos.

A análise dos trabalhos correlatos revela uma clara trajetória na pesquisa de segurança de IVNs: partindo da demonstração prática de vulnerabilidades, como nos trabalhos de Koscher et al., evoluiu-se para a proposição de IDSs. Inicialmente baseados em regras e análise temporal, como em Lee et al. e Song et al., há um crescimento na literatura atual de IDSs baseados em ML. Estudos como os de Alshammari et al., Purohit et al. e Anjum et al. demonstram o alto potencial de algoritmos como *XGBoost*, RF e KNN para identificar ataques com precisão, utilizando o consolidado dataset do HCRL.

Contudo, uma lacuna comum e significativa identificada na maioria dessas pesquisas é a ausência de uma análise aprofundada sobre a viabilidade prática da implementação dos modelos em um cenário real. Muitos estudos, embora apresentem excelentes métricas de acurácia, não investigam a robustez de seus modelos contra *overfitting* com métodos como a validação cruzada, nem mensuram o tempo de classificação de novas amostras — um fator crítico para qualquer aplicação embarcada. Nesse contexto, este trabalho busca não apenas alcançar um desempenho de classificação superior, mas, diferencialmente, preencher essa lacuna ao realizar uma avaliação rigorosa da capacidade de generalização dos modelos e, de forma central, ao medir e analisar seu tempo de execução em plataformas embarcadas de baixo custo. Essa abordagem permite uma discussão concreta sobre a viabilidade de integrar tais soluções diretamente em uma ECU veicular, avançando da prova de conceito teórica para uma análise de aplicabilidade prática.

3 MATERIAIS E MÉTODOS

Neste capítulo, detalha-se os experimentos seguidos neste trabalho, fundamentados na metodologia CRISP-DM, detalhada na Subseção 2.1.5. A aplicação dessa abordagem garantiu uma experimentação sistemática e robusta para o processo de classificação de ataques em redes CAN automotivas. As seções seguintes descrevem como cada uma das quatro primeiras etapas foi implementada neste trabalho.

Baseado no CRISPM-DM, na primeira fase do projeto, conhecida como entendimento do negócio (problema), foi definido o objetivo principal do trabalho: identificar mensagens atacantes injetadas em redes veiculares baseadas no protocolo CAN utilizando técnicas de ML. Os ataques analisados foram do tipo Negação de Serviço (DoS) e *Impersonation*, com foco na criação de um IDS capaz de distinguir mensagens CAN normal de mensagens maliciosas.

3.1 ENTENDIMENTO E PREPARAÇÃO DOS DADOS

A segunda etapa do CRISP-DM, conhecida como Entendimento dos Dados é quando ocorre a coleta, a exploração e a avaliação da qualidade dos dados disponíveis para assegurar sua relevância para o projeto. Já a terceira etapa é conhecida como Preparação dos Dados. Nessa etapa, é realizada a limpeza, a transformação e a seleção dos dados para criar o conjunto de dados necessário para o desenvolvimento do(s) modelo(s).

Para este trabalho, foram utilizados dois *datasets*, sendo um público, disponível no site do HCRL; e o outro coletado de um carro real pelos próprios integrantes do IASE, projeto do LISHA. As subseções seguintes descrevem as características desses conjuntos de dados, assim como a preparação realizada neles para serem utilizados no processo de treinamento e teste dos modelos de ML propostos neste trabalho.

3.1.1 HCRL Dataset

Os *datasets* descritos nessa subseção foram publicados pelo HCRL, utilizados inicialmente no trabalho de Lee et al. (LEE; JEONG; KIM, 2017) e posteriormente usados em diversos outros trabalhos da literatura, inclusive os descritos na Seção 2.2 deste documento. Esses conjuntos de dados incluem ataques DoS, ataques *Fuzzy*, ataques de *Impersonation* e mensagens livres de ataques. Os conjuntos de dados foram construídos registrando o tráfego CAN através da porta OBD-II de um veículo real - um KIA Soul - enquanto os ataques de injeção de mensagens eram realizados.

Apesar de ser descrito, inclusive pelos próprios autores, como um conjunto de dados único, na verdade, esse *dataset* está dividido em quatro subconjuntos, disponíveis no formato *.txt* no *website* do HCRL: <https://ocslab.hksecurity.net/Dataset/CAN-intrusion-dataset> (HCRL, 2017). Como descrito na Tabela 1, o subconjunto saudável, ou seja, que contém apenas mensagens CAN normais, conta com aproximadamente 2,4 milhões de mensagens; o subconjunto

com ataques DoS tem mais de 650 mil mensagens, sendo quase metade delas de ataques DoS; o *dataset* de *Fuzzy* tem quase 600 mil; e o subconjunto com ataques de *Impersonation* contém quase 1 milhão de mensagens, em que aproximadamente 100 mil são ataques.

Os *datasets* não estão anotados, o que tornou essa anotação primordial para o uso desses conjuntos, porém a página do site do HCRL esclarece quais instâncias são ataques e quais não são, como segue:

- **Instâncias com ataque DoS:** toda mensagem enviada pela ECU falsa com o ID=0x000 é um ataque DoS. É importante frisar que esse identificador não foi escolhido sem motivos, visto que é o ID com maior prioridade em uma rede CAN;
- **Instâncias com ataque *Fuzzy*:** não há rótulos indicando se cada registro é normal ou anormal, apenas que o período de 0 segundos até 250 segundos é livre de ataques, enquanto o período a partir de 250 segundos contém ataques. Por conta dessa impossibilidade de diferenciar as mensagens normais das mensagens atacantes do tipo *Fuzzy*, este *dataset* foi descartado e não utilizado neste trabalho;
- **Instâncias com ataque de *Impersonation*:** assim como para os ataques *Fuzzy*, não há rótulo indicando se cada registro é normal ou anormal, porém os autores explicitam qual ECU foi impersonificada: a de ID=0x164. Como eles informam também o período sob ataque (o mesmo do *dataset* de ataques *Fuzzy*), é possível identificar quais mensagens são ataques e quais não são e anotar o *dataset*.

Dataset	Número de instâncias
Mensagens Normais (MN)	2,369,868
MN + DoS	656,579
MN + Fuzzy	591,990
MN + Impersonation	995,472

Tabela 1 – Descrição do conjunto de dados publicado pelo HCRL.

3.1.2 IASE Dataset

Esse conjunto de dados, assim como o fornecido pelo HCRL, foi coletado a partir da rede CAN de um veículo real, neste caso, uma Oroch 2022, que foi disponibilizada pela Renault do Brasil para o projeto IASE, uma parceria da fabricante de carros com o LISHA. A coleta de dados, entretanto, foi realizada de uma forma diferente, pois utilizou a *IASE Board* para coletar as mensagens do barramento. A conexão entre a ECU do carro e o hardware do IASE é realizada por meio do protocolo XCP sobre o barramento CAN. A conexão física entre a ECU e a interface CAN é feita através de conectores DB9. A Figura 10 mostra uma visão geral do hardware utilizado no IASE para a coleta de dados do veículo a partir do barramento CAN.

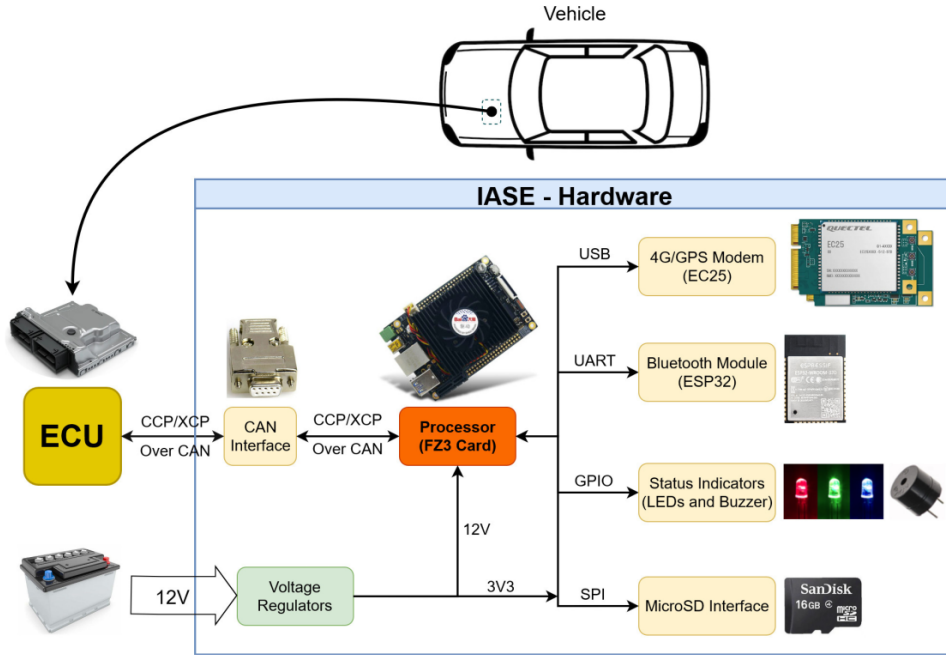


Figura 10 – Visão geral do hardware de aquisição de dados do projeto IASE. Fonte: (BEDRETSCHUK et al., 2023).

Os ataques presentes neste *dataset* também se diferenciam dos ataques realizados no conjunto de dados do HCRL: enquanto o primeiro teve seus ataques injetados diretamente no barramento CAN do veículo, este teve suas instâncias de ataques sintetizadas no conjunto de dado *a posteriori*.

Embora esse conjunto seja menor que o primeiro, o processo de sintetização do *dataset* seguiu a mesma proporção e periodicidade de ataques realizada no trabalho referência (LEE; JEONG; KIM, 2017). Além disso, os ataques do tipo DoS foram realizados também usando o identificador de prioridade máxima na rede CAN (ID=0x000) e os ataques do tipo *Impersonation* foram realizados utilizando o identificador de uma das ECUs presentes na rede, substituindo o conteúdo de suas mensagens de dados por mensagens maliciosas. Por serem dados sigilosos da composição eletrônica dos veículos da Renault, esse *dataset* não pode ser publicado, nem mesmo os IDs das ECUs presentes no tráfego CAN coletado, assim como os códigos de sintetização de ataques foram omitidos no Apêndice C, que traz o código-fonte deste trabalho.

3.1.3 Preparação dos Dados

A etapa de preparação dos dados envolveu principalmente a organização e rotulação dos dados e um pequeno processo de *feature engineering* (engenharia de características), mais especificamente, criação de *features* derivadas e transformação de formato de representação. Como os *datasets* originais do HCRL estavam em formato *.txt*, enquanto o tráfego CAN exportado pela IASE Board estava em *.log*, foi necessário converter os arquivos com os conjuntos de dados para estruturas compatíveis com as bibliotecas de ML utilizadas. No caso, todos os

arquivos de dados foram convertidos para o formato *.csv*.

As instâncias de ataques foram identificadas e anotadas com base nas regras definidas por cada *dataset*, como descrito na Subseção 3.1.1. Além disso, como mencionado na mesma subseção, houve uma filtragem, em que um subconjunto com informações incompletas foi descartado. No caso, o subconjunto que continha ataques *Fuzzy* no *dataset* HCRL.

Além disso, foi necessário realizar uma conversão no formato das *features* dos *datasets*. Como visto na Figura 2, os campos de uma mensagem CAN estão em base hexadecimal. Todos os valores foram convertidos para base decimal. Essa conversão é útil porque muitos algoritmos de ML não conseguem operar diretamente com formatos como hexadecimal, que são tratados como *strings*, mas exigem entradas numéricas. Ao converter os valores, mantém-se a informação original, porém agora processável.

Por fim, também foi necessário realizar outro processo de *feature engineering*, conhecido como criação de *features* derivadas. O campo de 'Data', que estava em formato de *string* com oito valores hexadecimais separados por espaço, representando os oito bytes de dados enviados em uma mensagem CAN, além de serem convertidos para decimal em uma *feature* única, também foram separados em *features* individuais. Também foi criada uma *feature* chamada "*Diff_Timestamp*", que representa a diferença de tempo entre uma instância n e a instância $n - 1$ no *dataset*. A Figura 11 ilustra esse processo, bem como as *features* finais dos conjuntos de dados, desconsiderando a coluna *target* (alvo), que leva o nome do tipo de ataque presente no *dataset*.

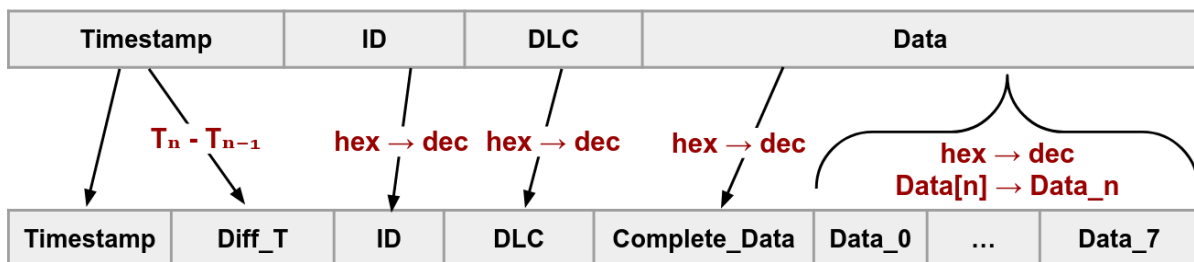


Figura 11 – Ilustração do processo de *feature engineering* realizado nos conjuntos de dados utilizados. Fonte: Elaboração própria.

3.2 MODELAGEM

Durante a etapa de modelagem, quarta do CRISP-DM, cria-se e treina os modelos selecionados, aplicando técnicas de mineração de dados para responder às questões levantadas. Neste trabalho, foram utilizados cinco algoritmos ditos clássicos de *Machine Learning* (ML), sendo quatro deles algoritmos supervisionados e um não supervisionado, além de uma RNN.

3.2.1 Classificadores

Para o treinamento dos modelos de classificação utilizados, utilizou-se a versão 3.10.12 do Python. Os quatro classificadores escolhidos para este trabalho foram os seguintes:

1. Árvore de Decisão (*Decision Tree* - DT);
2. Floresta Aleatória (*Random Forest* - RF);
3. *K-Nearest Neighbor* (KNN), e;
4. *XGBoost*.

Dentre os classificadores escolhidos, os três primeiros são implementações da biblioteca de aprendizado de máquina Scikit-Learn, versão 1.1.3, com os parâmetros padrões (*default*) da biblioteca sendo passados no treinamento dos modelos. O *XGBoost* é o único classificador utilizado que não é uma implementação do Scikit. Como descrito na Subseção 2.1.5.7, o *XGBoost* é uma biblioteca própria que implementa uma otimização do método *gradient boost*. A versão utilizada dessa biblioteca neste trabalho também foi a 2.0.3.

3.2.2 Clusterizador

K-Means é o único algoritmo de aprendizado não supervisionado usado neste trabalho, como descrito na Subseção 2.1.5.8, ele é utilizado especialmente para tarefas de clusterização. Essa escolha se justifica pelo fato de os algoritmos supervisionados serem melhores em tarefas de classificação, já que utilizam de rótulos (*labels*) fornecidos nos conjuntos de dados de treino para a variável alvo, algo que não existe em modelos não supervisionados, como o *K-Means*.

Apesar do ponto fraco dos clusterizadores descrito acima, é extremamente válido avaliar o desempenho de um algoritmo dessa classe no trabalho de detecção de ataques, visto que, comprovado um bom resultado, mostra-se possível utilizar de modelos que não precisam ser treinados com conjuntos de dados rotulados. Em suma, isso significa que um *dataset* de mensagens CAN que não se sabe se contém ou não ataques pode ser usado nesse modelo com confiança. A implementação do algoritmo utilizada neste trabalho foi a da versão 1.1.3 da biblioteca Scikit-Learn.

3.2.3 Rede Neural Recorrente (RNN)

Para este trabalho, apenas uma RNN foi escolhida para ser comparada com os classificadores clássicos: a LSTM. A implementação de redes LSTM para detecção de ataques em barramentos CAN representa uma abordagem inovadora que explora fundamentalmente a natureza sequencial das comunicações automotivas. Diferentemente de métodos tradicionais como *XGBoost* que analisam características isoladas, as LSTMs mantêm um estado interno

que captura dependências temporais de longo prazo, permitindo identificar padrões anômalos que se desenvolvem ao longo do tempo. Esta capacidade é particularmente valiosa para detectar ataques de *Impersonation*, onde invasores podem replicar valores de campos individuais, mas frequentemente falham em reproduzir os precisos intervalos temporais e a ordenação das mensagens que caracterizam comunicações legítimas entre ECUs.

Para maximizar o potencial da LSTM, os dados do barramento CAN foram processados como sequências temporais utilizando janelas deslizantes (*sliding window*), onde cada mensagem é contextualizada por suas predecessoras. Esta abordagem permite que a rede neural aprenda não apenas os valores dos campos das mensagens, mas também as relações temporais entre mensagens consecutivas.

A estrutura de entrada da LSTM foi configurada com dimensões [*amostras, time_steps, features*], onde *time_steps* representa o número de mensagens consecutivas consideradas para cada previsão, permitindo que o modelo capture o comportamento dinâmico característico do sistema CAN. Por conta das características dos ataques abordados, especialmente o *Impersonation*, a LSTM foi treinada com 4 diferentes valores para o *time_steps*: 1, 10, 20, 50. A arquitetura da LSTM conta com duas camadas sequenciais: a primeira com 128 unidades e a segunda com 64 unidades, ambas com camadas de normalização em lote (*batch normalization*) e *dropout* de 20% para reduzir *overfitting*. Em seguida, o modelo inclui uma camada densa com 32 neurônios ativados por ReLU, e uma camada de saída com ativação sigmoide, adequada para a tarefa binária de detecção de ataques.

Durante o treinamento, foi empregada uma estratégia de *early stopping* com paciência de 10 épocas, monitorando a perda na validação para evitar *overfitting* e garantir que os melhores pesos fossem restaurados. Um *callback* adicional foi utilizado para salvar automaticamente o melhor modelo com base na menor perda de validação. O treinamento foi realizado por até 100 épocas com *batch size* de 64. Além disso, o código foi preparado para execução otimizada tanto em GPU quanto em CPU, aproveitando recursos de aceleração disponíveis para reduzir o tempo de treinamento, consideravelmente maior em uma RNN quando comparado com classificadores clássicos.

3.3 PLATAFORMAS EMBARCADAS

Para este trabalho, foram selecionados dois computadores de placa única - *Single Board Computer* (SBC), em inglês - para embarcar os modelos treinados, a fim de avaliar seus desempenhos em plataformas com poder computacional limitado, simulando um ambiente de tempo real como uma IVN.

3.3.1 VisionFive 2

A VisionFive 2 é uma SBC baseada na arquitetura RISC-V, fabricada pela StarFive, empresa chinesa, fundada em 2018, que atua no ecossistema de hardware e software baseado em

RISC-V. a placa conta com um processador dual-core ou quad-core (no caso na placa utilizada) SiFive U74, oferecendo suporte a velocidades de clock de até 1,5 GHz (STARFIVE, 2024).

A SBC também possui GPU integrada, suporte a aceleração de vídeo e conectividade robusta, incluindo portas HDMI, Ethernet Gigabit, e USB, além de um slot para cartão microSD. Projetada, principalmente, para aplicações em sistemas embarcados, IA e Internet das Coisas (IoT), a placa da StarFive oferece compatibilidade com sistemas operacionais Linux e suporte a ferramentas de código aberto. O Apêndice A aborda mais detalhes sobre as especificações da VisionFive 2.

3.3.2 Raspberry Pi 3 Model B

A Raspberry Pi 3 Model B é uma SBC, baseada na arquitetura ARM, desenvolvida pela Raspberry Pi Foundation, uma instituição britânica sem fins lucrativos fundada em 2009 com o objetivo de promover o ensino de computação básica. A SBC conta com um processador quad-core Broadcom BCM2837 de 64 bits, oferecendo suporte a velocidades de clock de até 1,2 GHz (FOUNDATION, 2025).

Assim como a VisionFive 2, a placa também possui conectividade integrada, incluindo suporte a Wi-Fi, Bluetooth Low Energy (BLE), portas HDMI e Ethernet, além de múltiplas portas USB e um slot para cartão microSD. Projetada, inicialmente, para fins educacionais, a plataforma expandiu seu uso para aplicações em sistemas embarcados, automação residencial e projetos de Internet das Coisas (IoT), oferecendo compatibilidade com diversos sistemas operacionais baseados em Linux. O Apêndice B aborda mais detalhes sobre as especificações de hardware da Raspberry Pi 3 Model B.

4 AVALIAÇÃO

A quinta etapa do CRISP-DM é a validação, em que se avalia o modelo, verificando se os resultados obtidos pelos algoritmos de ML atendem aos objetivos definidos na primeira etapa: entendimento do negócio. Uma avaliação criteriosa assegura que o modelo não apenas funcione bem em ambiente de teste, mas também seja robusto, confiável e capaz de gerar valor prático na aplicação final.

4.1 MÉTRICAS E MÉTODOS DE AVALIAÇÃO E VALIDAÇÃO

Para validação dos resultados, foram adotadas as métricas de acurácia, precisão, *recall* (sensibilidade) e *F1-score*, conforme trabalhos correlatos. A escolha dessas métricas fornece uma avaliação compreensível em cenários onde a distribuição das classes é desbalanceada ou o custo de erros de classificação varia entre as classes. Ao utilizar essas métricas, conseguimos capturar de forma mais precisa o desempenho detalhado do nosso modelo de classificação (ANJUM et al., 2022).

A escolha dessas métricas também permite comparar nossos resultados com os de outros trabalhos da literatura que usaram métricas diferentes, como a Taxa de Falsos Alarmes (FAR, do inglês *False Alarm Rate*) em vez de precisão, já que uma métrica pode ser inferida a partir da outra (PUROHIT; GOVINDARASU, 2022). A principal métrica utilizada foi o *F1-score* (Equação 2.4). O *F1-score* é a média harmônica da precisão (Equação 2.2) e do *recall* (Equação 2.3), representando simetricamente ambas as métricas em uma única medida.

4.1.1 Validação de Generalização dos Modelos

Como descrito na Subseção 2.1.5, para avaliar um bom modelo, não basta apenas verificar seus resultados nas principais métricas de desempenho; é preciso realizar um processo de validação do modelo a fim de garantir que ele tenha uma boa capacidade de generalização, o que significa que ele não tem um bom desempenho apenas com dados conhecidos, mas também com dados não apresentados a ele no seu treinamento. Sendo assim, como feito em outros trabalhos de ML do LISHA, realizados ao longo do projeto IASE, para garantir que os modelos propostos não sofram de *overfitting*, foi utilizada a *cross-validation* (validação cruzada) (CANAL; RIFFEL; GRACIOLI, 2023; CANAL et al., 2023; CANAL et al., 2024).

O método de particionamento utilizado neste trabalho, conhecido como *k-fold*, divide o conjunto de dados em k subconjuntos mutuamente exclusivos e de tamanhos iguais. Em cada iteração, um desses subconjuntos é reservado para teste, enquanto os $k - 1$ restantes são usados para treinar o modelo e estimar seus parâmetros. A acurácia do modelo é então calculada. Esse processo é repetido k vezes, alternando o subconjunto usado para teste de forma rotativa.

Ao final das k iterações, a acurácia média é calculada com base nos erros obtidos em cada rodada, seguindo a Equação 2.5, fornecendo uma avaliação mais confiável da capacidade

do modelo de generalizar e representar o processo gerador dos dados. Isso garante que a acurácia, precisão e sensibilidade do modelo não sejam apenas resultado de ajuste a um conjunto de dados específico, mas que ele possa generalizar bem para novos dados desconhecidos.

Além da *cross-validation*, os modelos treinados com os *datasets* do HCRL foram avaliados nos conjuntos de dados do IASE, com ataques sintetizados, descritos na Subseção 3.1.2, a fim de validar a generalização dos modelos a dados obtidos de fontes (veículos) diferentes.

4.2 AVALIAÇÃO DE DESEMPENHO

Para validar o desempenho dos modelos de classificação propostos, foi realizada a divisão simples dos *datasets* do HCRL (LEE; JEONG; KIM, 2017) em *train-test*. Cada conjunto de teste correspondeu a 30% do total de dados, e calculou-se a média do *F1-score* a partir dos *F1-scores* das amostras livres de ataque e das amostras sob ataque. A Tabela 2 apresenta uma comparação entre os resultados de classificação obtidos para os ataques de tipo DoS pelos classificadores clássicos. Já a Tabela 4 mostra a comparação para os resultados de ataques de *Impersonation*. A Tabela 3 apresenta o *F1-score* da rede LSTM para quatro diferentes valores de *time_steps* usados no treinamento da rede para detectar ataques DoS, enquanto a Tabela 5 mostra os resultados da RNN para ataques de *Impersonation*.

É importante frisar que neste trabalho, em vez de apenas replicar o IDS desenvolvido pelo HCRL, os resultados obtidos com os algoritmos propostos foram comparados com os principais trabalhos correlatos (ALSHAMMARI et al., 2018; ANJUM et al., 2022; GUNDU; MALEKI, 2022; PUROHIT; GOVINDARASU, 2022). Também é importante ressaltar que os resultados foram extraídos diretamente desses trabalhos, não houve replicação dos experimentos por eles apresentados.

Ainda, ressalta-se que, neste trabalho, os modelos foram treinados para cada tipo de ataque separadamente, de forma semelhante à abordagem de Anjum et al., enquanto os demais adotaram uma abordagem mais genérica no treinamento de seus algoritmos, treinando um mesmo modelo para classificar tanto ataques de DoS quanto ataques de *Impersonation* (ALSHAMMARI et al., 2018; GUNDU; MALEKI, 2022; PUROHIT; GOVINDARASU, 2022).

4.2.1 DoS Attack

A Tabela 2 divide os resultados do experimento em duas categorias: com e sem o atributo 'ID', o identificador da ECU que enviou a mensagem. Para a detecção de ataques DoS, treinamos dois conjuntos de modelos diferentes. No conjunto "com ID", os identificadores das ECUs estavam incluídos nas instâncias das mensagens como parte do conjunto de treinamento, enquanto no conjunto "sem ID", os IDs das ECUs foram removidos do conjunto de *features* do *dataset*. Conforme visto na tabela, mesmo sem os identificadores das mensagens, os modelos propostos demonstraram bom desempenho na identificação dos ataques DoS. Apesar disso, o desempenho de todos os modelos foi igual o superior quando munidos da *feature* de identi-

cação, visto que esse atributo tem relevância muito alta para ataques do tipo DoS, já que eles foram injetados no barramento com um ID único igual a 0.

Algoritmo	(Este Trabalho, 2025)		A	B	C	D
	com ID	sem ID				
KNN	99,97%	96,34%	93,5%	70%	-	-
DT	100%	94,35%	-	-	96,1%	-
RF	100%	95,02%	-	94%	94,3%	-
XGBoost	100%	96,39%	-	92%	97,2%	94,74%
K-Means	56,37%	56,37%	-	-	-	-

Tabela 2 – **Comparação dos resultados de *F1-score* para detecção de ataques de DoS.** Trabalhos utilizados para comparação: **A** (ALSHAMMARI et al., 2018), **B** (GUNDU; MALEKI, 2022), **C** (PUROHIT; GOVINDARASU, 2022), **D** (ANJUM et al., 2022).

Os modelos que apresentaram melhor desempenho foram o KNN, o *Decision Tree* (DT), o *Random Forest* (RF) e o *XGBoost*, todos modelos de classificação supervisionada. O experimento de treinar um modelo não supervisionado para avaliar a possibilidade de utilizar um modelo sem dados rotulados de ocorrência de ataques não foi bem sucedido. Conforme indicado pelo *F1-score* médio de 56,37% do modelo K-Means treinado, os algoritmos supervisionados de classificação se confirmaram como a abordagem ideal para essa tarefa.

Nos testes de detecção de ataques DoS, nossos modelos supervisionados superaram os quatro trabalhos correlatos (ALSHAMMARI et al., 2018; GUNDU; MALEKI, 2022; PUROHIT; GOVINDARASU, 2022; ANJUM et al., 2022). Os quatro modelos supervisionados que treinamos apresentaram ótimos resultados para ambos os ataques: *XGBoost*, RF e DT alcançaram 100% de *F1-score* quando treinados com o ‘ID’ da ECU como parte do conjunto de atributos, enquanto o KNN alcançou 99,97% de *F1-score* no teste de detecção de ataques DoS. Ao removermos a *feature* de identificação da ECU remetente, os resultados caem um pouco, embora todos se mantenham acima de 94% de *F1-score* e comparáveis ou superiores aos trabalhos de comparação.

<i>time_steps</i>	<i>F1-score</i>	
	com ID	sem ID
1	99,999%	96,360%
10	99,999%	96,362%
20	100%	96,364%
50	99,999%	96,356%

Tabela 3 – **Resultados de detecção de ataques de DoS para a rede LSTM com diferentes valores para o parâmetro *time_steps*.**

Como visto na Tabela 3, a LSTM - assim como a DT, a RF e o *XGboost* - alcançou os 100% de *F1-score*, quando configurada com *time_steps* igual a 20 e com o identificador das ECUs presente no conjunto de dados de treinamento. A variação dos resultados foi irrisória se baseada no valor do parâmetro de *time_steps*, embora haja uma clara diferença entre os

resultados dos modelos treinados com a coluna de ‘ID’ presente no *dataset* de treinamento para aqueles treinados sem essa informação, assim como observado nos demais modelos.

4.2.1.1 Generalização

A respeito da generalização dos modelos para ataques DoS, na avaliação realizada com o *dataset* de ataques DoS do IASE, os resultados foram consistentes e expressivos. O KNN obteve acurácia de até 99,98% nos testes, além de uma média de 95,0% na validação cruzada, evidenciando uma alta capacidade de generalização. Outros algoritmos, como DT, RF e XGBoost, também apresentaram desempenhos excelentes, com acurácia e *F1-score* de 100% no segundo *dataset* utilizado neste trabalho, além de médias de validação cruzada próximas de 100%, o que reflete a capacidade dos modelos em aprender padrões complexos. Esses resultados reforçam a robustez dos modelos propostos, indicando que estão aptos a identificar comportamentos anômalos de forma eficaz no contexto do tráfego CAN, mesmo quando aplicados a mensagens oriundas de veículos diferentes.

Por outro lado, a LSTM obteve resultados consideravelmente inferiores em relação aos demais modelos ao ser avaliada no conjunto de dados do IASE, alcançando um *F1-score* máximo de 77%. Esse desempenho ocorreu no cenário em que o modelo foi treinado e avaliado com a presença da *feature* de identificação (‘ID’). De modo geral, a ausência dessa informação afetou negativamente o desempenho de todos os modelos: ao remover a coluna de identificação das ECUs tanto no treinamento quanto na validação, observou-se uma queda significativa nas métricas de avaliação no segundo *dataset* e na *cross-validation*.

4.2.2 Impersonation Attack

No teste de detecção de ataques de *Impersonation*, o KNN também alcançou 99,97% de *F1-score*, mesmo resultado obtido para detecção de DoS. Todos os outros três modelos supervisionados alcançaram 99,51% de *F1-score*. Para essa classe de ataques, os modelos supervisionados propostos também superaram os resultados dos quatro trabalhos correlatos usados como comparativos. Em partes, isso se justifica pelo fato de três desses trabalhos terem usado um mesmo modelo para detectar DoS e *Impersonation* (ALSHAMMARI et al., 2018; GUNDU; MALEKI, 2022; PUROHIT; GOVINDARASU, 2022), enquanto o outro validou apenas um algoritmo (ANJUM et al., 2022). Diferente dos modelos treinados para detectar ataques de DoS, os modelos treinados para ataques de *Impersonation* não demonstraram diferença nos valores de *F1-score* até a terceira casa decimal para o treinamento realizado com e sem a coluna de ‘ID’, demonstrando uma ótima independência da identificação das ECUs para a classificação desse tipo de ataque.

Os resultados obtidos com a rede LSTM demonstram desempenho sutilmente superior ou comparável aos demais modelos tradicionais na detecção de ataques de *Impersonation*, com *F1-scores* variando entre 99,79% e 99,97%. Em especial, o desempenho máximo da LSTM

Algoritmo	A	B	C	D	E
KNN	99,97%	93,5%	70%	-	-
DT	99,51%	-	-	96,1%	-
RF	99,51%	-	94%	94,3%	-
XGBoost	99,51%	-	92%	97,2%	96,49%
K-Means	49,29%	-	-	-	-

Tabela 4 – Comparação dos resultados de *F1-score* para detecção de ataques de *Impersonation*. A: Este Trabalho (2025); B: (ALSHAMMARI et al., 2018); C: (GUNDU; MALEKI, 2022); D: (PUROHIT; GOVINDARASU, 2022); E: (ANJUM et al., 2022).

<i>time_steps</i>	<i>F1-score</i>	
	com ID	sem ID
1	99,94%	99,97%
10	99,90%	99,89%
20	99,79%	99,90%
50	99,89%	99,92%

Tabela 5 – Resultados de detecção de ataques de *Impersonation* para a rede LSTM com diferentes valores de *time_steps*.

(99,97%) supera quase todos os modelos testados, igualando o resultado do KNN (99,97%), que foi o melhor entre os métodos convencionais. Assim como os demais modelos, esta RNN não demonstrou variância discrepante entre os resultados obtidos sendo treinada com e sem a coluna de 'ID'. Além disso, é válido destacar que a LSTM mantém altos índices mesmo com o aumento de *time_steps*, mostrando sua capacidade de lidar com dependências temporais, embora neste caso não seja possível traçar uma relação entre o valor de *time_steps* e os resultados obtidos. Esses resultados evidenciam o potencial das RNNs, como a LSTM, na tarefa de detecção de ataques em redes veiculares, especialmente quando informações temporais são relevantes para a identificação do comportamento malicioso.

4.2.2.1 Generalização

No cenário de avaliação da capacidade de generalização dos modelos para ataques de *Impersonation*, os resultados obtidos por meio da validação cruzada indicam um desempenho consistente e robusto. XGBoost, DT, RF e KNN apresentaram *cross-validation score* médio superiores a 99%. Os resultados obtidos demonstram a habilidade dos modelos em generalizar bem para novos dados. A Figura 12 apresenta as curvas de aprendizado do modelo XGBoost, representando o desempenho em termos de acurácia em função do tamanho do conjunto de treinamento. Observa-se que a acurácia tanto no conjunto de treinamento quanto na *cross-validation* permanece consistentemente alta, com valores próximos a 100% para todos os tamanhos de conjunto de treinamento. Isso indica uma capacidade do modelo de generalizar bem para novos dados, sem sinais de *overfitting* evidentes até os limites avaliados. A consistência entre as curvas também reforça a estabilidade do modelo durante o processo de treinamento.

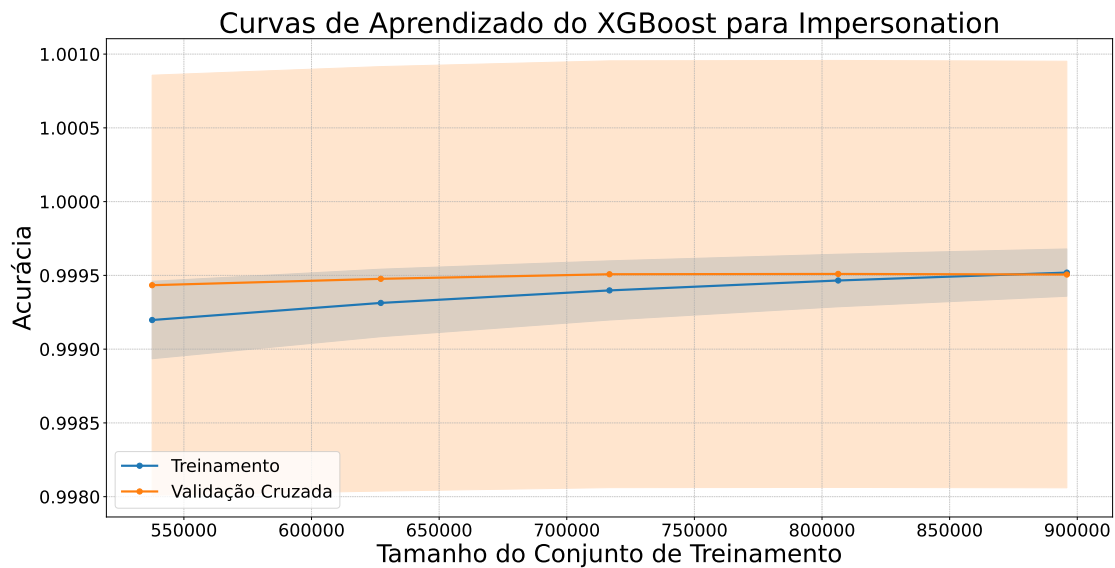


Figura 12 – Curvas de aprendizado do modelo XGBoost para o conjunto de dados de ataques de *Impersonation*.

Além da avaliação da generalização pelo método da validação cruzada, os modelos treinados no *dataset* de ataques de *Impersonation* do HCRL também foram avaliados no conjunto de dados do IASE com ataques desse mesmo tipo sintetizados. Os resultados foram bons, porém comparados aos resultados obtidos para o DoS (descritos na Subseção 4.2.1.1), houve uma nítida queda. Os quatro classificadores clássicos obtiveram resultados muito próximos, obtendo 73%, 86% e 79% de precisão, *recall* e *F1-score* médios, além de uma acurácia média também de 86%. Esses resultados desse experimento de generalização podem ser considerados bons, dado que os dois conjuntos usados no experimento foram extraídos de dois veículos diferentes, com ECUs diferentes, que comunicam dados diferentes, porém, se tratando de uma aplicação de segurança, os resultados não validam a utilização dos modelos treinados com o *dataset* extraído de um carro em outro. As redes LSTM também apresentaram um desempenho inferior nesse experimento, com 43% de precisão, 50% de *recall* e 46% de *F1-score*.

4.3 AVALIAÇÃO DE TEMPO DE EXECUÇÃO

Mais do que bons resultados de acurácia, precisão e sensibilidade, é necessário verificar se os modelos podem ser executados de forma viável no contexto automotivo. Nesta seção, o tempo de execução dos modelos embarcados, se aproximando mais de um cenário realista em que eles seriam executados diretamente na ECU, processando os dados da CAN conforme recebidos, por exemplo, via *sniffer*.

4.3.1 Tempo de Treino e Teste

Antes da validação do tempo de execução dos modelos propostos, foram avaliados os tempos de treino e teste em três sistemas diferentes: as duas SBCs selecionadas para o *embedded* dos modelos, tanto a VisionFive 2 quanto a Raspberry Pi 3 Model B, executando um sistema Debian 12 e; um notebook com processador Intel Core i5-12500H de 12ª geração com 16 GB de RAM, executando o sistema Ubuntu 22.04.3 LTS. A Tabela 6 mostra o resultado coletado para o tempo de treino e teste dos modelos no notebook, enquanto a Tabela 7 mostra a comparação dos tempos de execução dos modelos de treinamento e teste nos dois computadores de placa única para o conjunto de dados HCRL. O tempo foi medido com a biblioteca *time* do Python.

Tempo de Treino e Teste dos Modelos com Intel Core i5-12500H		
Algoritmo	Ataques de DoS	Ataques de Impersonation
KNN	0,64 s	1,15 s
DT	0,75 s	0,25 s
RF	28,76 s	11,04 s
XGBoost	0,41 s	0,57 s
K-Means	1,24 s	1,35 s

Tabela 6 – Tempo, em segundos (s), de treino e teste combinados de cada algoritmo para os *datasets* do HCRL de ataques dos tipos DoS e *Impersonation*.

Algoritmo	Plataforma			
	Raspberry Pi 3 Model B		VisionFive 2	
	Ataques de DoS	Ataques de <i>Impersonation</i>	Ataques de DoS	Ataques de <i>Impersonation</i>
KNN	17,24 s	29,34 s	11,67 s	20,38 s
DT	14,42 s	13,30 s	25,00 s	8,78 s
RF	718,93 s	333,81 s	1161,57 s	227,66 s
XGBoost	274,64 s	476,59 s	156,68 s	289,14 s
K-Means	2,70 s	4,16 s	2,55 s	3,77 s

Tabela 7 – Comparação do tempo, em segundos (s), de treino e teste combinados de cada algoritmo para os *datasets* do HCRL de ataques dos tipos DoS e *Impersonation* nas duas plataformas embarcadas selecionadas.

O tempo de treino e testes na SBC RISC-V foi satisfatoriamente curto, o que indica a possibilidade de retreinamento dos modelos diretamente em ambiente embarcado, descartando a necessidade de utilizar um equipamento mais robusto como um notebook para a realização dessa tarefa. Como visto na Tabela 7, alguns modelos levaram menos de um minuto para serem treinados e testados no sistema embarcado, como por exemplo a DT, que levou menos de 9 segundos para ser treinado e testado em um conjunto de dados com quase um milhão de instâncias (*dataset* com ataques de *Impersonation*). Além disso, com exceção do modelo de RF para o ataque DoS, todos os modelos levaram menos de 5 minutos para serem treinados na plataforma embarcada.

4.3.2 Avaliação do tempo de treino, teste e classificação da LSTM

O tempo de treino e teste da rede LSTM também foi avaliado. Por conta do seu maior tempo de treinamento, esse experimento foi realizado apenas no notebook citado anteriormente, que conta com uma GPU NVIDIA GeForce RTX 3050 Mobile. A Tabela 8 indica o tempo combinado de treino e teste levado para cada configuração de parâmetros e *datasets* utilizados para a LSTM.

time_steps	DoS		Impersonation	
	com ID	sem ID	com ID	sem ID
1	531,8 s	1038,69 s	1840,22 s	3374,42 s
10	1001,84 s	1900,36 s	924,17 s	1233,41 s
20	489,4 s	614,33 s	1092,12 s	1276,35 s
50	730,43 s	1503,13 s	1170,76 s	1586,93 s

Tabela 8 – Comparação dos tempos de treino de cada configuração de *time_steps* usada no treinamento da rede LSTM com os *datasets* do HCRL de ataques dos tipos DoS e *Impersonation*, com e sem a *feature* de identificação das ECUs, medidos em segundos (s).

Como esperado, a Tabela 8 denota um tempo de treinamento muito superior para a rede neural quando comparada com os modelos de classificação ditos clássicos. Mesmo com treinamento realizado em uma GPU, o menor tempo de treinamento para uma rede LSTM nos *datasets* do HCRL foi superior a 8 minutos. Embora esse não seja um tempo considerado longo no treinamento de RNNs, é um tempo consideravelmente maior do que o levado no treinamento dos demais modelos, mesmo quando comparado com alguns dos modelos treinados nas plataformas embarcadas. Também é salientável que não há uma correlação notável entre o aumento de *time_steps* e o tempo de treinamento do modelo, tendo os menores tempos de treino e teste sendo apresentados com o parâmetro configurado em valores intermediários, e não no menor ou maior valor dado ao *time_steps*.

Realizado o treinamento da rede com quatro diferentes configurações para o parâmetro de *time_steps*, foi feita uma validação do seu tempo de classificação para instâncias no dataset do HCRL, a fim de verificar a possibilidade de seu *embedded* como feito com os demais modelos. A Figura 13 mostra o tempo médio de classificação de uma única mensagem para as diferentes configurações de LSTM treinadas para o ataque de tipo DoS. Já a Figura 14 mostra a média do tempo de classificação de uma instância para o ataque de *Impersonation*.

Ao analisar-se as Figuras 13 e 14, nota-se que não apenas o tempo levado pela LSTM em seu período de treino e teste foi muito superior aos demais modelos como também o seu tempo de classificação de mensagens. Como um comparativo, o tempo de classificação da DT, no notebook com processador i5, foi em média 0,12 milissegundos (ms) para cada instância, tanto no modelo de detecção de DoS quanto no modelo de detecção de *Impersonation*. Comparativamente, mesmo sendo processada em uma GPU NVIDIA GeForce RTX 3050 Mobile, a RNN tem um tempo de classificação muito superior inclusive a quatro dos cinco modelos em-

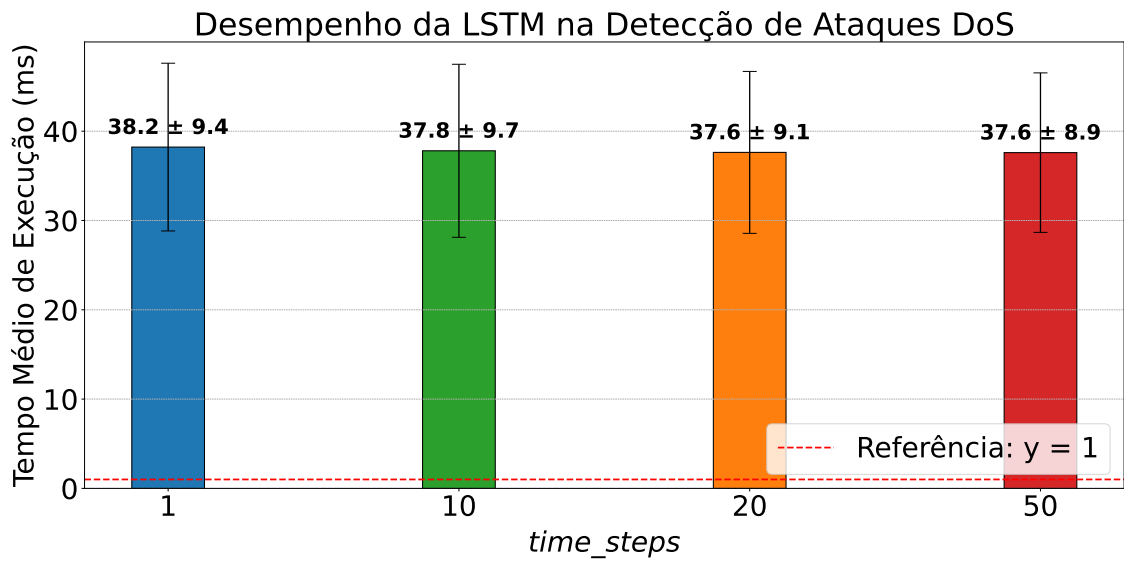


Figura 13 – Tempo médio de classificação de uma mensagem como normal ou ataque DoS em milissegundos (ms) para as diferentes configurações da LSTM. A legenda do gráfico ressalta a periodicidade do barramento CAN de 1 ms.

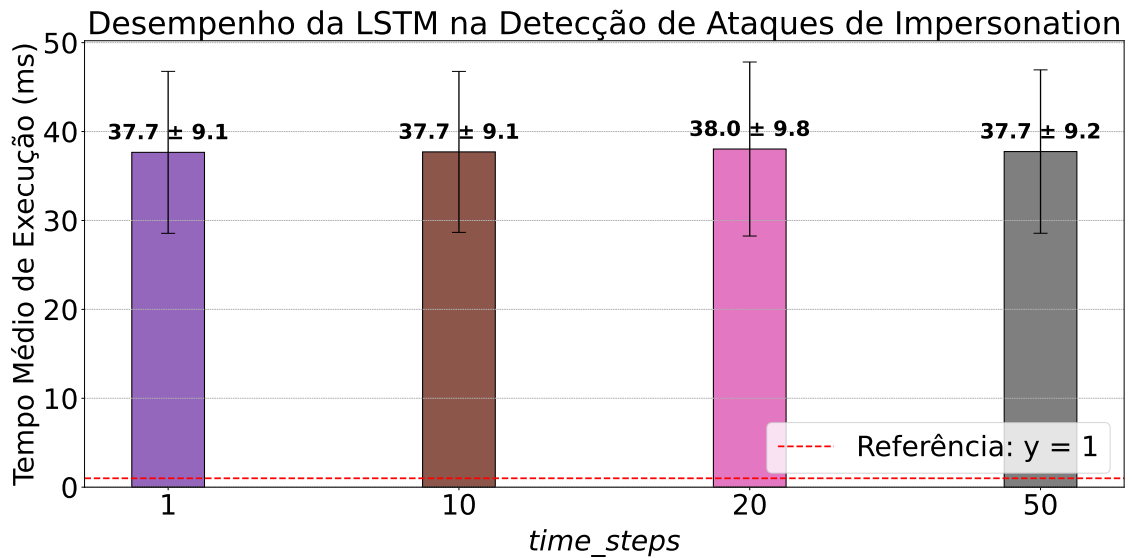


Figura 14 – Tempo médio de classificação de uma mensagem como normal ou ataque de *Impersonation* em milissegundos (ms) para as diferentes configurações da LSTM. A legenda do gráfico ressalta a periodicidade do barramento CAN de 1 ms.

barcados na VF2, chegando a ser mais de 30 vezes mais lenta que a DT embarcada na média, mesmo sendo avaliada em um hardware superior, como é possível ver na Tabela 9.

Após a análise dos tempos de treino, teste e classificação das redes LSTM desenvolvidas — conduzida em um hardware significativamente mais potente do que ambas as SBCs, a implementação embarcada das RNNs foi desconsiderada. Essa decisão fundamenta-se não

apenas na inviabilidade prática observada, mas também no Princípio da Navalha de Ockham, discutido no final da Seção 2.1 deste trabalho, que orienta a escolha por soluções mais simples quando alternativas oferecem desempenho equivalente (MURPHY, 2012). Isso também se mostrou na validação da generalização dos modelos, vista nas Subseções 4.2.1.1 e 4.2.2.1, onde é possível ver uma capacidade de generalização da LSTM inferior aos demais modelos. Sendo assim, optou-se pelo *embedded* apenas dos classificadores clássicos.

4.3.3 Tempo de Execução Embarcada

De acordo com o fluxo do CRISPM-DM, visto na Figura 5, embora alguns modelos já tenham sido descartados para uma implantação em uma cenário real - como o K-Means, por seu baixo desempenho, e a LSTM, por sua elevada complexidade e tempo de classificação -, no contexto deste trabalho, a análise do tempo de execução embarcada dos modelos ainda faz parte da quinta etapa do CRISP-DM: a de validação (SHEARER, 2000).

4.3.3.1 VisionFive 2

O tempo de classificação para uma instância (uma mensagem CAN) foi avaliado com os modelos embarcados na placa RISC-V. Para essa avaliação, cada experimento foi repetido 1000 vezes, variando a entrada para o classificador (1, 100 e 1000 amostras de dados de entrada). A execução do experimento 1000 vezes garante a confiabilidade dos resultados, reduzindo as variações aleatórias e fornecendo uma estimativa mais precisa do desempenho do modelo. Para as 100 e 1000 amostras de dados de entrada, calculou-se a média dividindo o tempo total pelo número de instâncias.

O desvio padrão, mostrado nas barras de erro das Figuras 15 e 16, corrobora a consistência dos resultados, com um baixo desvio padrão, que indica tempos de classificação estáveis entre as repetições dos testes. A DT, o KNN e o XGBoost são os algoritmos com os melhores resultados para essa métrica. Como destaque, a DT leva menos de 1,2 milissegundos (ms) para classificar uma mensagem como maliciosa ou não para ambos os tipos de ataque, mesmo quando fornecemos apenas uma única instância para o classificador. Esse resultado é interessante, uma vez que a periodicidade do barramento CAN é 1 ms. Ou seja, o modelo consegue classificar uma mensagem quase simultaneamente ao disparo de uma nova mensagem no barramento.

Ao observar a Tabela 9, é possível notar uma redução drástica no tempo médio para classificar uma mensagem conforme se fornece mais instâncias para o preditor. Essa discrepância pode se justificar por fatores como:

1. **Overhead de Inicialização para Cada Instância:** ao fornecer uma única instância para o modelo, o processo de classificação pode envolver algum tipo de *overhead* de inicialização que é mais notável quando o modelo é chamado repetidamente para cada instância.

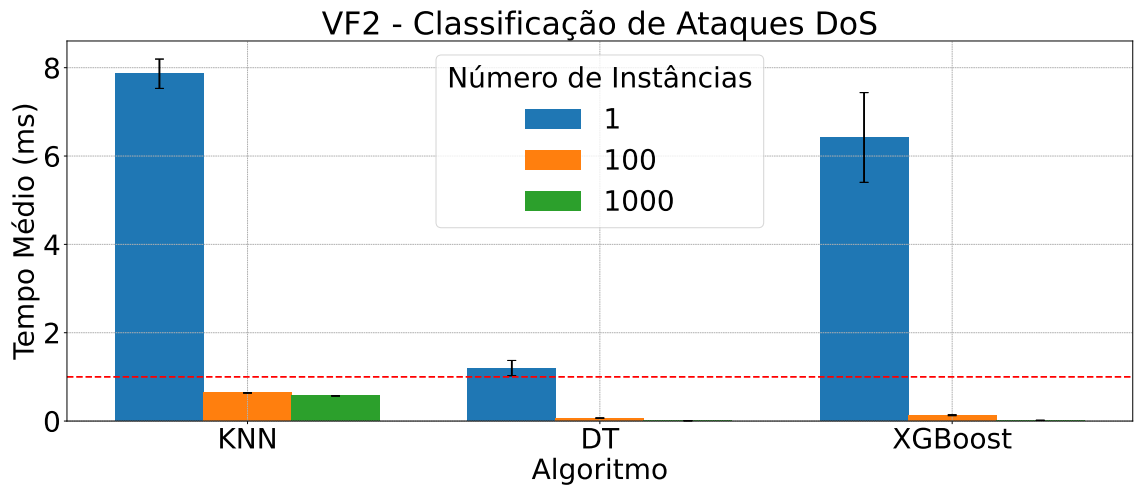


Figura 15 – Tempo médio de classificação de uma mensagem como normal ou ataque DoS em milissegundos (ms) na SBC VisionFive 2. A legenda da figura descreve quantas instâncias foram fornecidas ao modelo.

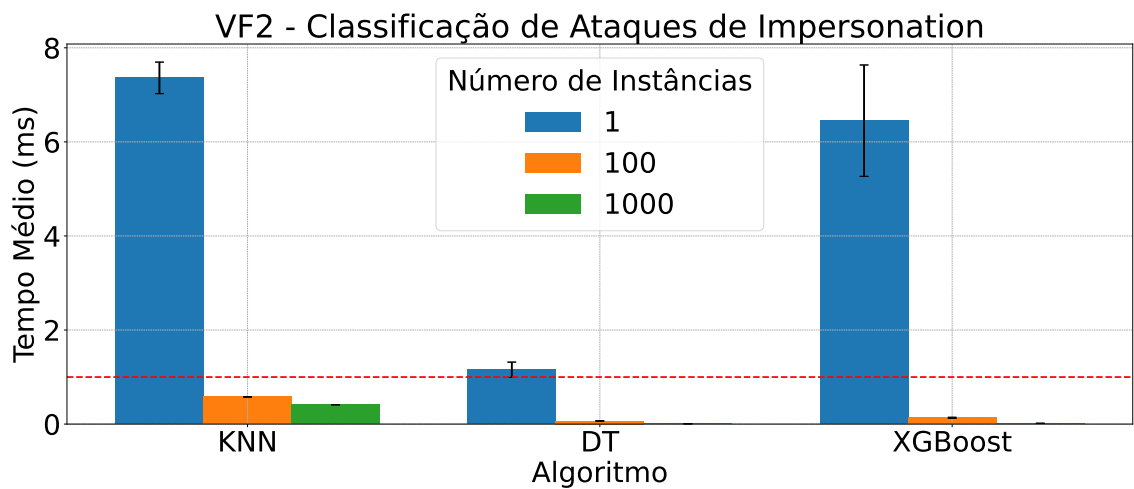


Figura 16 – Tempo médio de classificação de uma mensagem como normal ou ataque de *impersonation* em milissegundos (ms) na SBC VisionFive 2. A legenda da figura descreve quantas instâncias foram fornecidas ao modelo.

Esse *overhead* é amortizado quando se fornece várias instâncias de uma vez, evitando o custo de inicializar ou preparar o modelo para cada instância individual;

2. **Operações em Lote (*Batch Processing*):** muitos modelos de ML, são otimizados para trabalhar em lotes de dados. Quando fornecidas 1000 instâncias ao mesmo tempo, o mo-

Algoritmo	Info.	Tempo para Classificar Cada Mensagem como Normal ou Ataque na VF2					
		DoS			Impersonation		
		Instância única	100 instâncias	1000 instâncias	Instância única	100 instâncias	1000 instâncias
KNN	Faixa	7,507 - 10,614	0,59 - 0,701	0,565 - 0,58	7,02 - 10,235	0,532 - 0,6	0,409 - 0,419
	Média	7,863	0,637	0,569	7,361	0,578	0,412
	Desv. pad.	0,033	0,006	0,001	0,335	0,004	0,001
DT	Faixa	1,016 - 3,033	0,06 - 0,078	0,006 - 0,009	0,998 - 3,066	0,059 - 0,103	0,006 - 0,011
	Média	1,199	0,067	0,007	1,155	0,066	0,007
	Desv. pad.	0,174	0,004	< 0,001	0,163	0,005	< 0,001
RF	Faixa	55,063 - 62,472	0,623 - 2,654	0,089 - 0,095	55,299 - 62,069	0,593 - 2,63	0,071 - 0,077
	Média	57,843	0,634	0,09	57,499	0,605	0,072
	Desv. pad.	0,448	0,064	< 0,001	0,438	0,064	< 0,001
XGBoost	Faixa	5,474 - 11,028	0,123 - 0,254	0,014 - 0,027	5,451 - 17,631	0,122 - 0,275	0,014 - 0,026
	Média	6,42	0,133	0,016	6,451	0,132	0,016
	Desv. pad.	1,016	0,011	0,002	1,184	0,013	0,002

Tabela 9 – Tempo de classificação embarcada na VisionFive 2 em milissegundos (ms).

delo pode processá-las simultaneamente, aproveitando melhor a arquitetura do hardware (por exemplo, a utilização de GPUs ou o paralelismo de múltiplos núcleos da CPU);

3. **Paralelismo e Vetorização:** o modelo pode utilizar otimizações baseadas em paralelismo e vetorização ao lidar com múltiplas instâncias;
4. **Caches e Previsões em Lote:** algumas implementações de modelos de ML podem usar *caches* internos ou otimizações específicas para classificações em lote. Quando enviamos várias instâncias de uma vez, o modelo é capaz de agrupar as operações de cálculo de maneira mais eficiente.

4.3.3.2 Raspberry Pi 3 Model B

O processo de experimentação adotado para a Raspberry Pi 3 Model B (RP3B) foi o mesmo da VF2. A Tabela 10 apresenta os resultados dos experimentos realizados com os modelos clássicos embarcados na segunda SBC abordada neste trabalho. A análise dos dados denota um aumento significativo no tempo de classificação de todos os algoritmos quando comparado com suas execuções experimentadas na VF2. A DT, modelo mais eficiente em ambas as SBCs, mais que dobrou seu tempo de classificação médio na RP3B quando comparado com os experimentos realizados na VF2.

Apesar do aumento do tempo médio de classificação em todos os modelos, a RP3B ainda demonstrou um desempenho satisfatório para uma plataforma embarcada que conta com uma memória RAM de 1GB e não ostenta uma GPU como a VF2. Assim como ocorrido com a primeira SBC analisada, ao processarmos uma quantidade maior de mensagens CAN por vez nos modelos embarcados na RP3B, o tempo médio de classificação de cada mensagem diminuiu significativamente, permitindo um maior processamento de dados por uma dada unidade de tempo.

As Figuras 17 e 18, que demonstram o tempo médio de classificação de uma instância do conjunto de dados de ataques de DoS e de *Impersonation*, respectivamente, para os três algoritmos com melhor tempo de classificação. Os gráficos também carregam outras informa-

Algoritmo	Info.	Tempo para Classificar Cada Mensagem como Normal ou Ataque na RP3B					
		DoS			Impersonation		
		Instância única	100 instâncias	1000 instâncias	Instância única	100 instâncias	1000 instâncias
KNN	Faixa	10,434 - 541,516	0,729 - 1,435	0,802 - 1,545	9,650 - 36,277	0,648 - 0,923	0,510 - 0,665
	Média	11,181	0,793	0,808	10,007	0,721	0,518
	Desv. pad.	1,678	0,029	0,032	0,897	0,014	0,007
DT	Faixa	2,123 - 6,067	0,079 - 0,179	0,009 - 0,018	2,109 - 14,746	0,077 - 0,162	0,009 - 0,017
	Média	2,674	0,110	0,012	2,685	0,114	0,012
	Desv. pad.	0,660	0,023	0,003	0,849	0,022	0,003
RF	Faixa	68,080 - 107,195	0,695 - 3,489	0,120 - 0,196	69,735 - 86,796	0,670 - 0,738	0,087 - 0,135
	Média	69,239	0,713	0,122	70,806	0,682	0,089
	Desv. pad.	1,884	0,089	0,004	0,869	0,006	0,002
XGBoost	Faixa	9,014 - 21,192	0,106 - 0,188	0,014 - 0,023	8,899 - 13,165	0,104 - 0,189	0,013 - 0,019
	Média	9,416	0,109	0,014	9,174	0,106	0,013
	Desv. pad.	0,893	0,007	0,001	0,156	0,003	0,0002

Tabela 10 – Tempo médio de classificação embarcada na Raspberry Pi 3 Model B em milissegundos (ms).

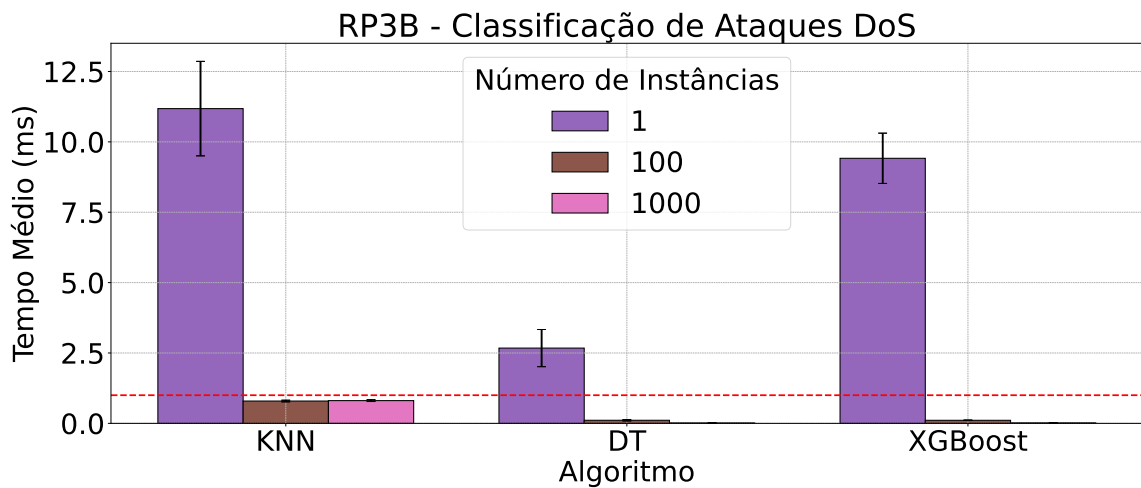


Figura 17 – Tempo médio de classificação de uma mensagem como normal ou ataque DoS em milissegundos (ms) na Raspberry Pi 3 Model B. A legenda da figura descreve quantas instâncias foram fornecidas ao modelo.

ções como o desvio padrão e a periodicidade do barramento CAN (marcada pela linha vermelha tracejada, indicando a marca de 1 ms). Inclusive, é notável um desvio padrão muito maior em relação aos resultados apresentados pela VF2 nas Figuras 15 e 16.

4.4 DISCUSSÃO DOS RESULTADOS

Concluída a experimentação, foram identificados pontos importantes sobre a eficácia e a aplicabilidade dos modelos de ML propostos para detecção dos ataques DoS e de *Impersonation* na rede CAN. Os resultados mostraram que os classificadores supervisionados, especialmente a DT, o KNN, a RF e o XGBoost, tiveram desempenho superior, com *F1-score* acima de 99% para ataques DoS e *Impersonation*, superando trabalhos anteriores. Por outro lado, o

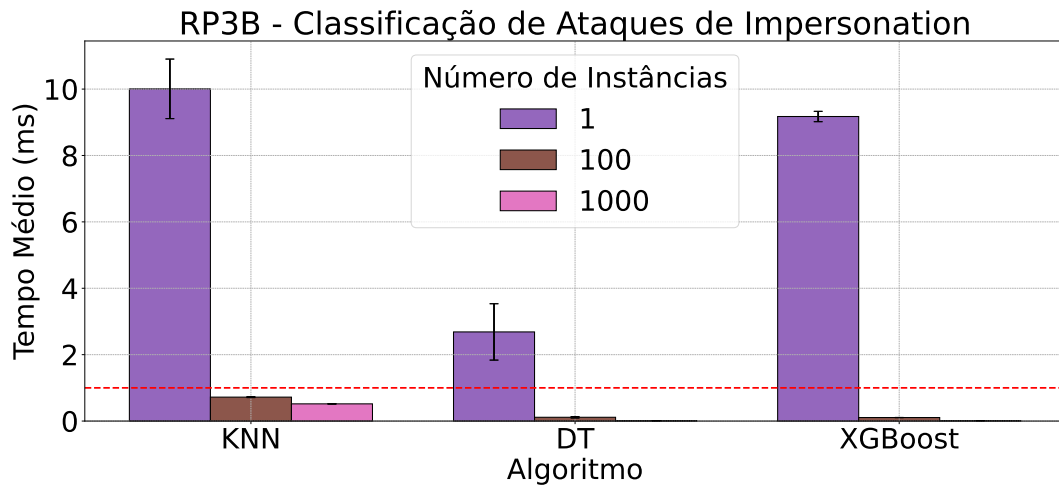


Figura 18 – Tempo médio de classificação de uma mensagem como normal ou ataque de *Impersonation* em milissegundos (ms) na Raspberry Pi 3 Model B. A legenda da figura descreve quantas instâncias foram fornecidas ao modelo.

K-Means, não supervisionado, apresentou desempenho insatisfatório, e a rede LSTM, apesar da boa classificação, teve baixa capacidade de generalização e alto custo computacional, o que inviabiliza seu uso embarcado.

Ainda, a avaliação da execução embarcada aponta para a viabilidade de rodar os modelos diretamente em ECUs. Mesmo com o *overhead* associado à execução em SBCs que operam com sistemas operacionais contendo outros processos ativos, como a própria interface gráfica, os modelos apresentaram tempos de resposta satisfatórios. Embora a classificação de mensagens individuais possa desafiar o intervalo típico de 1 ms da CAN, a estratégia de processamento em pequenos lotes (*batches*) reduziu o tempo médio de classificação para a ordem de microssegundos. Essa abordagem permite uma detecção compatível com a aplicação veicular, especialmente com o modelo de DT. Para futuras aplicações práticas, a migração dos modelos de Python para linguagens mais eficientes, como C ou C++, pode otimizar ainda mais o desempenho, consolidando a viabilidade do IDS embarcado.

5 CONCLUSÃO

Este TCC teve como objetivo principal avaliar o uso de algoritmos de ML para a detecção de ataques em IVNs baseadas no protocolo CAN, com foco nos ataques de negação de serviço (DoS) e *Impersonation*. A partir de uma abordagem metodológica fundamentada na CRISP-DM e respaldada por uma revisão bibliográfica aprofundada, foi possível alcançar esse objetivo por meio da construção e validação de modelos eficazes de detecção.

Os resultados obtidos indicam que, com um pré-processamento adequado e uma seleção criteriosa de algoritmos, é possível atingir níveis de desempenho comparáveis ou superiores aos encontrados na literatura. Isso se aplica tanto a classificadores clássicos, como *Decision Tree* e KNN, quanto a modelos mais complexos, como redes neurais recorrentes, como a LSTM. A experimentação com diferentes tipos de ataques e plataformas embarcadas reforça a viabilidade da aplicação prática desses modelos em ambientes reais.

Apesar dos resultados promissores, o trabalho também identificou limitações importantes. A complexidade computacional de certos algoritmos pode comprometer sua aplicação em tempo real em sistemas embarcados com recursos restritos. Além disso, os modelos apresentaram desempenho inferior ao generalizar entre dados de veículos distintos, evidenciando a necessidade de abordagens mais robustas para cenários heterogêneos.

Como principais contribuições deste trabalho, destacam-se:

- A avaliação comparativa de diferentes algoritmos de ML na detecção de ataques DoS e *Impersonation*;
- A demonstração de que modelos relativamente simples podem atingir bons resultados;
- A experimentação da implantação em plataformas embarcadas;
- A criação de um *pipeline* de detecção reutilizável;
- A análise da relevância de *features* específicas, como o ID das mensagens, nos resultados dos modelos.

Conclui-se, portanto, que técnicas de aprendizado de máquina têm forte potencial para compor soluções eficientes de segurança veicular. Este trabalho contribui para o avanço dessa área, oferecendo uma base sólida para estudos futuros voltados à robustez, generalização e aplicação em cenários do mundo real.

Por fim, é válida uma menção ao trabalho "*Machine Learning-Based Intrusion Detection for Automotive CAN Networks on Embedded Platforms*", escrito a partir dos resultados preliminares deste TCC e premiado como melhor trabalho do *XIV Symposium on Computing Systems Engineering*, em 2024 (BONOMO et al., 2024).

5.1 TRABALHOS FUTUROS

A partir do desenvolvido, alguns trabalhos e estudos foram pensados, dentre eles:

- Avançar para a implementação prática dos melhores modelos obtidos, avaliando seu desempenho em um cenário real. Essa etapa envolveria a injeção controlada de ataques de DoS e *Impersonation* no veículo de testes do projeto IASE, possibilitando a validação dos modelos em condições mais próximas das situações reais encontradas em redes automotivas.
- Expandir o escopo de detecção para outros tipos de ataques, como *Fuzzy*, enriquecendo a abrangência da solução.
- Otimizar a abordagem proposta, explorando a implementação dos modelos em linguagens de programação mais eficientes, como C++, buscando viabilizar aplicações embarcadas em sistemas críticos de tempo real, típicos do contexto de segurança veicular.

REFERÊNCIAS

- ALSHAMMARI, A. et al. Classification approach for intrusion detection in vehicle systems. **Wireless Engineering and Technology**, Scientific Research Publishing, v. 9, n. 4, p. 79–94, 2018.
- ANJUM, A. et al. In-vehicle network anomaly detection using extreme gradient boosting machine. In: **11th MECO**. [S.l.: s.n.], 2022. p. 1–6.
- BARBOSA, G. et al. Segurança em redes 5g: Oportunidades e desafios em detecção de anomalias e previsão de tráfego baseadas em aprendizado de máquina. In: _____. [S.l.]: SBC, 2021. p. 145–189. ISBN 9786587003658.
- BEDRECHUK, J. P. et al. Low-cost data acquisition system for automotive electronic control units. **Sensors**, v. 23, n. 4, 2023. ISSN 1424-8220.
- BISHOP, C. M. **Pattern Recognition and Machine Learning**. New York: Springer, 2006.
- BONOMO, J. P. A. et al. Machine learning-based intrusion detection for automotive can networks on embedded platforms. In: **2024 XIV Brazilian Symposium on Computing Systems Engineering (SBESC)**. [S.l.: s.n.], 2024. p. 1–6. ISSN 2324-7894.
- CANAL, R. et al. Torque regression using machine learning techniques in automotive ecus. **Design Automation for Embedded Systems**, 2024. Disponível em: <https://doi.org/10.1007/s10617-024-09289-9>.
- CANAL, R. et al. Misfire detection in combustion engines using machine learning techniques. In: **XIII SBESC**. [S.l.: s.n.], 2023. p. 1–6.
- CANAL, R.; RIFFEL, F. K.; GRACIOLI, G. Driving profile analysis using machine learning techniques and ecu data. In: **2023 IEEE 32nd International Symposium on Industrial Electronics (ISIE)**. [S.l.: s.n.], 2023. p. 1–6.
- CHO, K.-T.; SHIN, K. G. Fingerprinting electronic control units for vehicle intrusion detection. In: **25th USENIX Security Symposium (USENIX Security 16)**. Austin, TX: USENIX Association, 2016. p. 911–927. ISBN 978-1-931971-32-4. Disponível em: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/cho>.
- CHO, P. H. **XGBoost Documentation**. 2022. Acesso em: 06 dez. 2024. Disponível em: <https://xgboost.readthedocs.io/en/stable/>.
- FOUNDATION, R. P. **Raspberry Pi 3 Model B**. 2025. Acesso em: 7 maio 2025. Disponível em: <https://www.raspberrypi.com/products/raspberry-pi-3-model-b/>.
- GREENBERG, A. **Hackers remotely kill a Jeep on the highway—with me in it**. WIRED, 2015. Acesso em: 04 dez. 2024. Disponível em: <https://www.wired.com/2015/07/hackers-remotely-kill-jeep-highway/>.
- GUNDU, R.; MALEKI, M. Securing can bus in connected and autonomous vehicles using supervised machine learning approaches. In: **IEEE eIT**. [S.l.: s.n.], 2022. p. 042–046.
- GÉRON, A. **Mãos à Obra: Aprendizado de Máquina com Scikit-Learn & TensorFlow**. 1. ed. [S.l.]: Alta Books, 2019. 576 p.

HARRISON, M. **Machine Learning – Guia de Referência Rápida: Trabalhando com Dados Estruturados em Python**. [S.l.]: Novatec Editora, 2019. 272 p.

HCRL. **CAN Dataset for intrusion detection (OTIDS)**. 2017. Acesso em: 29 nov. 2024. Disponível em: <https://ocslab.hksecurity.net/Dataset/CAN-intrusion-dataset>.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. **Neural computation**, v. 9, p. 1735–80, 12 1997.

KOHAVI, R. A study of cross-validation and bootstrap for accuracy estimation and model selection. v. 14, 03 2001.

KOSCHER, K. et al. Experimental security analysis of a modern automobile. In: **IEEE Symposium on Security and Privacy**. [S.l.: s.n.], 2010. p. 447–462.

LEE, H.; JEONG, S. H.; KIM, H. K. Otds: A novel intrusion detection system for in-vehicle network by using remote frame. In: **15th Annual Conference on Privacy, Security and Trust (PST)**. [S.l.: s.n.], 2017. p. 57–5709.

MUKHERJEE, S. et al. Practical dos attacks on embedded networks in commercial vehicles. In: SPRINGER. **12th International Conference (ICISS 2016)**. [S.l.], 2016. p. 23–42.

MURPHY, K. P. **Machine Learning: A Probabilistic Perspective**. Cambridge, MA: MIT Press, 2012.

PALANCA, A. et al. A stealth, selective, link-layer denial-of-service attack against automotive networks. In: SPRINGER. **14th International Conference (DIMVA 2017)**. [S.l.], 2017. p. 185–206.

PRASHANTH, R. Improving the flashing speed of electronic control unit in automotive industries. In: . [S.l.]: IEEE, 2019. v. 1, p. 1–6.

PUROHIT, S.; GOVINDARASU, M. MI-based anomaly detection for intra-vehicular can-bus networks. In: IEEE. **IEEE CSR**. [S.l.], 2022. p. 233–238.

QUINLAN, J. R. Induction of decision trees. **Machine Learning**, v. 1, n. 1, p. 81–106, 1986. ISSN 1573-0565. Disponível em: <https://doi.org/10.1007/BF00116251>.

RAJAPAKSHA, S. et al. Ai-based intrusion detection systems for in-vehicle networks: A survey. **ACM Comput. Surv.**, Association for Computing Machinery, New York, NY, USA, v. 55, n. 11, fev. 2023. ISSN 0360-0300. Disponível em: <https://doi.org/10.1145/3570954>.

SHALEV-SHWARTZ, S.; BEN-DAVID, S. **Understanding Machine Learning: From Theory to Algorithms**. Cambridge: Cambridge University Press, 2014.

SHEARER, C. The crisp-dm model: the new blueprint for data mining. **Journal of Data Warehousing**, v. 5, n. 4, p. 13–22, 2000.

SHUKLA, S. **Embedded Security for Vehicles : ECU Hacking**. [S.l.: s.n.], 2015.

SONG, H. M.; KIM, H. R.; KIM, H. K. Intrusion detection system based on the analysis of time intervals of can messages for in-vehicle network. In: **2016 ICOIN**. [S.l.: s.n.], 2016. p. 63–68.

STANDARDIZATION, I. O. for. **Road vehicles - Controller area network (CAN) - Part 1: Data link layer and physical signalling**. Geneva, CH, 2015. Car informatics. On board computer systems.

STARFIVE. **Boards**. 2024. Acesso em: 24 nov. 2024. Disponível em: <https://www.starfivetechnology.com/en/site/boards>.

SUNDARAM, A. An introduction to intrusion detection. **Crossroads**, 1996.

TINDELL, K. **Car theft with can injection**. 2023. <https://kentindell.github.io/2023/04/03/can-injection/>. Acessado: 02-11-2024.

TINGWALL, E. **Electronics Account for 40 Percent of the Cost of a New Car**. Car and Driver, 2020. Acesso em : 05 dez. 2024. Disponível em: <https://www.caranddriver.com/features/a32034437/computer-chips-in-cars/>.

VOSS, W. **A Comprehensible Guide to Controller Area Network**. [S.l.]: Copperhill Media Corporation, 2008. 152 p.

YANG, L.; MOUBAYED, A.; SHAMI, A. Mth-ids: A multitiered hybrid intrusion detection system for internet of vehicles. **IEEE Internet of Things Journal**, v. 9, p. 616–632, 2021. Disponível em: <https://api.semanticscholar.org/CorpusID:235212223>.

YIN, L. et al. Detecting can overlapped voltage attacks with an improved voltage-based in-vehicle intrusion detection system. **Journal of Systems Architecture**, v. 143, p. 102957, 2023. ISSN 1383-7621. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1383762123001364>.

ZHOU; LI; SHEN. Anomaly detection of can bus messages using a deep neural network for autonomous vehicles. **Applied Sciences**, v. 9, p. 3174, 08 2019.

A ESPECIFICAÇÃO DE HARDWARE - VISIONFIVE 2

A Tabela A sintetiza as principais especificações de hardware da SBC VisionFive 2, utilizada neste trabalho para validar o desempenho dos modelos de ML propostos, em um ambiente de recursos computacionais limitados, análogo a uma rede interna de um veículo.

Item	Descrição
Processador/SoC	StarFive JH7110 SoC 64 bits com RV64GC, até 1,5GHz
Memória	LPDDR4, 2GB/4GB/8GB
Armazenamento	Slot para cartão TF Flash para Uboot
Saída de vídeo	HDMI 2.0 MIPI-DSI
Multimídia	Câmera com MIPI CSI, até 4k@30fps Decodificação H.264 e H.265 4k@60fps Codificação H.265 1080p@30fps Codificação/decodificação JPEG 1/ Áudio estéreo de 4 polos 2/ HDMI
Conectividade	2x RJ45 Gigabit Ethernet 2x USB2.0 + 2x USB 3.0 M.2 M-Key
Energia	Porta USB-C, 5V DC via USB-C com PD, até 30W Entrada GPIO, 5V DC via cabeçalho GPIO (mínimo de 3A+) PoE
GPIO	Cabeçalho GPIO de 40 pinos
Dimensões	100 x 72mm
Conformidade	RoHS, FCC, CE
Botão	Botão de reset
Outros	Cabeçalhos de pinos para depuração

Tabela 11 – Especificações da VisionFive 2. Adaptado e traduzido do site oficial da fabricante StarFive (STARFIVE, 2024).

B ESPECIFICAÇÃO DE HARDWARE - RASPBERRY PI 3 MODEL B

A Tabela B descreve as principais especificações de hardware da Raspberry Pi 3 Model B, SBC utilizada neste trabalho para validar o desempenho de alguns dos modelos de ML propostos neste trabalho, em um ambiente embarcado com recursos computacionais limitados.

Item	Descrição
Processador/SoC	Quad Core 1.2GHz Broadcom BCM2837 64 bits
Memória	1GB RAM
Armazenamento	Slot para cartão Micro SD
Saída de vídeo	Full size HDMI Porta de vídeo composto via conector de 4 polos
Multimídia	Porta de áudio/vídeo estéreo de 4 polos Suporte para câmera via porta CSI Suporte para display via porta DSI
Conectividade	BCM43438 wireless LAN (Wi-Fi) Bluetooth Low Energy (BLE) Ethernet 100Base-T 4x USB 2.0
Energia	Fonte Micro USB com comutação, até 2.5A
GPIO	Cabeçalho GPIO de 40 pinos
Dimensões	85,6 x 56,5mm

Tabela 12 – Especificações da Raspberry Pi 3 Model B. Informações adaptadas e traduzidas do site oficial da Raspberry Pi Foundation (FOUNDATION, 2025).

C CÓDIGO-FONTE

O código-fonte, os *datasets* e os arquivos de saída gerados — como gráficos, modelos e arquivos de resultados (salvos no formato `.json`) — estão organizados conforme ilustrado nas Figuras 19, 20 e 21. As seções seguintes deste apêndice apresentam o código correspondente.

Todo o material também está disponível de forma virtual em um repositório público, na conta do autor no GitHub, acessível em: <https://github.com/BonomoJoaoPaulo/can-ids-ml>.

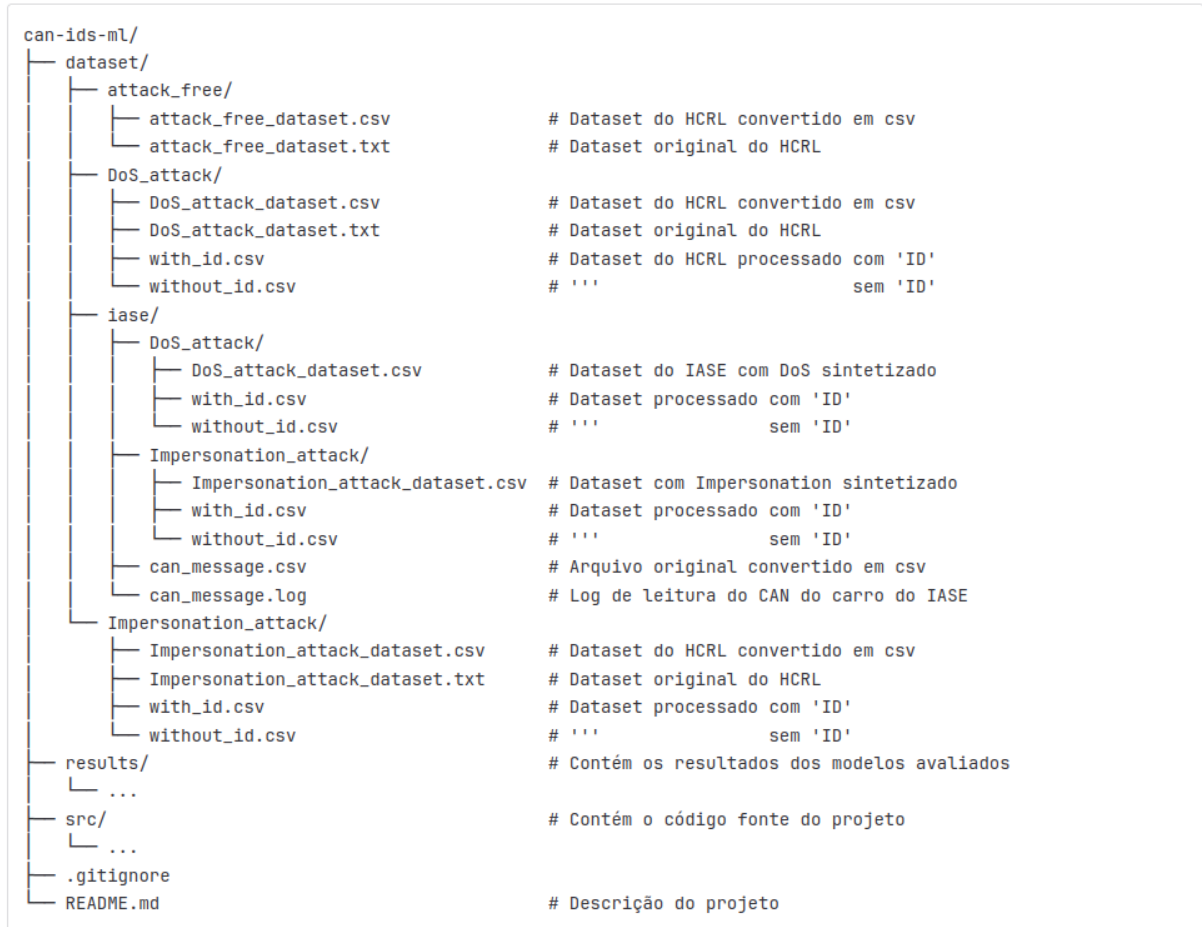


Figura 19 – Organização interna da pasta *dataset/* do repositório do trabalho no GitHub.

```

can-ids-ml/
├── dataset/                                # Contém os datasets do projeto
│   └── ...
├── results/
│   ├── {ATTACK_NAME}                      # DoS_attack || Impersonation_attack
│   │   ├── {MODEL_NAME}                  # Ex: DecisionTreeClassifier, KNN...
│   │   │   ├── {DATASET_NAME}           # Ex: with_id, without_id...
│   │   │   ├── models
│   │   │   │   └── model_{ATTACK_NAME}_{MODEL_NAME}.joblib
│   │   │   ├── plots
│   │   │   │   ├── confusion_matrix.png
│   │   │   │   ├── feature_importance.png
│   │   │   │   └── learning_curves.png
│   │   │   └── results
│   │   │       └── results.json          # Informações sobre treinamento e validação
│   └── time_results
│       ├── {PLATFORM}                    # i5 || rasp || RISCV
│       │   ├── {ATTACK_NAME}             # DoS_attack || Impersonation_attack
│       │   │   ├── {MODEL_NAME}          # Ex: DecisionTreeClassifier, KNN...
│       │   │   │   ├── {DATASET_NAME}    # Ex: with_id, without_id...
│       │   │   │   └── time_results.json  # Resultados do tempo de execução
│       └── ...
├── src/
│   └── ...
├── .gitignore
└── README.md                             # Descrição do projeto

```

Figura 20 – Organização interna da pasta *results/* do repositório do trabalho no GitHub.

```

can-ids-ml/
├── dataset/                                # Contém os datasets do projeto
│   └── ...
├── results/                                # Contém os resultados dos modelos avaliados
│   └── ...
├── src/
│   ├── classifiers/
│   │   ├── decision_tree.py              # Treinamento e validação da Decision Tree
│   │   ├── kmeans.py                     # ' ' ' do KMeans
│   │   ├── knn.py                        # ' ' ' do KNN
│   │   ├── xgboost.py                     # ' ' ' do XGBoost
│   │   └── random_forest.py               # ' ' ' da Random Forest
│   ├── data_processing/
│   │   ├── drop_columns.py               # 'Dropa' colunas selecionadas do dataset
│   │   ├── log_to_csv.py                  # Converte arquivos .log ou .txt em .csv
│   │   └── preprocess_data.py             # Pré-processamento do dataset
│   ├── experiments/
│   │   ├── iase_eval_lstm.py              # Valida a LSTM no dataset do IASE
│   │   ├── iase_eval.py                   # Valida o modelo no dataset do IASE
│   │   └── time_validation.py              # Valida o tempo de execução do modelo
│   ├── injectors/
│   │   ├── dos_injector.py               # Sintetiza DoS no dataset do IASE
│   │   └── impersonation_injector.py      # Sintetiza Impersonation ' ' '
│   ├── plots/
│   │   └── et_plots.py                    # Plots dos tempos de execução dos modelos
│   ├── rnn/
│   │   └── lstm.py                        # Treinamento e validação da LSTM
│   └── main.py                            # Arquivo vazio
├── .gitignore
└── README.md                             # Descrição do projeto

```

Figura 21 – Organização interna da pasta *src/* do repositório do trabalho no GitHub.

C.1 SRC/

src/main.py

```

1  """
2  Código-fonte do Trabalho de Conclusão de Curso (TCC) em Ciência da Computação
3  da Universidade Federal de Santa Catarina (UFSC), desenvolvido por
4  João Paulo Araujo Bonomo.
5
6  Título: 'Detecção de Ataques em Redes Intraveiculares CAN com Técnicas de Machine
   ↪ Learning'.
7  Ano: 2025
8  """
9

```

C.2 SRC/CLASSIFIERS/

src/classifiers/decision_tree.py

```

1  import os
2  import pandas as pd
3  import time
4  from sklearn.model_selection import train_test_split, cross_val_score, learning_curve
5  from sklearn.metrics import accuracy_score, confusion_matrix, classification_report,
   ↪ roc_curve, auc, precision_recall_curve
6  from sklearn.tree import DecisionTreeClassifier
7  import seaborn as sns
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import json
11 import joblib
12 from pathlib import Path
13
14 # Configuracoes do modelo e dataset
15 MODEL_NAME = "DecisionTreeClassifier"
16 ATTACK_NAME = "Impersonation_attack" # DoS_attack // Impersonation_attack
17 DATASET_NAME = "without_id.csv"      # with_id.csv // without_id.csv
18
19 # Configuracoes de Paths
20 SRC_PATH = os.path.dirname(os.path.abspath(__file__))
21 ROOT_PATH = Path(SRC_PATH).parent.parent
22 OUTPUTS_PATH = os.path.join(ROOT_PATH, 'results')
23 DATASET_PATH = os.path.join(ROOT_PATH, 'dataset', ATTACK_NAME, DATASET_NAME)
24 PLOTS_DIR = os.path.join(OUTPUTS_PATH,
   ↪ f'{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/plots')
25 RESULTS_DIR = os.path.join(OUTPUTS_PATH,
   ↪ f'{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/results')

```

```

26 MODELS_DIR = os.path.join(OUTPUTS_PATH,
    ↪ f'{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/models')
27
28 if not os.path.exists(PLOTS_DIR):
29     os.makedirs(PLOTS_DIR)
30 if not os.path.exists(RESULTS_DIR):
31     os.makedirs(RESULTS_DIR)
32 if not os.path.exists(MODELS_DIR):
33     os.makedirs(MODELS_DIR)
34
35 data = pd.read_csv(DATASET_PATH)
36
37 # Divisao o conjunto de dados em features (X) e rótulos (y)
38 X = data.drop(columns=[f'{ATTACK_NAME}', 'IDK'], axis=1)
39 y = data[f'{ATTACK_NAME}']
40
41 # Divisao dos dados em conjuntos de treinamento e teste
42 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    ↪ random_state=42)
43
44 # Inicializacao do classificador Decision Tree
45 model = DecisionTreeClassifier()
46
47 start_time_training = time.time()
48
49 model.fit(X_train, y_train)
50
51 end_time_training = time.time()
52
53 training_time = end_time_training - start_time_training
54 print(f"Tempo de treinamento: {training_time:.4f} segundos")
55
56 start_time_testing = time.time()
57
58 y_pred = model.predict(X_test)
59
60 end_time_testing = time.time()
61
62 testing_time = end_time_testing - start_time_testing
63 print(f"Tempo de teste: {testing_time:.2f} segundos")
64
65 y_pred_proba = model.predict_proba(X_test)[: , 1]
66
67 # Calculo da acuracia do modelo no conjunto de teste
68 accuracy = accuracy_score(y_test, y_pred)
69 print("Acuracia no teste:", accuracy)
70
71 # Calculo da confusion matrix
72 conf_matrix = confusion_matrix(y_test, y_pred)

```

```

73
74 # Plot da matrix
75 plt.figure(figsize=(8, 6))
76 sns.heatmap(conf_matrix, annot=True, fmt='d', cmap="Blues", cbar=False)
77 plt.xlabel('Predicted labels')
78 plt.ylabel('True labels')
79 plt.title('Confusion Matrix')
80 plt.savefig(f'{PLOTS_DIR}/confusion_matrix.png')
81 plt.close()
82
83 # Calculo e print das metricas precision, recall e f1-score
84 report = classification_report(y_test, y_pred, output_dict=True)
85 print("Classification Report:")
86 print(report)
87
88 # Calculo da acuracia no conjunto de treinamento
89 train_pred = model.predict(X_train)
90 train_accuracy = accuracy_score(y_train, train_pred)
91 print("Acuracia no treinamento:", train_accuracy)
92
93 # Avaliacao do modelo usando cross validation
94 cv_scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
95 print("Cross-validation scores:", cv_scores)
96 print("Mean cross-validation score:", cv_scores.mean())
97
98 # Geracao de learning curves
99 train_sizes, train_scores, test_scores = learning_curve(model, X, y, cv=5,
    ↪ scoring='accuracy', n_jobs=-1, train_sizes=np.linspace(0.1, 1.0, 10))
100
101 # Calculo da media e desvio padrao das pontuacoes de treinamento e teste
102 train_scores_mean = train_scores.mean(axis=1)
103 train_scores_std = train_scores.std(axis=1)
104 test_scores_mean = test_scores.mean(axis=1)
105 test_scores_std = test_scores.std(axis=1)
106
107 # Plote das learning curves
108 plt.figure(figsize=(10, 6))
109 plt.fill_between(train_sizes, train_scores_mean - train_scores_std, train_scores_mean
    ↪ + train_scores_std, alpha=0.1, color="r")
110 plt.fill_between(train_sizes, test_scores_mean - test_scores_std, test_scores_mean +
    ↪ test_scores_std, alpha=0.1, color="g")
111 plt.plot(train_sizes, train_scores_mean, 'o-', color="r", label="Training score")
112 plt.plot(train_sizes, test_scores_mean, 'o-', color="g", label="Cross-validation
    ↪ score")
113 plt.xlabel('Training Size')
114 plt.ylabel('Score')
115 plt.legend(loc='best')
116 plt.title('Learning Curves')
117 plt.savefig(f'{PLOTS_DIR}/learning_curves.png')

```

```

118 plt.close()
119
120 # Salvando os resultados em um arquivo JSON
121 results = {
122     "test_accuracy": accuracy,
123     "classification_report": report,
124     "train_accuracy": train_accuracy,
125     "cross_validation_scores": cv_scores.tolist(),
126     "mean_cross_validation_score": cv_scores.mean(),
127     "training_time": training_time
128 }
129
130 # Escrevendo os resultados em um arquivo JSON
131 with open(f'{RESULTS_DIR}/results.json', 'w') as f:
132     json.dump(results, f, indent=4)
133
134 # Salvando o modelo treinado
135 joblib.dump(model, f'{MODELS_DIR}/model_{ATTACK_NAME}_{MODEL_NAME}.joblib')
136
137
138 if __name__ == "__main__":
139     print("\n" + "="*50)
140     print(f"{MODEL_NAME} executed successfully!")
141     print(f"Attack type: {ATTACK_NAME}")
142     print(f"Dataset used: {DATASET_NAME}")
143     print(f"Model saved at: {MODELS_DIR}")
144     print(f"Plots saved at: {PLOTS_DIR}")
145     print(f"Results saved at: {RESULTS_DIR}/results.json")
146

```

src/classifiers/kmeans.py

```

1 import os
2 import pandas as pd
3 import time
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import accuracy_score, classification_report
6 from sklearn.cluster import KMeans
7 import numpy as np
8 import json
9 import joblib
10 from pathlib import Path
11 from scipy.stats import mode
12
13 # Configuracoes do modelo e dataset
14 MODEL_NAME = "KMeans"
15 ATTACK_NAME = "Impersonation_attack" # DoS_attack || Impersonation_attack
16 DATASET_NAME = "without_id.csv" # with_id.csv || without_id.csv

```

```

17
18 # Configuracoes de Paths
19 SRC_PATH = os.path.dirname(os.path.abspath(__file__))
20 ROOT_PATH = Path(SRC_PATH).parent.parent
21 OUTPUTS_PATH = os.path.join(ROOT_PATH, 'results')
22 DATASET_PATH = os.path.join(ROOT_PATH, 'dataset', ATTACK_NAME, DATASET_NAME)
23 PLOTS_DIR = os.path.join(OUTPUTS_PATH,
    ↳ f'{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/plots')
24 RESULTS_DIR = os.path.join(OUTPUTS_PATH,
    ↳ f'{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/results')
25 MODELS_DIR = os.path.join(OUTPUTS_PATH,
    ↳ f'{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/models')
26
27 if not os.path.exists(PLOTS_DIR):
28     os.makedirs(PLOTS_DIR)
29 if not os.path.exists(RESULTS_DIR):
30     os.makedirs(RESULTS_DIR)
31 if not os.path.exists(MODELS_DIR):
32     os.makedirs(MODELS_DIR)
33
34 data = pd.read_csv(DATASET_PATH)
35
36 X = data.drop(f'{ATTACK_NAME}', axis=1)
37 y = data[f'{ATTACK_NAME}']
38
39 # Divisao do conjunto de dados em features (X) e target (y)
40 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    ↳ random_state=42)
41
42 # Ajustes do modelo KMeans
43 n_clusters = len(np.unique(y)) # Numero de clusters igual ao numero de classes (2)
44 model = KMeans(n_clusters=n_clusters, random_state=42)
45
46 start_time_training = time.time()
47 model.fit(X_train)
48 end_time_training = time.time()
49
50 training_time = end_time_training - start_time_training
51 print(f"Tempo de treinamento: {training_time:.2f} segundos")
52
53 # Previsao dos clusters para o conjunto de teste
54 y_pred_clusters = model.predict(X_test)
55
56 # Mapeamento dos clusters para os rotulos verdadeiros
57 labels = np.zeros_like(y_pred_clusters)
58 for i in range(n_clusters):
59     mask = (y_pred_clusters == i)
60     labels[mask] = mode(y_test[mask])[0]
61

```

```

62 # Calculo da acuracia do modelo no conjunto de teste
63 accuracy = accuracy_score(y_test, labels)
64 print("Acuracia no teste:", accuracy)
65
66 # Relatorio de classificacao
67 report = classification_report(y_test, labels, output_dict=True)
68 print("Classification Report:")
69 print(report)
70
71 results = {
72     "test_accuracy": accuracy,
73     "classification_report": report,
74     "training_time_seconds": training_time
75 }
76
77 # Escrita dos resultados
78 with open(f'{RESULTS_DIR}/results.json', 'w') as f:
79     json.dump(results, f, indent=4)
80
81 # Save do modelo treinado
82 joblib.dump(model, f'{MODELS_DIR}/model_{ATTACK_NAME}_{MODEL_NAME}.joblib')
83
84
85 if __name__ == "__main__":
86     print("\n" + "="*50)
87     print(f"{MODEL_NAME} executed successfully!")
88     print(f"Attack type: {ATTACK_NAME}")
89     print(f"Dataset used: {DATASET_NAME}")
90     print(f"Model saved at: {MODELS_DIR}")
91     print(f"Plots saved at: {PLOTS_DIR}")
92     print(f"Results saved at: {RESULTS_DIR}/results.json")
93

```

src/classifiers/knn.py

```

1  import os
2  import pandas as pd
3  import time
4  from sklearn.model_selection import train_test_split, cross_val_score, learning_curve
5  from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
6  from sklearn.neighbors import KNeighborsClassifier
7  import seaborn as sns
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import json
11 import joblib
12 from pathlib import Path
13

```

```

14 # Configuracoes do modelo e dataset
15 MODEL_NAME = "KNN"
16 K_NEIGHBORS = 5 # Numero de vizinhos para o KNN
17 ATTACK_NAME = "Impersonation_attack" # DoS_attack || Impersonation_attack
18 DATASET_NAME = "without_id.csv" # with_id.csv || without_id.csv
19
20 # Configuracoes de Paths
21 SRC_PATH = os.path.dirname(os.path.abspath(__file__))
22 ROOT_PATH = Path(SRC_PATH).parent.parent
23 OUTPUTS_PATH = os.path.join(ROOT_PATH, 'results')
24 DATASET_PATH = os.path.join(ROOT_PATH, 'dataset', ATTACK_NAME, DATASET_NAME)
25 PLOTS_DIR = os.path.join(OUTPUTS_PATH,
    ↪ f'{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/plots')
26 RESULTS_DIR = os.path.join(OUTPUTS_PATH,
    ↪ f'{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/results')
27 MODELS_DIR = os.path.join(OUTPUTS_PATH,
    ↪ f'{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/models')
28
29 if not os.path.exists(PLOTS_DIR):
30     os.makedirs(PLOTS_DIR)
31 if not os.path.exists(REULTS_DIR):
32     os.makedirs(REULTS_DIR)
33 if not os.path.exists(MODELS_DIR):
34     os.makedirs(MODELS_DIR)
35
36 data = pd.read_csv(DATASET_PATH)
37
38 # Divisao do conjunto de dados em features (X) e target (y)
39 X = data.drop(columns=[f'{ATTACK_NAME}', 'IDK'], axis=1)
40 y = data[f'{ATTACK_NAME}']
41
42 # Divisao dos dados em conjuntos de treinamento e teste
43 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    ↪ random_state=42)
44
45 # Inicializacao do classificador KNN
46 model = KNeighborsClassifier(n_neighbors=K_NEIGHBORS)
47
48 start_time_training = time.time()
49
50 model.fit(X_train, y_train)
51
52 end_time_training = time.time()
53
54 training_time = end_time_training - start_time_training
55 print(f"Tempo de treinamento: {training_time:.2f} segundos")
56
57 start_time_testing = time.time()
58

```

```

59 y_pred = model.predict(X_test)
60
61 end_time_testing = time.time()
62
63 testing_time = end_time_training - start_time_testing
64 print(f"Tempo de teste: {testing_time:.2f} segundos")
65
66 y_pred_proba = model.predict_proba(X_test)[: , 1]
67
68 # Calculo da acuracia do modelo no conjunto de teste
69 accuracy = accuracy_score(y_test, y_pred)
70 print("Acuracia no teste:", accuracy)
71
72 # Calculo da matriz de confusao
73 conf_matrix = confusion_matrix(y_test, y_pred)
74
75 # Visualizacao da matriz de confusao
76 plt.figure(figsize=(8, 6))
77 sns.heatmap(conf_matrix, annot=True, fmt='d', cmap="Blues", cbar=False)
78 plt.xlabel('Predicted labels')
79 plt.ylabel('True labels')
80 plt.title('Confusion Matrix')
81 plt.savefig(f'{PLOTS_DIR}/confusion_matrix.png')
82 plt.close()
83
84 # Calculo e exibicao das metricas precision, recall e f1-score
85 report = classification_report(y_test, y_pred, output_dict=True)
86 print("Classification Report:")
87 print(report)
88
89 # Calculo da acuracia no conjunto de treinamento
90 train_pred = model.predict(X_train)
91 train_accuracy = accuracy_score(y_train, train_pred)
92 print("Acuracia no treinamento:", train_accuracy)
93
94 # Avaliacao do modelo usando validacao cruzada
95 cv_scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
96 print("Cross-validation scores:", cv_scores)
97 print("Mean cross-validation score:", cv_scores.mean())
98
99 # Geracao das learning curves
100 train_sizes, train_scores, test_scores = learning_curve(model, X, y, cv=5,
    ↪ scoring='accuracy', n_jobs=-1, train_sizes=np.linspace(0.1, 1.0, 10))
101
102 # Calculo da media e desvio padrao das pontuacoes de treinamento e teste
103 train_scores_mean = train_scores.mean(axis=1)
104 train_scores_std = train_scores.std(axis=1)
105 test_scores_mean = test_scores.mean(axis=1)
106 test_scores_std = test_scores.std(axis=1)

```

```

107
108 # Plot das learning curves
109 plt.figure(figsize=(10, 6))
110 plt.fill_between(train_sizes, train_scores_mean - train_scores_std, train_scores_mean
    ↪ + train_scores_std, alpha=0.1, color="r")
111 plt.fill_between(train_sizes, test_scores_mean - test_scores_std, test_scores_mean +
    ↪ test_scores_std, alpha=0.1, color="g")
112 plt.plot(train_sizes, train_scores_mean, 'o-', color="r", label="Training score")
113 plt.plot(train_sizes, test_scores_mean, 'o-', color="g", label="Cross-validation
    ↪ score")
114 plt.xlabel('Training Size')
115 plt.ylabel('Score')
116 plt.legend(loc='best')
117 plt.title('Learning Curves')
118 plt.savefig(f'{PLOTS_DIR}/learning_curves.png')
119 plt.close()
120
121 # Salvando os resultados em um arquivo JSON
122 results = {
123     "test_accuracy": accuracy,
124     "classification_report": report,
125     "train_accuracy": train_accuracy,
126     "cross_validation_scores": cv_scores.tolist(),
127     "mean_cross_validation_score": cv_scores.mean(),
128     "training_time": training_time
129 }
130
131 # Escrevendo os resultados em um arquivo JSON
132 with open(f'{RESULTS_DIR}/results.json', 'w') as f:
133     json.dump(results, f, indent=4)
134
135 # Salvando o modelo treinado
136 joblib.dump(model, f'{MODELS_DIR}/model_{ATTACK_NAME}_{MODEL_NAME}.joblib')
137
138
139 if __name__ == "__main__":
140     print("\n" + "="*50)
141     print(f'{MODEL_NAME} executed successfully!')
142     print(f'K Neighbors: {K_NEIGHBORS}')
143     print(f'Attack type: {ATTACK_NAME}')
144     print(f'Dataset used: {DATASET_NAME}')
145     print(f'Model saved at: {MODELS_DIR}')
146     print(f'Plots saved at: {PLOTS_DIR}')
147     print(f'Results saved at: {RESULTS_DIR}/results.json')
148

```

```

1  import os
2  import pandas as pd
3  import time
4  from sklearn.model_selection import train_test_split, cross_val_score, learning_curve
5  from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
6  from sklearn.ensemble import RandomForestClassifier
7  import seaborn as sns
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import json
11 import joblib
12 from pathlib import Path
13
14 # Configuracoes do modelo e dataset
15 MODEL_NAME = "RandomForestClassifier"
16 ATTACK_NAME = "Impersonation_attack" # DoS_attack || Impersonation_attack
17 DATASET_NAME = "without_id.csv"      # with_id.csv || without_id.csv
18
19 # Configuracoes de Paths
20 SRC_PATH = os.path.dirname(os.path.abspath(__file__))
21 ROOT_PATH = Path(SRC_PATH).parent.parent
22 OUTPUTS_PATH = os.path.join(ROOT_PATH, 'results')
23 DATASET_PATH = os.path.join(ROOT_PATH, 'dataset', ATTACK_NAME, DATASET_NAME)
24 PLOTS_DIR = os.path.join(OUTPUTS_PATH,
    ↪ f'{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/plots')
25 RESULTS_DIR = os.path.join(OUTPUTS_PATH,
    ↪ f'{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/results')
26 MODELS_DIR = os.path.join(OUTPUTS_PATH,
    ↪ f'{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/models')
27
28 if not os.path.exists(PLOTS_DIR):
29     os.makedirs(PLOTS_DIR)
30 if not os.path.exists(RESULTS_DIR):
31     os.makedirs(RESULTS_DIR)
32 if not os.path.exists(MODELS_DIR):
33     os.makedirs(MODELS_DIR)
34
35 data = pd.read_csv(DATASET_PATH)
36
37 # Divisao do conjunto de dados em features (X) e rotulos (y)
38 X = data.drop(columns=[f'{ATTACK_NAME}', 'IDK'], axis=1)
39 y = data[f'{ATTACK_NAME}']
40
41 # Divisao dos dados em conjuntos de treinamento e teste
42 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    ↪ random_state=42)
43
44 # Inicializacao do classificador Random Forest

```

```

45 model = RandomForestClassifier()
46
47 start_time_training = time.time()
48
49 model.fit(X_train, y_train)
50
51 end_time_training = time.time()
52
53 training_time = end_time_training - start_time_training
54 print(f"Tempo de treinamento: {training_time:.2f} segundos")
55
56 start_time_testing = time.time()
57
58 y_pred = model.predict(X_test)
59
60 end_time_testing = time.time()
61
62 testing_time = end_time_training - start_time_testing
63 print(f"Tempo de teste: {testing_time:.2f} segundos")
64
65 y_pred_proba = model.predict_proba(X_test)[:, 1]
66
67 # Calculo da acuracia do modelo no conjunto de teste
68 accuracy = accuracy_score(y_test, y_pred)
69 print("Acuracia no teste:", accuracy)
70
71 # Calculo da matriz de confusao
72 conf_matrix = confusion_matrix(y_test, y_pred)
73
74 # Plot da matriz de confusao
75 plt.figure(figsize=(8, 6))
76 sns.heatmap(conf_matrix, annot=True, fmt='d', cmap="Blues", cbar=False)
77 plt.xlabel('Predicted labels')
78 plt.ylabel('True labels')
79 plt.title('Confusion Matrix')
80 plt.savefig(f'{PLOTS_DIR}/confusion_matrix.png')
81 plt.close()
82
83 # Calculo e print de precision, recall e f1-score
84 report = classification_report(y_test, y_pred, output_dict=True)
85 print("Classification Report:")
86 print(report)
87
88 # Calculo da acuracia no conjunto de treinamento
89 train_pred = model.predict(X_train)
90 train_accuracy = accuracy_score(y_train, train_pred)
91 print("Acuracia no treinamento:", train_accuracy)
92
93 # Avaliacao do modelo usando cross validation

```

```

94 cv_scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
95 print("Cross-validation scores:", cv_scores)
96 print("Mean cross-validation score:", cv_scores.mean())
97
98 # Geracao das learning curves
99 train_sizes, train_scores, test_scores = learning_curve(model, X, y, cv=5,
    ↪ scoring='accuracy', n_jobs=-1, train_sizes=np.linspace(0.1, 1.0, 10))
100
101 # Calculo da media e desvio padrao dos scores de treinamento e teste
102 train_scores_mean = train_scores.mean(axis=1)
103 train_scores_std = train_scores.std(axis=1)
104 test_scores_mean = test_scores.mean(axis=1)
105 test_scores_std = test_scores.std(axis=1)
106
107 # Plot das learning curves
108 plt.figure(figsize=(10, 6))
109 plt.fill_between(train_sizes, train_scores_mean - train_scores_std, train_scores_mean
    ↪ + train_scores_std, alpha=0.1, color="r")
110 plt.fill_between(train_sizes, test_scores_mean - test_scores_std, test_scores_mean +
    ↪ test_scores_std, alpha=0.1, color="g")
111 plt.plot(train_sizes, train_scores_mean, 'o-', color="r", label="Training score")
112 plt.plot(train_sizes, test_scores_mean, 'o-', color="g", label="Cross-validation
    ↪ score")
113 plt.xlabel('Training Size')
114 plt.ylabel('Score')
115 plt.legend(loc='best')
116 plt.title('Learning Curves')
117 plt.savefig(f'{PLOTS_DIR}/learning_curves.png')
118 plt.close()
119
120 # Plot de feature importance
121 plt.figure(figsize=(10, 8))
122 importances = model.feature_importances_
123 indices = np.argsort(importances)[::-1]
124 plt.title('Feature Importances')
125 plt.bar(range(X.shape[1]), importances[indices], color='b', align='center')
126 plt.xticks(range(X.shape[1]), [X.columns[i] for i in indices], rotation=90)
127 plt.xlim([-1, X.shape[1]])
128 plt.savefig(f'{PLOTS_DIR}/feature_importance.png')
129 plt.close()
130
131 # Escrita dos resultados em um arquivo JSON
132 results = {
133     "test_accuracy": accuracy,
134     "classification_report": report,
135     "train_accuracy": train_accuracy,
136     "cross_validation_scores": cv_scores.tolist(),
137     "mean_cross_validation_score": cv_scores.mean(),
138     "training_time_seconds": training_time,

```

```

139     "testing_time_seconds": testing_time
140 }
141
142 with open(f'{RESULTS_DIR}/results.json', 'w') as f:
143     json.dump(results, f, indent=4)
144     print("Resultados salvos com sucesso!")
145
146 # Save do modelo treinado
147 joblib.dump(model, f'{MODELS_DIR}/model_{ATTACK_NAME}_{MODEL_NAME}.joblib')
148
149
150 if __name__ == "__main__":
151     print("\n" + "="*50)
152     print(f'{MODEL_NAME} executed successfully!')
153     print(f"Attack type: {ATTACK_NAME}")
154     print(f"Dataset used: {DATASET_NAME}")
155     print(f"Model saved at: {MODELS_DIR}")
156     print(f"Plots saved at: {PLOTS_DIR}")
157     print(f"Results saved at: {RESULTS_DIR}/results.json")
158

```

src/classifiers/xgb.py

```

1  import os
2  import pandas as pd
3  import time
4  from sklearn.model_selection import train_test_split, cross_val_score, learning_curve
5  from sklearn.metrics import (
6      accuracy_score,
7      confusion_matrix,
8      classification_report,
9      roc_curve,
10     auc,
11     precision_recall_curve,
12 )
13 from xgboost import XGBClassifier, plot_importance
14 import seaborn as sns
15 import matplotlib.pyplot as plt
16 import numpy as np
17 import json
18 import joblib
19 from pathlib import Path
20
21 # Configuracoes do modelo e dataset
22 MODEL_NAME = "XGBClassifier"
23 ATTACK_NAME = "Impersonation_attack" # DoS_attack || Impersonation_attack
24 DATASET_NAME = "without_id.csv" # with_id.csv || without_id.csv
25

```

```

26  # Configuracoes de Paths
27  SRC_PATH = os.path.dirname(os.path.abspath(__file__))
28  ROOT_PATH = Path(SRC_PATH).parent.parent
29  OUTPUTS_PATH = os.path.join(ROOT_PATH, "results")
30  DATASET_PATH = os.path.join(ROOT_PATH, "dataset", ATTACK_NAME, DATASET_NAME)
31  PLOTS_DIR = os.path.join(
32      OUTPUTS_PATH, f"{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/plots"
33  )
34  RESULTS_DIR = os.path.join(
35      OUTPUTS_PATH, f"{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/results"
36  )
37  MODELS_DIR = os.path.join(
38      OUTPUTS_PATH, f"{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/models"
39  )
40
41  if not os.path.exists(PLOTS_DIR):
42      os.makedirs(PLOTS_DIR)
43  if not os.path.exists(RESULTS_DIR):
44      os.makedirs(RESULTS_DIR)
45  if not os.path.exists(MODELS_DIR):
46      os.makedirs(MODELS_DIR)
47
48  data = pd.read_csv(DATASET_PATH)
49
50  # Divisao do conjunto de dados em features (X) e target (y)
51  X = data.drop(columns=[f"{ATTACK_NAME}", "IDK"], axis=1)
52  y = data[f"{ATTACK_NAME}"]
53
54  # Divisao dos dados em conjuntos de treinamento e teste
55  X_train, X_test, y_train, y_test = train_test_split(
56      X, y, test_size=0.3, random_state=42
57  )
58
59  # Inicializacao do classificador XGBoost
60  model = XGBClassifier()
61
62  start_time_training = time.time()
63
64  model.fit(X_train, y_train)
65
66  end_time_training = time.time()
67
68  training_time = end_time_training - start_time_training
69  print(f"Tempo de treinamento: {training_time:.2f} segundos")
70
71  start_time_testing = time.time()
72
73  y_pred = model.predict(X_test)
74

```

```

75 end_time_testing = time.time()
76
77 testing_time = end_time_training - start_time_testing
78 print(f"Tempo de teste: {testing_time:.2f} segundos")
79
80 y_pred_proba = model.predict_proba(X_test)[: , 1]
81
82 # Calculo da acuracia do modelo no conjunto de teste
83 accuracy = accuracy_score(y_test, y_pred)
84 print("Acuracia no teste:", accuracy)
85
86 # Plot da matriz de confusao
87 conf_matrix = confusion_matrix(y_test, y_pred)
88
89 plt.figure(figsize=(8, 6))
90 sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues", cbar=False)
91 plt.xlabel("Predicted labels")
92 plt.ylabel("True labels")
93 plt.title("Confusion Matrix")
94 plt.savefig(f"{PLOTS_DIR}/confusion_matrix.png")
95 plt.close()
96
97 report = classification_report(y_test, y_pred, output_dict=True)
98 print("Classification Report:")
99 print(report)
100
101 train_pred = model.predict(X_train)
102 train_accuracy = accuracy_score(y_train, train_pred)
103 print("Acurácia no treinamento:", train_accuracy)
104
105 # Avaliacao do modelo usando validacao cruzada
106 cv_scores = cross_val_score(model, X, y, cv=5, scoring="accuracy")
107 print("Cross-validation scores:", cv_scores)
108 print("Mean cross-validation score:", cv_scores.mean())
109
110 # Geracao das learning curves
111 train_sizes, train_scores, test_scores = learning_curve(
112     model,
113     X,
114     y,
115     cv=10,
116     scoring="accuracy",
117     n_jobs=-1,
118     train_sizes=np.linspace(0.1, 1.0, 10),
119 )
120
121 # Calculo da media e desvio padrao dos scores de treinamento e teste
122 train_scores_mean = train_scores.mean(axis=1)
123 train_scores_std = train_scores.std(axis=1)

```

```

124 test_scores_mean = test_scores.mean(axis=1)
125 test_scores_std = test_scores.std(axis=1)
126
127 # Plot de learning curves
128 plt.figure(figsize=(12, 7))
129
130 # Area de desvio padrao com cores suaves
131 plt.fill_between(
132     train_sizes,
133     train_scores_mean - train_scores_std,
134     train_scores_mean + train_scores_std,
135     alpha=0.2,
136     color="#1f77b4",
137 )
138 plt.fill_between(
139     train_sizes,
140     test_scores_mean - test_scores_std,
141     test_scores_mean + test_scores_std,
142     alpha=0.2,
143     color="#ff7f0e",
144 )
145
146 # Curvas principais
147 plt.plot(
148     train_sizes,
149     train_scores_mean,
150     "o-",
151     color="#1f77b4",
152     linewidth=2.5,
153     label="Treinamento",
154 )
155 plt.plot(
156     train_sizes,
157     test_scores_mean,
158     "o-",
159     color="#ff7f0e",
160     linewidth=2.5,
161     label="Validação Cruzada",
162 )
163
164 # Eixos e titulo com fonte estilizada
165 plt.xlabel("Tamanho do Conjunto de Treinamento", fontsize=28)
166 plt.ylabel("Acurácia", fontsize=28)
167 plt.title("Curvas de Aprendizado do XGBoost para Impersonation", fontsize=32)
168 plt.grid(True, linestyle="--", linewidth=0.5, alpha=0.7)
169 plt.legend(loc="best", fontsize=20)
170 plt.xticks(fontsize=20)
171 plt.yticks(fontsize=20)
172

```

```

173 plt.tight_layout()
174 plt.savefig(f"{PLOTS_DIR}/learning_curves.png", dpi=300)
175 plt.show()
176 plt.close()
177
178 # Plot de feature importance
179 plt.figure(figsize=(10, 8))
180 plot_importance(model, max_num_features=10, importance_type="weight")
181 plt.title("Feature Importance")
182 plt.savefig(f"{PLOTS_DIR}/feature_importance.png")
183 plt.show()
184 plt.close()
185
186 # Plot de ROC curve
187 fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
188 roc_auc = auc(fpr, tpr)
189 plt.figure(figsize=(8, 6))
190 plt.plot(fpr, tpr, color="darkorange", lw=2, label=f"ROC curve (area =
    ↳ {roc_auc:0.2f})")
191 plt.plot([0, 1], [0, 1], color="navy", lw=2, linestyle="--")
192 plt.xlim([0.0, 1.0])
193 plt.ylim([0.0, 1.05])
194 plt.xlabel("False Positive Rate")
195 plt.ylabel("True Positive Rate")
196 plt.title("Receiver Operating Characteristic (ROC)")
197 plt.legend(loc="lower right")
198 plt.savefig(f"{PLOTS_DIR}/roc_curve.png")
199 plt.close()
200
201 # Plot de Precision-Recall curve
202 precision, recall, _ = precision_recall_curve(y_test, y_pred_proba)
203 plt.figure(figsize=(8, 6))
204 plt.plot(recall, precision, color="blue", lw=2)
205 plt.xlabel("Recall")
206 plt.ylabel("Precision")
207 plt.title("Precision-Recall Curve")
208 plt.savefig(f"{PLOTS_DIR}/precision_recall_curve.png")
209 plt.close()
210
211 # Plot de histograma das previsões
212 plt.figure(figsize=(8, 6))
213 sns.histplot(y_pred_proba, bins=10, kde=True, color="blue")
214 plt.xlabel("Predicted Probabilities")
215 plt.ylabel("Frequency")
216 plt.title("Histogram of Predicted Probabilities")
217 plt.savefig(f"{PLOTS_DIR}/predicted_probabilities_histogram.png")
218 plt.close()
219
220 # Escrita dos resultados em um arquivo JSON

```

```

221 results = {
222     "test_accuracy": accuracy,
223     "classification_report": report,
224     "train_accuracy": train_accuracy,
225     "cross_validation_scores": cv_scores.tolist(),
226     "mean_cross_validation_score": cv_scores.mean(),
227     "training_time_seconds": training_time,
228 }
229
230 with open(f"{RESULTS_DIR}/results.json", "w") as f:
231     json.dump(results, f, indent=4)
232
233 # Save do modelo treinado
234 joblib.dump(model, f"{MODELS_DIR}/model_{ATTACK_NAME}_{MODEL_NAME}.joblib")
235
236
237 if __name__ == "__main__":
238     print("\n" + "=" * 50)
239     print(f"{MODEL_NAME} executed successfully!")
240     print(f"Attack type: {ATTACK_NAME}")
241     print(f"Dataset used: {DATASET_NAME}")
242     print(f"Model saved at: {MODELS_DIR}")
243     print(f"Plots saved at: {PLOTS_DIR}")
244     print(f"Results saved at: {RESULTS_DIR}/results.json")
245

```

C.3 SRC/DATA_PROCESSING/

src/data_processing/drop_columns.py

```

1  import pandas as pd
2  import os
3  from pathlib import Path
4
5  # Paths
6  SRC_PATH = os.path.dirname(os.path.abspath(__file__))
7  ROOT_PATH = Path(SRC_PATH).parent.parent
8
9
10 def drop_columns(df) -> pd.DataFrame:
11     print("Columns before drop:", df.columns)
12
13     # Lista das colunas a serem dropadas
14     colunas_para_dropar = [
15         "Data",
16         "Data_0",
17         "Data_1",

```

```

18         "Data_2",
19         "Data_3",
20         "Data_4",
21         "Data_5",
22         "Data_6",
23         "Data_7",
24         "new_Data",
25     ]
26
27     # Drop das colunas especificadas
28     df = df.drop(columns=colunas_para_dropar)
29     df = df.drop(columns=["ID"]) # Remove a coluna 'ID'
30     print("Columns after drop:", df.columns)
31
32     return df
33
34
35 if __name__ == "__main__":
36     DATASET_PATH = os.path.join(ROOT_PATH, "dataset/iase/Impersonation_attack")
37     initial_dataset_name = "Impersonation_attack.csv"
38     file_path = os.path.join(DATASET_PATH, initial_dataset_name)
39
40     # Leitura do arquivo CSV
41     df = pd.read_csv(file_path)
42
43     # Chamada da funcao para remover colunas
44     df = drop_coluns(df)
45
46     # Save do DataFrame de volta para um novo arquivo CSV
47     df.to_csv(os.path.join(DATASET_PATH, "without_id.csv"), index=False)
48
49     print("Colunas removidas e arquivo salvo com sucesso!")
50

```

src/data_processing/log_to_csv.py

```

1  import os
2  import csv
3  from pathlib import Path
4
5  # Paths
6  SRC_PATH = os.path.dirname(os.path.abspath(__file__))
7  ROOT_PATH = Path(SRC_PATH).parent.parent
8  DATASET_PATH = os.path.join(ROOT_PATH, "dataset/iase")
9
10
11 def convert_log_to_csv(file_path: str, output_path: str) -> None:
12     """

```

```

13     Essa funcao converte um arquivo de log de mensagens CAN para um arquivo CSV.
14     Funciona para arquivos .txt ou .log que possuem mensagens CAN formatadas em linhas
15     ↪ separadas.
16     """
17     with open(file_path, mode="r") as file:
18         lines = file.readlines()
19
20     with open(
21         os.path.join(output_path, "can_messages.csv"), mode="w", newline=""
22     ) as file:
23         writer = csv.writer(file)
24         for line in lines:
25             writer.writerow(line.strip().split(", "))
26
27 if __name__ == "__main__":
28     initial_dataset = "can_messages.log"
29     file_path = os.path.join(DATASET_PATH, initial_dataset)
30
31     convert_log_to_csv(file_path, DATASET_PATH)
32
33     print("Arquivo convertido com sucesso de log para csv!")
34

```

src/data_processing/pp_data.py

```

1  from typing import List
2  import pandas as pd
3  import numpy as np
4  import os
5  from pathlib import Path
6
7  # Paths
8  SRC_PATH = os.path.dirname(os.path.abspath(__file__))
9  ROOT_PATH = Path(SRC_PATH).parent.parent
10 DATASET_PATH = os.path.join(ROOT_PATH, 'dataset')
11
12
13 def split_bytes(df: pd.DataFrame) -> pd.DataFrame:
14     """
15     Funcao para separar os bytes da coluna 'Data' e criar novas colunas.
16     Args:
17     df: O DataFrame que contem a coluna 'Data'.
18     Returns:
19     O DataFrame modificado com as novas colunas.
20     """
21     df["new_Data"] = df["Data"].str.replace(" ", "") # Cria coluna "new_Data" sem
22     ↪ espacos

```

```

22 df["Data"] = df["Data"].str.split(" ") # Separa os bytes em uma lista
23 for i in range(8):
24     df["Data_" + str(i)] = df["Data"].apply(lambda x: x[i]) # Cria colunas Data_0,
    ↪ Data_1...
25
26 return df
27
28
29 def convert_hex_to_decimal(df: pd.DataFrame, data_cols: List[str]) -> pd.DataFrame:
30     """
31     Funcao para converter valores hexadecimais em inteiros nas colunas especificadas.
32     Args:
33         df: O DataFrame que contem as colunas de dados.
34         data_cols: Lista de nomes das colunas que contem valores hexadecimais.
35     """
36     for col in data_cols:
37         new_col_name = f"{col}_int"
38         df[new_col_name] = df[col].apply(hex_to_dec)
39
40     return df
41
42
43 def hex_to_dec(hex_value: str) -> int:
44     """
45     Funcao para converter um valor hexadecimal para inteiro.
46     Args:
47         hex_value: O valor hexadecimal a ser convertido.
48     Returns:
49         O valor convertido para inteiro.
50     """
51     return int(hex_value, 16)
52
53
54 def map_ids(df: pd.DataFrame) -> pd.DataFrame:
55     """
56     Funcao para mapear os ids de forma que sejam sequenciais.
57     Args:
58         df: O DataFrame que contem a coluna 'ID'.
59     Returns:
60         O DataFrame modificado com os ids mapeados e inteiros.
61     """
62     print(df["ID"].value_counts())
63
64     # Cria um dicionario para mapear os IDs
65     ids = df['ID'].unique()
66     ids = np.sort(ids)
67
68     id_dict = {ids[i]: i for i in range(len(ids))}
69     print(id_dict)

```

```

70
71     # Mapeia os IDs
72     df['ID'] = df['ID'].map(id_dict)
73
74     return df
75
76
77 def create_delta_t(df: pd.DataFrame) -> pd.DataFrame:
78     """
79     Funcao para criar uma coluna de delta_t (diferenca de tempo) no DataFrame.
80     Args:
81         df: O DataFrame que contem a coluna 'Timestamp'.
82     Returns:
83         O DataFrame modificado com a coluna 'Diff_Timestamp'.
84     """
85     df = df.sort_values(by='Timestamp')
86
87     df['Diff_Timestamp'] = df['Timestamp'].diff()
88     df['Diff_Timestamp'].fillna(0, inplace=True)
89
90     return df
91
92
93 if __name__ == "__main__":
94     # Carrega o dataset inicial
95     initial_dataset = 'iase/Impersonation_attack/Impersonation_attack.csv'
96     file_path = os.path.join(DATASET_PATH, initial_dataset)
97     df = pd.read_csv(file_path)
98     print("Initial dataset loaded successfully.")
99
100    # Manipulacoes no DataFrame
101    df = map_ids(df) # Mapeia os IDs para inteiros sequenciais
102    df = split_bytes(df.copy()) # Separa os bytes da coluna 'Data' e cria novas colunas
103    data_cols = [f"Data_{i}" for i in range(8)]
104    data_cols.append("new_Data")
105    df = convert_hex_to_decimal(df.copy(), data_cols) # Converte os valores hexadecimais
106    ↪ para inteiros
107    df = create_delta_t(df) # Cria a coluna de delta_t
108
109    # Salva o novo DataFrame em um arquivo CSV
110    df.to_csv(os.path.join(DATASET_PATH, initial_dataset), index=False)
111    print("Data processing completed and saved successfully!")

```

C.4 SRC/EXPERIMENTS/

src/experiments/iase_eval.py

```

1  import joblib
2  import pandas as pd
3  from sklearn.metrics import accuracy_score, classification_report
4
5  # Configuracoes
6  attack = 'Impersonation_attack'
7  dataset = 'without_id.csv'
8  algorithm = 'RandomForestClassifier'
9
10 model_path =
    ↪ f'../results/{attack}/{algorithm}/{dataset}/models/model_{attack}_{algorithm}.joblib'
11
12 data = pd.read_csv(f'../dataset/iase/{attack}/{dataset}')
13
14 # Carrega o modelo salvo (treinado anteriormente no dataset do HCRL para o mesmo
    ↪ ataque)
15 model = joblib.load(model_path)
16
17 # Prepara os dados para a previsao
18 X_new = data.drop(attack, axis=1) # Features
19 y_new = data[attack] # Target
20
21 # Faz as previsoes
22 y_pred = model.predict(X_new)
23
24 # Avalia o desempenho do modelo
25 accuracy = accuracy_score(y_new, y_pred)
26
27
28 print("\n" + "="*50)
29 print("Results of the IASE Evaluation")
30 print(f'Model: {algorithm}')
31 print(f'Attack: {attack}')
32 print(f'Dataset: IASE - {dataset}')
33 print(f'Accuracy: {accuracy}')
34 print(classification_report(y_new, y_pred))
35

```

src/experiments/iase_eval_lstm.py

```

1  import numpy as np
2  import pandas as pd
3  from tensorflow.keras.models import load_model
4  from sklearn.metrics import accuracy_score, classification_report
5
6  # Configuracoes
7  attack = "Impersonation_attack"
8  dataset = "without_id.csv"

```

```

9  algorithm = "LSTM"
10 time_steps = 50  # 1 || 10 || 20 || 50
11
12 # Caminho para o modelo salvo
13 model_path =
14     ↪ f"../results/{attack}/{algorithm}/time_steps={time_steps}/{dataset}/models/best_model_{attack}_{alg
15 # Carrega a rede neural LSTM treinada
16 model = load_model(model_path)
17
18 # Carrega os dados
19 data = pd.read_csv(f"../dataset/iase/{attack}/{dataset}")
20
21 # Separa features e target
22 X = data.drop(columns=[attack]).values
23 y = data[attack].values
24
25 # Prepara os dados para a LSTM
26 X_lstm = X.reshape((X.shape[0], 1, X.shape[1]))
27
28 # Faz as previsoes
29 y_pred_probs = model.predict(X_lstm)
30 y_pred = (
31     (y_pred_probs > 0.5).astype(int).flatten()
32 ) # Threshold para classificar como 0 ou 1
33
34 # Avalia o desempenho da rede
35 accuracy = accuracy_score(y, y_pred)
36
37 print("\n" + "=" * 50)
38 print("LSTM Evaluation Results on IASE Dataset")
39 print(f"Attack: {attack}")
40 print(f"Dataset: IASE - {dataset}")
41 print(f"Time Steps: {time_steps}")
42 print(f"Accuracy: {accuracy:.4f}\n")
43 print(classification_report(y, y_pred))
44

```

src/experiments/time_validation.py

```

1  from typing import Dict
2  import os
3  import json
4  import time
5  import joblib
6  import pandas as pd
7  import numpy as np
8

```

```

9  def run_experiment(model: str, data: pd.DataFrame, n_executions: int) -> Dict[str,
    ↪ Dict[str, float]]:
10     # Carrega o modelo
11     model = joblib.load(model_path)
12
13     # Listas para armazenar os tempos de classificacao
14     classification_times_1_instance = []
15     classification_times_5_instances = []
16     classification_times_100_instances = []
17     classification_times_1000_instances = []
18
19     # Loop para executar o experimento n vezes
20     for _ in range(n_executions):
21         # Select instance for prediction (1 instance)
22         instance = data.iloc[0].values.reshape(1, -1)
23
24         # Measure prediction time
25         start_time = time.time()
26         prediction = model.predict(instance)
27         end_time = time.time()
28
29         # Calculate classification time and store it
30         classification_time = end_time - start_time
31         classification_times_1_instance.append(classification_time)
32
33         # Select instances for prediction (100 instances)
34         f_instances = 5
35         instances_to_predict = data.iloc[:f_instances]
36
37         # Measure prediction time
38         start_time = time.time()
39         prediction = model.predict(instances_to_predict)
40         end_time = time.time()
41
42         # Calculate classification time and store it
43         classification_time = end_time - start_time
44         classification_times_5_instances.append(classification_time / f_instances)
45
46         # Select instances for prediction (100 instances)
47         h_instances = 100
48         instances_to_predict = data.iloc[:h_instances]
49
50         # Measure prediction time
51         start_time = time.time()
52         prediction = model.predict(instances_to_predict)
53         end_time = time.time()
54
55         # Calculate classification time and store it
56         classification_time = end_time - start_time

```

```

57     classification_times_100_instances.append(classification_time / h_instances)
58
59     # Select instances for prediction (1000 instances)
60     k_instances = 1000
61     instances_to_predict = data.iloc[:k_instances]
62
63     # Measure prediction time
64     start_time = time.time()
65     prediction = model.predict(instances_to_predict)
66     end_time = time.time()
67
68     # Calculate classification time and store it
69     classification_time = end_time - start_time
70     classification_times_1000_instances.append(classification_time / k_instances)
71
72     # Calculate statistical metrics
73     mean_time_1_instance = np.mean(classification_times_1_instance)
74     min_time_1_instance = np.min(classification_times_1_instance)
75     max_time_1_instance = np.max(classification_times_1_instance)
76     std_dev_time_1_instance = np.std(classification_times_1_instance)
77
78     mean_time_5_instances = np.mean(classification_times_5_instances)
79     min_time_5_instances = np.min(classification_times_5_instances)
80     max_time_5_instances = np.max(classification_times_5_instances)
81     std_dev_time_5_instances = np.std(classification_times_5_instances)
82
83     mean_time_100_instances = np.mean(classification_times_100_instances)
84     min_time_100_instances = np.min(classification_times_100_instances)
85     max_time_100_instances = np.max(classification_times_100_instances)
86     std_dev_time_100_instances = np.std(classification_times_100_instances)
87
88     mean_time_1000_instances = np.mean(classification_times_1000_instances)
89     min_time_1000_instances = np.min(classification_times_1000_instances)
90     max_time_1000_instances = np.max(classification_times_1000_instances)
91     std_dev_time_1000_instances = np.std(classification_times_1000_instances)
92
93
94     # Salva os resultados em um dict
95     results = {
96         "1_instance": {
97             "mean_classification_time": mean_time_1_instance,
98             "minimum_classification_time": min_time_1_instance,
99             "maximum_classification_time": max_time_1_instance,
100             "std_dev_classification_time": std_dev_time_1_instance
101         },
102         "5_instances": {
103             "mean_classification_time_per_instance": mean_time_5_instances,
104             "minimum_classification_time_per_instance": min_time_5_instances,
105             "maximum_classification_time_per_instance": max_time_5_instances,

```

```

106         "std_dev_classification_time_per_instance": std_dev_time_5_instances
107     },
108     "100_instances": {
109         "mean_classification_time_per_instance": mean_time_100_instances,
110         "minimum_classification_time_per_instance": min_time_100_instances,
111         "maximum_classification_time_per_instance": max_time_100_instances,
112         "std_dev_classification_time_per_instance": std_dev_time_100_instances
113     },
114     "1000_instances": {
115         "mean_classification_time_per_instance": mean_time_1000_instances,
116         "minimum_classification_time_per_instance": min_time_1000_instances,
117         "maximum_classification_time_per_instance": max_time_1000_instances,
118         "std_dev_classification_time_per_instance": std_dev_time_1000_instances
119     }
120 }
121
122 return results
123
124
125 if __name__ == "__main__":
126     # Configuracoes do experimento
127     platform = 'Rasp'
128     attack = 'Impersonation_attack'
129     dataset = 'without_id.csv'
130     algorithm = 'DecisionTreeClassifier'
131     n_executions = 1000
132     model_path =
133     ↪ (f'../results/{attack}/{algorithm}/{dataset}/models/model_{attack}_{algorithm}.joblib')
134     data = pd.read_csv(f'../dataset/{attack}/{dataset}')
135     data = data.drop(columns=[attack, 'IDK'], axis=1)
136
137     print('Iniciando o experimento com as seguintes configuracoes:')
138     print(f'Plataforma: {platform}')
139     print(f'Algoritmo: {algorithm}')
140     print(f'Ataque: {attack}')
141     print(f'Dataset: {dataset}')
142     print(f'Numero de execucoes: {n_executions}')
143     total_time_init = time.time()
144
145     # Execucao do experimento
146     results = run_experiment(model_path, data, n_executions)
147
148     # Path para salvar os resultados
149     results_json_path =
150     ↪ f'../results/time_results/{platform}/{attack}/{algorithm}/{dataset}'
151     if not os.path.exists(results_json_path):
152         os.makedirs(results_json_path)
153
154     # Salva os resultados em um arquivo JSON

```

```

153     with open(f'{results_json_path}/time_results.json', 'w') as json_file:
154         json.dump(results, json_file, indent=4)
155
156     # Calcula o tempo total de execucao do experimento
157     total_time_end = time.time()
158     total_time = total_time_end - total_time_init
159     print('Tempo Total de Execucao do Experimento (s): ', total_time)
160

```

C.5 SRC/INJECTORS/

src/injectors/dos_injector.py

```

1  """
2  Este arquivo continha a logica de injecao sintetica de ataques DoS
3      no dataset de mensagens CAN obtido do veiculo do projeto IASE do LISHA.
4  O conteudo foi omitido por estar sujeito a um acordo de confidencialidade (NDA).
5  """
6

```

src/injectors/impersonation_injector.py

```

1  """
2  Este arquivo continha a logica de injecao sintetica de ataques de Impersonation
3      no dataset de mensagens CAN obtido do veiculo do projeto IASE do LISHA.
4  O conteudo foi omitido por estar sujeito a um acordo de confidencialidade (NDA).
5  """
6

```

C.6 SRC/PLOTS/

src/plots/et_plots.py

```

1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  # DoS Attack Plot
5  def plot_dos_attack(x, width, algorithms, sbc, dos_avg, dos_std):
6      fig, ax = plt.subplots(figsize=(10, 6))
7      bars = []
8      colors = ["#9467bd", "#8c564b", "#e377c2", "#7f7f7f"]
9      for i in range(3):
10         bars.append(
11             ax.bar(

```

```

12         x + i * width,
13         [dos_avg[j][i] for j in range(len(algorithms))],
14         width,
15         label=["1", "100", "1000"][i],
16         yerr=[dos_std[j][i] for j in range(len(algorithms))],
17         capsize=5,
18         color=colors[i],
19         edgecolor="black",
20         linewidth=1.2,
21     )
22 )
23
24 # Add error lines with increased width
25 for i in range(3):
26     ax.errorbar(
27         x + i * width,
28         [dos_avg[j][i] for j in range(len(algorithms))],
29         yerr=[dos_std[j][i] for j in range(len(algorithms))],
30         fmt="none",
31         ecolor="black",
32         elinewidth=2,
33         capsize=5,
34     )
35
36 ax.set_title(f"{sbc} - Classificação de Ataques DoS", fontsize=36)
37 ax.set_xticks(x + width)
38 ax.set_xticklabels(algorithms, fontsize=32)
39 ax.set_ylabel("Tempo Médio (ms)", fontsize=32)
40 ax.set_xlabel("Algoritmo", fontsize=32)
41 ax.legend(
42     title="Número de Instâncias", fontsize=32, title_fontsize=32, loc="upper
    ↪ center"
43 )
44 ax.axhline(1, color="red", linestyle="--", linewidth=2, label="y = 1")
45 ax.grid(True, which="both", linestyle="--", linewidth=0.5)
46 ax.tick_params(axis="both", which="major", labelsize=32)
47
48 plt.tight_layout(rect=[0, 0.03, 1, 0.95])
49 plt.show()
50
51
52 # Impersonation Attack Plot
53 def plot_impersonation_attack(
54     x, width, algorithms, sbc, impersonation_avg, impersonation_std
55 ):
56     fig, ax = plt.subplots(figsize=(10, 6))
57     bars = []
58     colors = ["#9467bd", "#8c564b", "#e377c2", "#7f7f7f"]
59     for i in range(3):

```

```

60     bars.append(
61         ax.bar(
62             x + i * width,
63             [impersonation_avg[j][i] for j in range(len(algorithms))],
64             width,
65             label=["1", "100", "1000"][i],
66             yerr=[impersonation_std[j][i] for j in range(len(algorithms))],
67             capsize=5,
68             color=colors[i],
69             edgecolor="black",
70             linewidth=1.2,
71         )
72     )
73
74     # Add error lines with increased width
75     for i in range(3):
76         ax.errorbar(
77             x + i * width,
78             [impersonation_avg[j][i] for j in range(len(algorithms))],
79             yerr=[impersonation_std[j][i] for j in range(len(algorithms))],
80             fmt="none",
81             ecolor="black",
82             elinewidth=2,
83             capsize=5,
84         )
85
86     ax.set_title(f"{sbc} - Classificação de Ataques de Impersonation", fontsize=36)
87     ax.set_xticks(x + width)
88     ax.set_xticklabels(algorithms, fontsize=32)
89     ax.set_ylabel("Tempo Médio (ms)", fontsize=32)
90     ax.set_xlabel("Algoritmo", fontsize=32)
91     ax.legend(
92         title="Número de Instâncias", fontsize=32, title_fontsize=32, loc="upper
93         ↪ center"
94     )
95     ax.axhline(1, color="red", linestyle="--", linewidth=2, label="y = 1")
96     ax.grid(True, which="both", linestyle="--", linewidth=0.5)
97     ax.tick_params(axis="both", which="major", labelsize=32)
98
99     plt.tight_layout(rect=[0, 0.03, 1, 0.95])
100     plt.show()
101
102     # LSTM Execution Time Plots
103     def plot_lstm_execution_time():
104         time_steps = [1, 10, 20, 50]
105         x = np.arange(len(time_steps))
106         width = 0.25
107

```

```

108     plot_lstm_dos(x, width, time_steps)
109     plot_lstm_imp(x, width, time_steps)
110
111
112     # LSTM DoS Plot
113     def plot_lstm_dos(x, width, time_steps):
114         lstm_dos_avg = [38.222, 37.807, 37.628, 37.601]
115         lstm_dos_std = [9.392, 9.689, 9.067, 8.931]
116
117         colors = ["#1f77b4", "#2ca02c", "#ff7f0e", "#d62728"]
118
119         fig, ax = plt.subplots(figsize=(12, 7))
120         bars = ax.bar(
121             x,
122             lstm_dos_avg,
123             width,
124             yerr=lstm_dos_std,
125             capsize=8,
126             color=colors,
127             edgecolor="black",
128             linewidth=1.2,
129         )
130
131         ax.set_title("Desempenho da LSTM na Detecção de Ataques DoS", fontsize=36)
132         ax.set_xticks(x)
133         ax.set_xticklabels(time_steps, fontsize=32)
134         ax.set_ylabel("Tempo Médio de Execução (ms)", fontsize=32)
135         ax.set_xlabel("time_steps", fontsize=32, fontstyle="italic", labelpad=10)
136         ax.axhline(1, color="red", linestyle="--", linewidth=2, label="Referência: y = 1")
137
138         for bar, avg, std in zip(bars, lstm_dos_avg, lstm_dos_std):
139             height = bar.get_height()
140             ax.annotate(
141                 f"{avg:.1f} ± {std:.1f}",
142                 xy=(bar.get_x() + bar.get_width() / 2, height),
143                 xytext=(0, 10),
144                 textcoords="offset points",
145                 ha="center",
146                 va="bottom",
147                 fontsize=24,
148                 fontweight="bold",
149                 color="black",
150             )
151
152         ax.grid(True, linestyle="--", linewidth=0.5, axis="y")
153         ax.tick_params(axis="both", labelsize=32)
154
155         ax.legend(fontsize=32, loc="lower right")
156

```

```

157     plt.tight_layout()
158     plt.show()
159
160
161     # LSTM Impersonation Plot
162     def plot_lstm_imp(x, width, time_steps):
163         lstm_imp_avg = [37.661, 37.711, 38.035, 37.749]
164         lstm_imp_std = [9.108, 9.0532, 9.787, 9.189]
165
166         colors = ["#9467bd", "#8c564b", "#e377c2", "#7f7f7f"]
167
168         fig, ax = plt.subplots(figsize=(12, 7))
169         bars = ax.bar(
170             x,
171             lstm_imp_avg,
172             width,
173             yerr=lstm_imp_std,
174             capsize=8,
175             color=colors,
176             edgecolor="black",
177             linewidth=1.2,
178         )
179
180         ax.set_title(
181             "Desempenho da LSTM na Detecção de Ataques de Impersonation", fontsize=36
182         )
183         ax.set_xticks(x)
184         ax.set_xticklabels(time_steps, fontsize=32)
185         ax.set_ylabel("Tempo Médio de Execução (ms)", fontsize=32)
186         ax.set_xlabel("time_steps", fontsize=32, fontstyle="italic", labelpad=10)
187         ax.axhline(1, color="red", linestyle="--", linewidth=2, label="Referência: y = 1")
188
189         for bar, avg, std in zip(bars, lstm_imp_avg, lstm_imp_std):
190             height = bar.get_height()
191             ax.annotate(
192                 f"{avg:.1f} ± {std:.1f}",
193                 xy=(bar.get_x() + bar.get_width() / 2, height),
194                 xytext=(0, 10),
195                 textcoords="offset points",
196                 ha="center",
197                 va="bottom",
198                 fontsize=24,
199                 fontweight="bold",
200                 color="black",
201             )
202
203         ax.grid(True, linestyle="--", linewidth=0.5, axis="y")
204         ax.tick_params(axis="both", labelsize=32)
205

```

```

206     ax.legend(fontsize=32, loc="lower right")
207
208     plt.tight_layout()
209     plt.show()
210
211
212 if __name__ == "__main__":
213     # Data
214     algorithms = ["KNN", "DT", "XGBoost"]
215
216     # Graphical settings
217     x = np.arange(len(algorithms))
218     width = 0.25
219
220     # Plots para a Raspberry Pi 3B
221     rasp_dos_avg = [
222         [11.181, 0.793, 0.808], # KNN
223         [2.674, 0.110, 0.012], # DT
224         [9.416, 0.109, 0.014], # XGBoost
225     ]
226     rasp_dos_std = [
227         [1.678, 0.029, 0.032], # KNN
228         [0.660, 0.023, 0.003], # DT
229         [0.893, 0.007, 0.001], # XGBoost
230     ]
231     rasp_impersonation_avg = [
232         [10.007, 0.721, 0.518], # KNN
233         [2.685, 0.114, 0.012], # DT
234         [9.174, 0.106, 0.013], # XGBoost
235     ]
236     rasp_impersonation_std = [
237         [0.897, 0.014, 0.007], # KNN
238         [0.849, 0.022, 0.003], # DT
239         [0.156, 0.003, 0.0002], # XGBoost
240     ]
241
242     plot_dos_attack(x, width, algorithms, "RP3B", rasp_dos_avg, rasp_dos_std)
243
244     plot_impersonation_attack(
245         x, width, algorithms, "RP3B", rasp_impersonation_avg, rasp_impersonation_std
246     )
247
248     # Plots para a VisionFive 2
249     vf2_dos_avg = [
250         [7.863, 0.637, 0.569], # KNN
251         [1.199, 0.067, 0.007], # DT
252         [6.42, 0.133, 0.016], # XGBoost
253     ]
254     vf2_dos_std = [

```

```

255         [0.332, 0.006, 0.002], # KNN
256         [0.173, 0.004, 0.001], # DT
257         [1.016, 0.011, 0.002], # XGBoost
258     ]
259     vf2_impersonation_avg = [
260         [7.361, 0.578, 0.412], # KNN
261         [1.155, 0.066, 0.007], # DT
262         [6.451, 0.132, 0.016], # XGBoost
263     ]
264     vf2_impersonation_std = [
265         [0.335, 0.004, 0.002], # KNN
266         [0.163, 0.005, 0.001], # DT
267         [1.184, 0.013, 0.002], # XGBoost
268     ]
269
270     plot_dos_attack(x, width, algorithms, "VF2", vf2_dos_avg, vf2_dos_std)
271     plot_impersonation_attack(
272         x, width, algorithms, "VF2", vf2_impersonation_avg, vf2_impersonation_std
273     )
274
275     # Plots da LSTM na RTX 3050
276     plot_lstm_execution_time()
277
278     print("Plots gerados com sucesso!")
279

```

C.7 SRC/RNN/

src/rnn/lstm.py

```

1
2 import os
3 import pandas as pd
4 import time
5 import numpy as np
6 import json
7 import matplotlib.pyplot as plt
8 import seaborn as sns
9 from pathlib import Path
10 from sklearn.model_selection import train_test_split
11 from sklearn.preprocessing import StandardScaler
12 from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
13
14 import tensorflow as tf
15 from tensorflow.keras.models import Sequential, load_model
16 from tensorflow.keras.layers import Dense, LSTM, Dropout, BatchNormalization
17 from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

```

```

18 from tensorflow.keras.utils import plot_model
19
20 # Verificacao da disponibilidade de GPU
21 print("TensorFlow version:", tf.__version__)
22 print("Verificando disponibilidade de GPUs:")
23 gpus = tf.config.list_physical_devices('GPU')
24 if gpus:
25     print(f"GPUs disponíveis: {len(gpus)}")
26     for gpu in gpus:
27         print(f" {gpu}")
28     print("TensorFlow utilizara GPU para treinamento")
29
30     # Configuracao de memoria da GPU
31     try:
32         for gpu in gpus:
33             tf.config.experimental.set_memory_growth(gpu, True)
34             print("Memoria de GPU configurada para crescimento dinâmico")
35     except RuntimeError as e:
36         print(f"Erro ao configurar memoria de GPU: {e}")
37 else:
38     print("Nenhuma GPU encontrada. TensorFlow utilizara CPU para treinamento")
39
40 # Configuracao dos caminhos
41 SRC_PATH = os.path.dirname(os.path.abspath(__file__))
42 ROOT_PATH = Path(SRC_PATH).parent.parent
43 DATASETS_PATH = os.path.join(ROOT_PATH, 'dataset')
44
45 # Configuracoes do modelo
46 MODEL_NAME = "LSTM"
47 ATTACK_NAME = "Impersonation_attack"
48 DATASET_NAME = "with_id.csv"
49 FILE_PATH = os.path.join(DATASETS_PATH, ATTACK_NAME, DATASET_NAME)
50
51 print(f"Carregando dados do arquivo: {FILE_PATH}")
52 data = pd.read_csv(FILE_PATH)
53
54 # Criacao dos diretorios para outputs
55 plots_dir = f'../results/{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/plots'
56 results_dir = f'../results/{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/results'
57 models_dir = f'../results/{ATTACK_NAME}/{MODEL_NAME}/{DATASET_NAME}/models'
58
59 for directory in [plots_dir, results_dir, models_dir]:
60     if not os.path.exists(directory):
61         os.makedirs(directory)
62
63 # Separacao de features e target
64 X = data.drop(f'{ATTACK_NAME}', axis=1)
65 y = data[f'{ATTACK_NAME}']
66

```

```

67 # Divisao de dados em treino e teste
68 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    ↪ random_state=42)
69
70 # Normalizacao dos dados (importante para redes neurais)
71 scaler = StandardScaler()
72 X_train_scaled = scaler.fit_transform(X_train)
73 X_test_scaled = scaler.transform(X_test)
74
75 # Primeiro, definimos o numero de time steps
76 TIME_STEPS = 50 # Usados: 1 || 10 || 20 || 50
77
78 def create_sequences(X, time_steps=TIME_STEPS):
79     Xs = []
80     for i in range(len(X) - time_steps + 1):
81         v = X[i:(i + time_steps)]
82         Xs.append(v)
83     return np.array(Xs)
84
85 # Criacao das sequencias para treino e teste
86 X_train_seq = create_sequences(X_train_scaled)
87 X_test_seq = create_sequences(X_test_scaled)
88
89 # Ajuste do tamanho de y_train e y_test para corresponder ao tamanho
90 # de X_train_seq e X_test_seq
91 # Valores a partir do indice time_steps - 1
92 y_train_seq = y_train.values[TIME_STEPS - 1:]
93 y_test_seq = y_test.values[TIME_STEPS - 1:]
94
95 print(f"Formato X_train_seq: {X_train_seq.shape}") # (samples, time_steps, features)
96 print(f"Formato y_train_seq: {y_train_seq.shape}") # (samples,)
97
98 # Construcao do modelo LSTM
99 def create_lstm_model(input_shape):
100     model = Sequential()
101
102     # Primeira camada LSTM
103     model.add(LSTM(128, input_shape=input_shape, return_sequences=True))
104     model.add(BatchNormalization())
105     model.add(Dropout(0.2))
106
107     # Segunda camada LSTM
108     model.add(LSTM(64, return_sequences=False))
109     model.add(BatchNormalization())
110     model.add(Dropout(0.2))
111
112     # Camada Densa
113     model.add(Dense(32, activation='relu'))
114     model.add(BatchNormalization())

```

```

115     model.add(Dropout(0.2))
116
117     # Saida
118     model.add(Dense(1, activation='sigmoid'))
119
120     model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
121
122     return model
123
124 # Criacao o modelo
125 input_shape = (X_train_seq.shape[1], X_train_seq.shape[2])
126 model = create_lstm_model(input_shape)
127 model.summary()
128
129 # Informacoes do dispositivo para qual o modelo esta compilado
130 print("\nInformações do dispositivo para qual o modelo esta compilado:")
131 print(f"Dispositivo de estrategia: {tf.distribute.get_strategy()}")
132 print(f"Dispositivos logicos do TensorFlow: {tf.config.list_logical_devices()}")
133 for device in tf.config.list_logical_devices():
134     print(f" {device.name} ({device.device_type})")
135
136 # Callbacks para early stopping e salvamento do melhor modelo
137 early_stopping = EarlyStopping(
138     monitor='val_loss',
139     patience=10,
140     restore_best_weights=True
141 )
142
143 model_checkpoint = ModelCheckpoint(
144     filepath=os.path.join(models_dir, f'best_model_{ATTACK_NAME}_{MODEL_NAME}.h5'),
145     monitor='val_loss',
146     save_best_only=True
147 )
148
149 # Treinamento do modelo
150 start_time_training = time.time()
151
152 # Verificacao de onde o modelo esta sendo executado
153 print("\nInformacoes sobre dispositivo de calculo:")
154 try:
155     with tf.device('/gpu:0'):
156         print("Tentando usar GPU: Dispositivo atual =", tf.device('/gpu:0'))
157         # Teste de operacao em GPU
158         a = tf.constant([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])
159         b = tf.constant([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])
160         c = tf.matmul(a, b)
161         print("Operacao de teste em GPU concluída com sucesso")
162 except RuntimeError as e:
163     print("Erro ao usar GPU:", e)

```

```

164     print("Usando CPU para treinamento")
165
166     print("\nIniciando treinamento...")
167     history = model.fit(
168         X_train_seq,
169         y_train_seq,
170         epochs=100,
171         batch_size=64,
172         validation_split=0.2,
173         callbacks=[early_stopping, model_checkpoint],
174         verbose=1
175     )
176
177     end_time_training = time.time()
178     training_time = end_time_training - start_time_training
179     print(f"\nTempo de treinamento: {training_time:.2f} segundos")
180
181     # Conversao do tempo para formato mais legivel
182     hours, remainder = divmod(training_time, 3600)
183     minutes, seconds = divmod(remainder, 60)
184     print(f"Tempo de treinamento: {int(hours)}h {int(minutes)}m {seconds:.2f}s")
185
186     # Avaliacao do modelo no conjunto de teste
187     start_time_testing = time.time()
188     y_pred_proba = model.predict(X_test_seq)
189     y_pred = (y_pred_proba > 0.5).astype(int).flatten()
190     end_time_testing = time.time()
191     testing_time = end_time_testing - start_time_testing
192     print(f"Tempo de teste: {testing_time:.2f} segundos")
193
194     # Metricas de avaliacao
195     accuracy = accuracy_score(y_test_seq, y_pred)
196     print(f"Acuracia no teste: {accuracy:.4f}")
197
198     conf_matrix = confusion_matrix(y_test_seq, y_pred)
199     report = classification_report(y_test_seq, y_pred, output_dict=True)
200     print("Classification Report:")
201     print(classification_report(y_test_seq, y_pred))
202
203
204     # Plots
205     # 1. Plot de confusion matrix
206     plt.figure(figsize=(8, 6))
207     sns.heatmap(conf_matrix, annot=True, fmt='d', cmap="Blues", cbar=False)
208     plt.xlabel('Predicted labels')
209     plt.ylabel('True labels')
210     plt.title('Confusion Matrix')
211     plt.savefig(f'{plots_dir}/confusion_matrix.png')
212     plt.close()

```

```

213
214 # 2. Plot de acuracia durante o treinamento
215 plt.figure(figsize=(10, 6))
216 plt.plot(history.history['accuracy'], label='Train Accuracy')
217 plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
218 plt.title('Model Accuracy')
219 plt.ylabel('Accuracy')
220 plt.xlabel('Epoch')
221 plt.legend(loc='lower right')
222 plt.grid(True)
223 plt.savefig(f'{plots_dir}/accuracy_history.png')
224 plt.close()
225
226 # 3. Plot de loss durante o treinamento
227 plt.figure(figsize=(10, 6))
228 plt.plot(history.history['loss'], label='Train Loss')
229 plt.plot(history.history['val_loss'], label='Validation Loss')
230 plt.title('Model Loss')
231 plt.ylabel('Loss')
232 plt.xlabel('Epoch')
233 plt.legend(loc='upper right')
234 plt.grid(True)
235 plt.savefig(f'{plots_dir}/loss_history.png')
236 plt.close()
237
238 # 4. Histograma das probabilidades previstas
239 plt.figure(figsize=(8, 6))
240 sns.histplot(y_pred_proba.flatten(), bins=10, kde=True, color='blue')
241 plt.xlabel('Predicted Probabilities')
242 plt.ylabel('Frequency')
243 plt.title('Histogram of Predicted Probabilities')
244 plt.savefig(f'{plots_dir}/predicted_probabilities_histogram.png')
245 plt.close()
246
247 # Salvar resultados em formato JSON
248 results = {
249     "test_accuracy": float(accuracy),
250     "classification_report": report,
251     "training_time_seconds": training_time,
252     "testing_time_seconds": testing_time,
253     "model_params": {
254         "lstm_units": [128, 64],
255         "dense_units": [32],
256         "dropout_rate": 0.2,
257         "batch_size": 64,
258         "early_stopping_patience": 10
259     }
260 }
261

```

```
262 with open(f'{results_dir}/results.json', 'w') as f:
263     json.dump(results, f, indent=4)
264
265 # Salvamento do modelo final (alem do melhor modelo salvo pelo callback)
266 model.save(f'{models_dir}/final_model_{ATTACK_NAME}_{MODEL_NAME}.h5')
267
268 print(f"Treinamento completo. Resultados e modelos salvos em {results_dir} e
    ↪ {models_dir}")
269
```

D ARTIGO NO PADRÃO SBC

Detecção de Ataques em Redes Intraveiculares CAN com Técnicas de Machine Learning

João Paulo Araujo Bonomo¹

¹Departamento de Informática e Estatística – Universidade Federal de Santa Catarina (UFSC) – Florianópolis, SC – Brasil

joao.paulo.bonomo@grad.ufsc.br

Abstract. *Modern vehicles increasingly rely on electronic components interconnected via the CAN bus, which, despite its efficiency, presents security vulnerabilities. This work investigates the use of machine learning algorithms to detect DoS and Impersonation attacks on CAN networks. The models were trained and tested on data from a real vehicle under both normal and attack conditions, and validated using a distinct dataset from a different manufacturer, provided by LISHA-UFSC. This validation aimed to demonstrate the models' ability to generalize to different in-vehicle networks. The results achieved up to 100% F1-score, surpassing previous studies. Furthermore, the models were deployed on two embedded platforms with different architectures, RISC-V and ARM, achieving execution times on the order of 1.2 milliseconds—compatible with CAN message periodicity—demonstrating the feasibility of an efficient, real-time IDS for vehicle protection against cyber-attacks.*

Resumo. *Veículos modernos dependem cada vez mais de componentes eletrônicos, interconectados pelo barramento CAN, que apesar de eficiente, apresenta vulnerabilidades de segurança. Este trabalho investiga o uso de algoritmos de machine learning para detectar ataques à rede CAN dos tipos DoS e Impersonation. Os modelos foram treinados e testados com dados de um veículo real em condições normais e sob ataque, e validados com um dataset distinto, oriundo de um veículo de outra fabricante, disponibilizado pelo LISHA-UFSC. A validação buscou demonstrar a capacidade de generalização dos modelos para diferentes redes intraveiculares. Os resultados apresentaram até 100% de F1-score, superando trabalhos anteriores. Além disso, os modelos foram embarcados em duas plataformas de arquiteturas distintas, RISC-V e ARM, apresentando tempos de execução da ordem de 1,2 milissegundos, compatíveis com a periodicidade do CAN, evidenciando a viabilidade de um IDS eficiente e em tempo real para proteção veicular contra ataques cibernéticos.*

1. Introdução

O estudo da segurança em redes veiculares baseadas no protocolo *Controller Area Network* (CAN) tem ganhado destaque devido ao aumento do número de componentes eletrônicos e da conectividade em veículos modernos. Essa transformação digital fez com que os automóveis passassem a depender de arquiteturas de rede para garantir a comunicação eficiente entre suas diversas partes. O barramento CAN possui características como baixa latência, transmissão em *broadcast* e arbitragem por prioridade.

No entanto, essas mesmas características também ampliam sua vulnerabilidade a ataques de segurança [Young et al. 2019]. Pesquisas anteriores demonstraram que a comunicação interna do veículo pode ser alvo de entidades maliciosas, que podem assumir o controle do automóvel ou provocar falhas, levando a situações perigosas. Miller et al. realizaram ataques ao sistema Uconnect da Jeep, o que levou à convocação de 1,4 milhão de veículos [Greenberg 2015]. Explorando vulnerabilidades do sistema, foi possível assumir remotamente o controle do carro. Outro exemplo é apresentado por Tindell, que revelou técnicas de roubo de veículos baseadas em intrusão via CAN [Tindell 2023].

Uma forma de aumentar a segurança das redes veiculares, como o barramento CAN, é por meio de técnicas de detecção de intrusão [Young et al. 2019]. Um Sistema de Detecção de Intrusão (IDS) pode ser usado para detectar e mitigar ataques em redes computacionais. Entretanto, abordagens tradicionais de segurança de redes nem sempre podem ser aplicadas diretamente ao contexto automotivo [Young et al. 2019]. Trabalhos anteriores exploram diversas técnicas para a implementação de IDSs em redes CAN, como o uso de hardware para medir variações de voltagem e identificar invasores [Schell and Kneib 2020], a análise da frequência das mensagens para inferir comportamentos suspeitos [Song et al. 2016], e o uso de modelos de *Machine Learning* (ML) [Alfardus and Rawat 2021] e híbridos (ML com análise estática) para detectar anomalias [Purohit and Govindarasu 2022].

Contudo, nenhum desses trabalhos com ML avalia a viabilidade de seus modelos em ambientes embarcados com restrições de tempo real, nem explora modelos não supervisionados, deixando uma lacuna que este trabalho busca preencher. Além disso, testamos uma gama mais ampla de técnicas de ML em *datasets* já existentes e também aplicamos os modelos em um novo conjunto de mensagens CAN, coletado a partir de um veículo real em cenários reais, o que ainda não havia sido explorado na literatura. Em resumo, as principais contribuições deste trabalho são:

- Desenvolvimento de seis modelos de ML para detectar intrusões no barramento CAN, testando-os em dois conjuntos de dados: um proveniente de um trabalho relacionado e outro coletado de um veículo real, com os ataques sintetizados posteriormente;
- Por meio de análise experimental, alcançamos resultados superiores aos registrados em estudos anteriores: 100% e 99,97% de F1-Score para os ataques de DoS e *Impersonation*, respectivamente;
- Implantação dos modelos em duas plataformas embarcadas modernas, uma VisionFive 2 e uma Raspberry Pi 3 Model B, coletando métricas como tempos de treinamento e classificação, demonstrando a viabilidade do uso de um IDS em ambiente real.

2. Revisão Bibliográfica

O esforço da indústria automotiva em padronizar a comunicação entre diferentes componentes do veículo, motivado pela crescente complexidade da comunicação interna, levou à criação do protocolo CAN, definido pela norma ISO 11898-1 [for Standardization 2015]. O barramento CAN é considerado o sistema nervoso central de um carro, permitindo que componentes separados troquem informações e coordenem funções para controlar diversas partes do veículo.

O protocolo especifica quatro tipos de mensagens CAN: quadro de dados (*data frame*), quadro remoto (*remote frame*), quadro de erro (*error frame*) e quadro de sobrecarga (*overload frame*). O *data frame* é o tipo padrão de mensagem. Um transmissor pode emitir um *remote frame* para solicitar dados de um nó específico. Um *error frame* pode ser transmitido por qualquer nó que detecte um erro. Já os *overload frames* introduzem atrasos adicionais entre quadros de dados ou quadros remotos. O quadro de dados do CAN é composto por sete campos, conforme ilustrado na Figura 1:

1. **Start of Frame (SOF):** Indica o início da mensagem.
2. **Arbitration Field:** Contém o identificador e o bit RTR (Remote Transmission Request), responsável por determinar a prioridade entre mensagens simultâneas.
3. **Control Field:** Inclui o bit IDE (*Identifier Extension*), o bit reservado (*r0*) e o código de comprimento de dados (DLC), que define o número de bytes.
4. **Data Field:** Contém até 8 bytes de dados.
5. **CRC Field:** Inclui a sequência de verificação cíclica (CRC) e seu delimitador, usados para detecção de erros.
6. **ACK Field:** Composto pelo *slot* e delimitador de reconhecimento (*acknowledge*), onde os receptores indicam o recebimento da mensagem.
7. **End of Frame (EOF):** Marca o fim da mensagem.

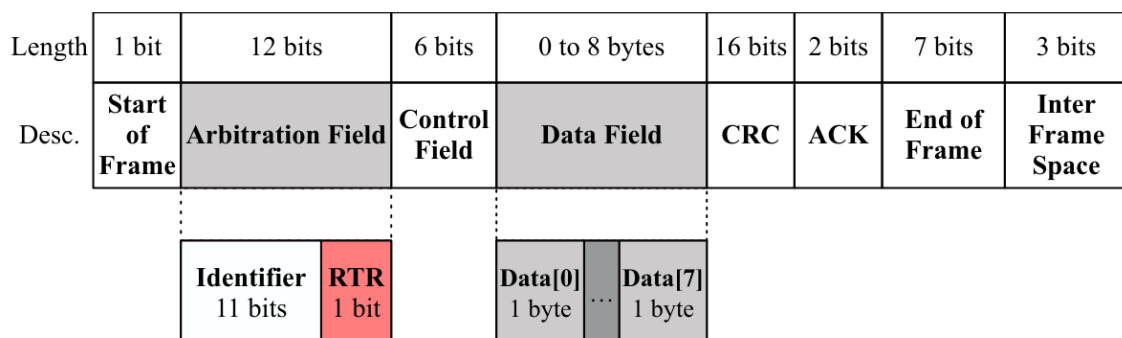


Figure 1. Estrutura de uma mensagem CAN [Lee et al. 2017].

Na rede CAN, todos os nós que desejam transmitir se sincronizam com o início da mensagem (SOF) e tentam transmitir simultaneamente. Um esquema de arbitragem decide qual nó terá prioridade na transmissão. Esse esquema garante que a mensagem com maior prioridade (menor valor numérico do identificador) seja a transmitida. A rede opera segundo o protocolo de acesso múltiplo por escuta de portadora com detecção de colisão (CSMA/CD). Durante a transmissão, cada nó monitora o estado do barramento. Se um nó transmitir um bit recessivo e ler um bit dominante, detecta uma colisão e interrompe imediatamente a transmissão.

Fabricantes de automóveis têm buscado oferecer mais conforto e tecnologia aos seus clientes, integrando sistemas como *infotainment* e controle digital de componentes críticos, como suspensão, motor e transmissão. Com isso, cresce o número de dispositivos conectados à rede CAN, o que aumenta sua exposição e os pontos passíveis de ataque. Embora o simples acesso ao meio físico da rede CAN não represente uma ameaça imediata, ele permite que atacantes explorem vulnerabilidades e conduzam ataques potencialmente perigosos.

Apesar de a literatura descrever outros tipos ataques, este trabalho foca nos ataques de Negação de Serviço (Denial of Service - DoS) e de *Impersonation* [Lee et al. 2017]:

1. **Ataque DoS:** Um invasor pode inundar o sistema com mensagens de alta prioridade, congestionando a rede e impedindo a comunicação legítima.
2. **Ataque de *Impersonation*:** Um invasor envia mensagens usando o identificador de um nó legítimo, simulando seu comportamento e respondendo a solicitações como se fosse ele.

Diversos estudos documentam os efeitos desses ataques. Alguns autores demonstraram vulnerabilidades em módulos críticos como o painel de instrumentos, o módulo de controle da carroceria, os freios e o motor. Foram capazes de manipular a ignição, o RPM do motor e até desativar os freios com o veículo em movimento [Koscher et al. 2010]. Outros aplicaram um ataque DoS saturando a rede com mensagens, levando ao transbordo dos *buffers* e à paralisação da comunicação [Mukherjee et al. 2016]. De forma semelhante, Poutro grupo de pesquisadores realizou um ataque DoS furtivo usando um nó malicioso que injetava quadros de erro, forçando nós transmissores ao estado de *bus-off*, impossibilitando-os de enviar mensagens [Palanca et al. 2017].

Portanto, um sistema capaz de detectar essas intrusões em tempo real e alertar uma unidade central ou até reagir automaticamente é essencial para a segurança veicular. A próxima subseção revisa os principais tipos de IDS propostos na literatura.

2.1. Trabalhos Correlatos

Um dos estudos mais relevantes da área foi conduzido por Lee et al., que propuseram um método baseado na análise da razão de deslocamento e dos intervalos de tempo entre mensagens de solicitação e resposta no barramento CAN. Os autores observaram que, em condições normais, os nós da rede apresentam padrões temporais estáveis, que sofrem variações significativas durante ataques. O método, portanto, monitora essas variações para detectar intrusões de forma rápida e precisa, sem necessidade de alterações no protocolo. Foram detectados com sucesso diversos tipos de ataque, como DoS, Fuzzy e *Impersonation*, além da identificação de mensagens injetadas e ECUs comprometidas. Os conjuntos de dados utilizados, obtidos a partir de um veículo real durante ataques simulados, foram disponibilizados pelo HCRL e tornaram-se referência em diversos estudos posteriores — incluindo este [Lee et al. 2017].

Song et al. também propuseram um IDS eficiente, confiável e de baixo custo computacional, com base na análise dos intervalos de tempo entre mensagens CAN. Seu trabalho identificou ataques de injeção dupla, quádrupla e de desacoplamento de velocidade, além de simular ataques DoS com aumento da taxa de mensagens de 2000 para cerca de 6000 por segundo. Para detecção, definiram um limiar de 0,2 milissegundos e aplicaram um sistema de pontuação que reduziu falsos positivos ao penalizar sequências de mensagens fora do padrão. Com limiares mais altos, alcançaram 100% de precisão e detecção em menos de 1 milissegundo, o que é crucial para prevenir falhas graves em redes CAN [Song et al. 2016].

Alshammari et al. exploraram o uso de algoritmos de aprendizado supervisionado, como KNN e SVM, para detecção de intrusões em redes CAN. Utilizaram dois conjuntos de dados do HCRL, o “DoS *dataset*” e o “Fuzzy *dataset*”, e obtiveram precisão superior a 96%. No entanto, não aplicaram técnicas para evitar *overfitting* e não reportaram o

tempo necessário para classificar uma instância, limitando a avaliação da aplicabilidade prática [Alshammari et al. 2018]. Em linha semelhante, Gundu et al. também utilizaram KNN, mas exploraram adicionalmente os modelos *Random Forest* (RF) e XGBoost, buscando melhorar a performance da classificação [Gundu and Maleki 2022].

Purohit et al. propuseram três modelos baseados em ML para detecção de intrusões: *Decision Tree* (DT), RF e XGBoost. O destaque desse estudo está na medição dos tempos de treinamento e teste dos modelos — uma métrica rara nos trabalhos da área. No entanto, o tempo de classificação de uma única amostra, essencial para implementação embarcada, não foi mensurado [Purohit and Govindarasu 2022].

Por fim, Anjum et al. utilizaram o XGBoost para detectar injeções maliciosas e falhas de funcionamento em redes CAN. Diferentemente de outros trabalhos, criaram modelos específicos para cada tipo de ataque, como DoS, Fuzzy e *spoofing* de RPM. Avaliando os dados do HCRL, obtiveram *F1-score* de 94,74% e 96,49% para ataques de *flooding* e mau funcionamento, respectivamente. Ainda assim, não analisaram o tempo de execução dos modelos, deixando em aberto a viabilidade para uso em tempo real [Anjum et al. 2022].

3. Metodologia

Para o desenvolvimento deste trabalho, foram utilizados dois conjuntos de dados contendo mensagens CAN capturadas em ambientes automotivos reais. O primeiro é o conjunto disponibilizado por Lee et al., amplamente utilizado na literatura, que inclui cerca de 2,4 milhões de mensagens saudáveis e mais de 2,2 milhões de mensagens com ataques dos tipos DoS, Fuzzy e *Impersonation* [Lee et al. 2017]. Já o segundo conjunto de dados foi coletado com o auxílio do hardware IASE-Board, desenvolvido pelo laboratório LISHA (UFSC), em um veículo Renault Oroch 2021 em funcionamento. As mensagens CAN foram registradas em cenário real e, posteriormente, ataques de DoS e *Impersonation* foram simulados por meio de pós-processamento, replicando a metodologia usada no dataset de Lee.

O trabalho se baseou fortemente na metodologia CRISP-DM na organização das etapas de coleta, análise, normalização e demais tarefas de pré-processamento dos dados, criação e avaliação dos modelos e implantação, que no escopo deste trabalho consiste em embarcar os modelos nas duas SBCs.

Cada amostra dos conjuntos de dados contém um identificador da ECU (ID), *timestamp*, *payload* e um rótulo indicando se a mensagem é legítima ou maliciosa. Essas informações foram usadas para treinar e avaliar seis algoritmos de ML, sendo quatro classificadores clássicos e um clusterizador, além de uma Rede Neural Recorrente (RNN): *K-Nearest Neighbors* (KNN), *Decision Tree* (DT), *Random Forest* (RF), *XGBoost*, *K-Means* (não supervisionado) e, por fim, a rede LSTM. A implementação foi realizada em Python, com a biblioteca Scikit-Learn (versão 1.1.3), exceto para o XGBoost, que utilizou a versão 2.0.3, e a rede LSTM, que usou o Tensorflow. O KNN foi configurado com cinco vizinhos, buscando mitigar o impacto de desbalanceamento de classes. A rede LSTM foi treinada com quatro diferentes valores para o parâmetro de *time_steps*: 1, 10, 20 e 50. Os demais modelos utilizaram parâmetros padrão.

Todos os modelos foram treinados com 70% dos dados e testados com os 30% restantes, seguindo práticas comuns em tarefas de mineração de da-

dos [Purohit and Govindarasu 2022, Canal et al. 2023]. A avaliação considerou não apenas métricas de desempenho como *F1-score*, mas também o tempo de execução, com foco na viabilidade de implantação em sistemas embarcados.

4. Resultados

Para avaliar o desempenho dos modelos, foram utilizadas as métricas de acurácia, precisão, *recall* e *F1-score*, conforme os principais trabalhos da área. Essas métricas são adequadas para cenários com classes desbalanceadas ou com diferentes custos de erro, permitindo uma análise mais precisa da performance dos classificadores [Anjum et al. 2022]. Além disso, facilitam a comparação com estudos que usam métricas equivalentes, como a taxa de falsos alarmes (FAR), já que é possível derivar uma a partir da outra [Purohit and Govindarasu 2022]. A principal métrica adotada foi o *F1-score*, a média harmônica entre precisão e *recall* (sensibilidade).

As Tabelas 1 e 2 mostram os resultados de *F1-score* dos ataques DoS e *Impersonation* para os algoritmos proposto neste trabalho. Diferente dos demais estudos, que treinam um único modelo para todos os tipos de ataque, neste trabalho os modelos foram treinados individualmente para cada ataque, seguindo a abordagem de Anjum et al. [Anjum et al. 2022]. Por fim, os resultados comparativos foram extraídos diretamente dos artigos, sem replicação dos experimentos originais.

4.1. Ataque de DoS

A Tabela 1 apresenta os resultados da detecção de ataques de DoS com e sem a utilização da *feature* ‘ID’, que identifica a ECU remetente da mensagem. Mesmo sem o ‘ID’, os modelos tiveram bom desempenho, mas os resultados foram consistentemente melhores quando essa informação foi usada, já que os ataques DoS foram injetados com um identificador fixo igual a zero.

Algoritmo	(Este Trabalho, 2025)		A	B	C	D
	com ID	sem ID				
XGBoost	100%	96,39%	-	92%	97,2%	94,74%
LSTM	100%	96,36%	-	-	-	-
KNN	99,97%	96,34%	93,5%	70%	-	-
RF	100%	95,02%	-	94%	94,3%	-
DT	100%	94,35%	-	-	96,1%	-
K-Means	56,37%	56,37%	-	-	-	-

Table 1. Comparação dos resultados de *F1-score* para detecção de ataques de DoS. Trabalhos utilizados para comparação: A [Alshammari et al. 2018], B [Gundu and Maleki 2022], C [Purohit and Govindarasu 2022], D [Anjum et al. 2022].

Os modelos supervisionados KNN, DT, RF e XGBoost, e a rede LSTM apresentaram os melhores resultados, alcançando até 100% de *F1-score* com o uso do ‘ID’. Mesmo sem essa *feature*, mantiveram desempenho acima de 94%, superando trabalhos anteriores [Alshammari et al. 2018, Gundu and Maleki 2022, Purohit and Govindarasu 2022, Anjum et al. 2022]. Por outro lado, o modelo não supervisionado *K-Means* teve desempenho inferior (*F1-score* médio de 56,37%), indicando que algoritmos supervisionados são mais adequados para essa tarefa.

Ao testar os modelos no segundo conjunto de dados, coletado pelo IASE, os resultados foram igualmente positivos: KNN, DT, RF e XGBoost atingiram até 100% de *F1-score* e acurácia, com validação cruzada também próxima desse valor, demonstrando alta capacidade de generalização. A LSTM, por sua vez, teve desempenho inferior, com *F1-score* máximo de 77% mesmo com a presença do ‘ID’, sendo ainda mais afetada pela remoção dessa *feature*, um padrão observado em todos os modelos.

Além da acurácia, é essencial que os modelos atendam a requisitos de tempo em sistemas embarcados. A avaliação de tempo foi feita nas SBCs VisionFive 2 e Raspberry Pi 3, comparando o tempo médio de classificação de mensagens CAN para os ataques de DoS. Na VisionFive 2, a DT foi o modelo mais eficiente, com tempo médio de classificação abaixo de 1,2ms por mensagem, próximo à periodicidade típica do barramento CAN (1ms). KNN e XGBoost também apresentaram bons resultados, com tempos reduzidos, especialmente quando múltiplas mensagens foram processadas em lote, como representado no gráfico da Figura 2.

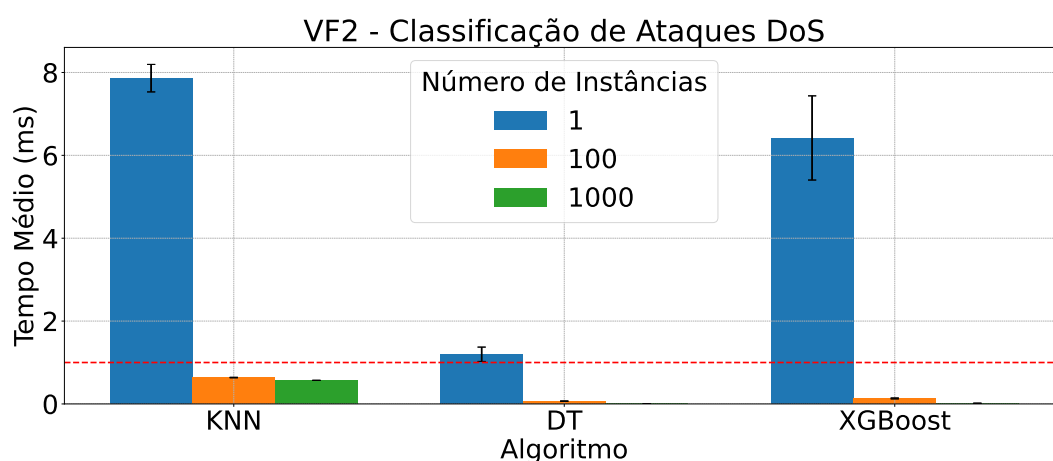


Figure 2. Tempo médio de classificação de uma mensagem como normal ou ataque DoS em milissegundos (ms) na SBC VisionFive 2. A legenda da figura descreve quantas instâncias foram fornecidas ao modelo.

Como visto na Figura 3, para a Raspberry Pi 3 Model B os tempos foram mais altos em comparação à outra SBC, mas ainda aceitáveis para a maioria dos modelos. A DT continuou se destacando, mesmo com o dobro do tempo observado na VisionFive 2. A redução do tempo médio ao processar lotes maiores foi evidente em ambas as SBCs, efeito atribuído a otimizações como paralelismo e vetorização durante o processamento em lote.

Em contraste com os classificadores mais simples, a rede LSTM teve tempo de treino e classificação significativamente altos, mesmo utilizando um computador de mesa com GPU. Com tempo de classificação até 30 vezes maior que o da DT em hardware inferior, sua implantação embarcada foi descartada, considerando tanto a complexidade quanto a menor generalização observada nos testes.

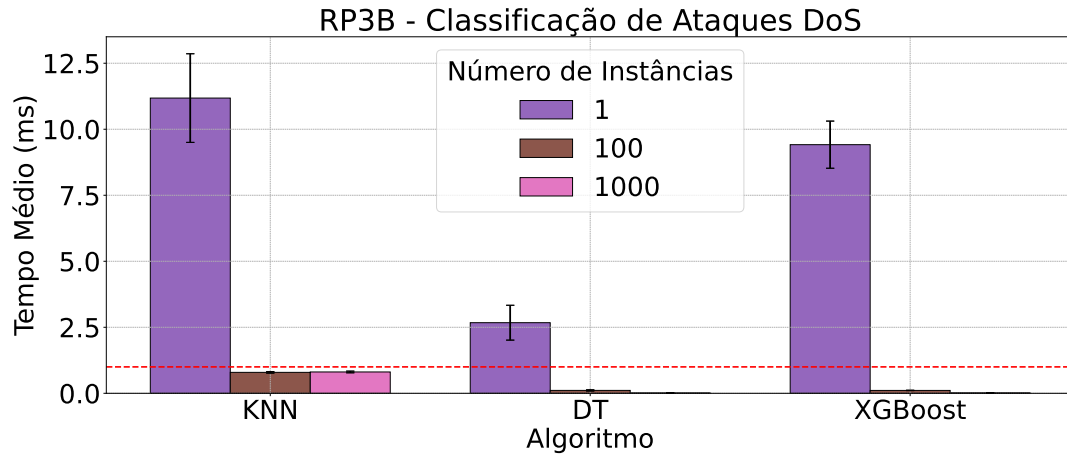


Figure 3. Tempo médio de classificação de uma mensagem como normal ou ataque DoS em milissegundos (ms) na Raspberry Pi 3 Model B. A legenda da figura descreve quantas instâncias foram fornecidas ao modelo.

4.2. Ataque de *Impersonation*

Nos testes de detecção de ataques de *Impersonation*, a rede LSTM e o KNN obtiveram um *F1-score* de 99,97%, como visto na Tabela 2. Os demais modelos supervisionados clássicos (DT, RF e XGBoost) alcançaram 99,51%. Esses resultados superaram os dos quatro trabalhos comparativos [Alshammari et al. 2018, Gundu and Maleki 2022, Purohit and Govindarasu 2022, Anjum et al. 2022], em parte porque alguns desses estudos utilizaram o mesmo modelo para diferentes tipos de ataque ou avaliaram apenas um algoritmo. Diferentemente do cenário DoS, aqui os modelos mantiveram um *F1-score* muito parecido com ou sem o uso do atributo 'ID', mostrando boa independência dessa informação na detecção de ataques de *Impersonation*.

Algoritmo	A	B	C	D	E
KNN	99,97%	93,5%	70%	-	-
LSTM	99,97%	-	-	-	-
DT	99,51%	-	-	96,1%	-
RF	99,51%	-	94%	94,3%	-
XGBoost	99,51%	-	92%	97,2%	96,49%
K-Means	49,29%	-	-	-	-

Table 2. Comparação dos resultados de *F1-score* para detecção de ataques de *Impersonation*. A: Este Trabalho (2025); B: [Alshammari et al. 2018]; C: [Gundu and Maleki 2022]; D: [Purohit and Govindarasu 2022]; E: [Anjum et al. 2022].

Na avaliação de generalização, todos os modelos supervisionados obtiveram médias superiores a 99% na validação cruzada, com destaque para o XGBoost, que manteve acurácia alta e estável em todos os tamanhos de conjunto de treino, sem sinais de *overfitting*. Esses resultados indicam forte capacidade de generalização dos modelos,

inclusive para dados oriundos de redes distintas, como observado nos 86% de acurácia obtidos de média desses modelos no conjunto de dados do IASE, mesmo sem o retreinamento.

Ao analisarmos o comportamento dos modelos para o ataque de *Impersonation*, há uma semelhança com os resultados obtidos para os ataques de DoS. Na VisionFive 2, a DT novamente foi o modelo com melhor desempenho, classificando mensagens em tempo inferior a 1,2 ms, como visto na Figura 4. Novamente, o KNN e XGBoost também apresentaram tempos baixos e estáveis, com pequenas variações entre execuções. Para a Raspberry Pi 3, os tempos também aumentaram bastante, mas seguiram o mesmo padrão observado para o DoS, como visto na Figura 5. Todos os modelos clássicos mantiveram desempenho adequado para execução embarcada, especialmente ao processar múltiplas instâncias por vez.

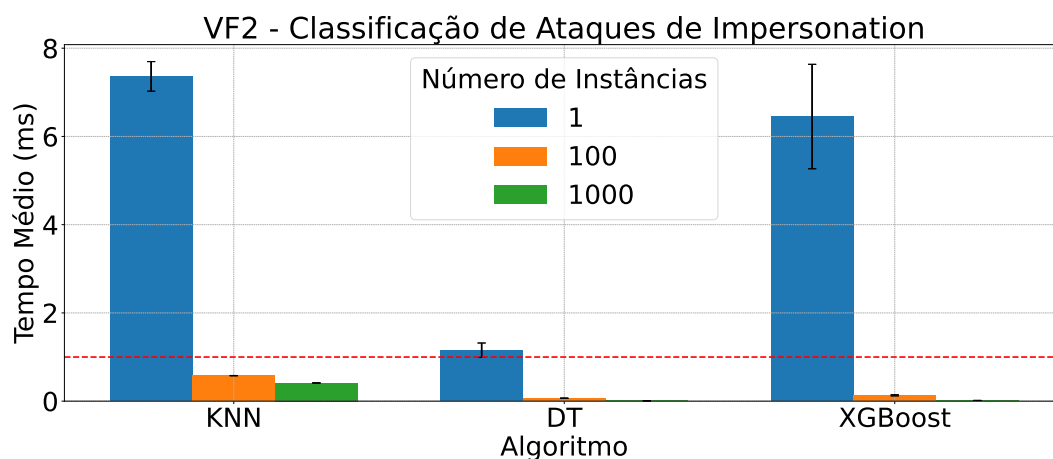


Figure 4. Tempo médio de classificação de uma mensagem como normal ou ataque de *impersonation* em milissegundos (ms) na SBC VisionFive 2. A legenda da figura descreve quantas instâncias foram fornecidas ao modelo.

5. Conclusão

Este trabalho avaliou o uso de algoritmos de aprendizado de máquina para detectar ataques DoS e Impersonation em redes veiculares baseadas no protocolo CAN. Por meio de uma abordagem fundamentada na metodologia CRISP-DM, foram desenvolvidos e validados modelos que apresentaram desempenho expressivo, com até 100% de *F1-score* em alguns cenários, superando estudos anteriores. A comparação entre diferentes algoritmos, a implantação em plataformas embarcadas com arquiteturas distintas e a análise do impacto de *features* relevantes demonstraram a viabilidade de um IDS eficiente e em tempo real para aplicações veiculares. Apesar das limitações observadas, como a complexidade de alguns modelos e a dificuldade de generalização entre veículos distintos, os resultados obtidos reforçam o potencial das técnicas de aprendizado de máquina como ferramentas de segurança em ambientes automotivos.

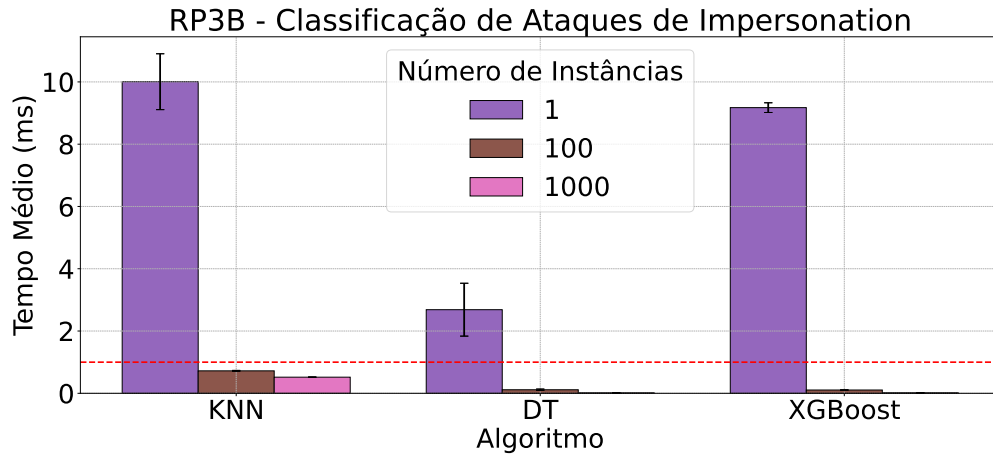


Figure 5. Tempo médio de classificação de uma mensagem como normal ou ataque de *Impersonation* em milissegundos (ms) na Raspberry Pi 3 Model B. A legenda da figura descreve quantas instâncias foram fornecidas ao modelo.

Como continuidade deste trabalho, propõe-se a validação prática dos modelos em veículos reais, com a injeção controlada de ataques para simular cenários reais de intrusão. Além disso, sugere-se a ampliação da detecção para outros tipos de ataques, como Fuzzy, bem como a otimização dos modelos desenvolvidos com a implementação em linguagens mais eficientes, como C++, visando aplicações embarcadas em sistemas críticos de tempo real. Essas iniciativas podem fortalecer a robustez, a generalização e a aplicabilidade prática das soluções de segurança propostas para redes intraveiculares baseadas no protocolo CAN.

References

- Alfardus, A. and Rawat, D. B. (2021). Intrusion detection system for can bus in-vehicle network based on machine learning algorithms. In *IEEE 12th UEMCON*, pages 0944–0949.
- Alshammari, A., Zohdy, M. A., Debnath, D., and Corser, G. (2018). Classification approach for intrusion detection in vehicle systems. *Wireless Engineering and Technology*, 9(4):79–94.
- Anjum, A., Agbaje, P., Hounsinnou, S., and Olufowobi, H. (2022). In-vehicle network anomaly detection using extreme gradient boosting machine. In *11th MECO*, pages 1–6.
- Canal, R., Riffel, F. K., Bonomo, J. P. A., de Carvalho, R. S., and Gracioli, G. (2023). Misfire detection in combustion engines using machine learning techniques. In *XIII SBESC*, pages 1–6.
- for Standardization, I. O. (2015). Road vehicles - controller area network (can) - part 1: Data link layer and physical signalling. Standard, International Organization for Standardization, Geneva, CH.

- Greenberg, A. (2015). Hackers remotely kill a jeep on the highway—with me in it. Acesso em: 04 dez. 2024.
- Gundu, R. and Maleki, M. (2022). Securing can bus in connected and autonomous vehicles using supervised machine learning approaches. In *IEEE eIT*, pages 042–046.
- Koscher, K., Czeskis, A., Roesner, F., Patel, S., Kohno, T., Checkoway, S., McCoy, D., Kantor, B., Anderson, D., Shacham, H., and Savage, S. (2010). Experimental security analysis of a modern automobile. In *IEEE Symposium on Security and Privacy*, pages 447–462.
- Lee, H., Jeong, S. H., and Kim, H. K. (2017). Otids: A novel intrusion detection system for in-vehicle network by using remote frame. In *15th Annual Conference on Privacy, Security and Trust (PST)*, pages 57–5709.
- Mukherjee, S., Shirazi, H., Ray, I., Daily, J., and Gamble, R. (2016). Practical dos attacks on embedded networks in commercial vehicles. In *12th International Conference (ICISS 2016)*, pages 23–42. Springer.
- Palanca, A., Evenchick, E., Maggi, F., and Zanero, S. (2017). A stealth, selective, link-layer denial-of-service attack against automotive networks. In *14th International Conference (DIMVA 2017)*, pages 185–206. Springer.
- Purohit, S. and Govindarasu, M. (2022). ML-based anomaly detection for intra-vehicular can-bus networks. In *IEEE CSR*, pages 233–238. IEEE.
- Schell, O. and Kneib, M. (2020). Valid: Voltage-based lightweight intrusion detection for the controller area network. In *IEEE 19th TrustCom*, pages 225–232.
- Song, H. M., Kim, H. R., and Kim, H. K. (2016). Intrusion detection system based on the analysis of time intervals of can messages for in-vehicle network. In *2016 ICOIN*, pages 63–68.
- Tindell, K. (2023). Car theft with can injection. <https://kentindell.github.io/2023/04/03/can-injection/>. Acessado: 02-11-2024.
- Young, C., Zambreno, J., Olufowobi, H., and Bloom, G. (2019). Survey of automotive controller area network intrusion detection systems. *IEEE Design & Test*, 36(6):48–55.